National College of Ireland

Project Submission Sheet

Student Name:	Eoin Fitzsimons		
Student ID:	X23151374		
Programme:	Computing	**Year:**	4
Module:	Final Project		
Lecturer:	Frances Sheridan		
Submission Due Date:	13/05/2026		
Project Title:	Momentum		
Word Count:			

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard supplied by the Library. To use other

author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Signature: Eoin Fitzsimons

Date: 12/12/2025

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).

2. Projects should be submitted to your Programme Coordinator.

3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.

4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**

5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**

6. Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

Final project

Momentum

Your Name/Student Number	Course	Date
Eoin Fitzsimons	Computing	12/12/2025

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click *here*.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
GPT-5 mini	Free version with github vscode	*gpt-5-mini Model | OpenAI API*
Claude Sonnet 4.5	Premium version with github vscode – free in limited usage	*Introducing Claude Sonnet 4.5 \ Anthropic*

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

GPT 5 mini	
To analyse the project and the rubric to see what is missing at the time	
I paste full rubric in and ask how what I currently have relates to it.	Project Analysis — Rubric Coverage and Evidence This document summarises how the current repository maps to the assessment rubric you provided. For each rubric area I list the evidence found in the repo, a short assessment of coverage (Covered / Partial / Missing), and recommended next steps that raise the quality of evidence for assessors.

	1. Executive summary - Purpose: give assessors a one-page view of aims, methods, principal results and conclusions. - Evidence in repo: `Planning/Technical Report/Contents.md`, `Planning/Technical Report/Introduction/introduction.md`, `README.md`. - Assessment: Partial — the README and Introduction provide context, but there is no single polished one-page Executive Summary file suitable for marking. - Recommendation: add `Planning/Technical Report/Executive_Summary.md` (one page) summarising aims, approach, key results and main conclusions. 2. Project Proposal (15%) - Evidence: - `Planning/Technical Report/Proposal/Proposal Draft.md` - Planning notes in `Planning/*` and `README.md` background sections. - Assessment: Covered (draft exists). The proposal contains research, motivation and scope; may need refinement for final submission. - Recommendation: produce a final version (polished language, clear aims/objectives, one-page summary for assessors) and place it as `Planning/Technical Report/Proposal/Proposal_Final.md`. 3. Communication Skills (10%) - Evidence: - `Planning/Midpoint Presentation/presentation_plan.md` (slide plan, demo script) - Diagrams in `Planning/Technical Report/diagrams/` and `Planning/Design and Architecture/` (draw.io and exports)

	- Assessment: Partial — planning and diagrams are strong, but the final slide deck and a sample demo recording are not present in the repo. - Recommendation: add `Planning/Midpoint Presentation/Slides.pdf` and a short demo video `Planning/Midpoint Presentation/demo_short.mp4`. Also add speaker notes and a short rehearsal checklist. 4. Project Analysis & Planning (10%) - Evidence: - `Planning/` folder: schedules, `update_dates*.ps1`, milestone notes, design diagrams and use case planning. - `Planning/Technical Report/Contents.md` and `introduction.md` show planned structure and mapping. - Assessment: Covered — conceptual diagrams, scheduling and milestone files exist. - Recommendation: add a short trace that links specific high-risk tasks to mitigation steps (e.g., "DNF tuning" → diagrams → tuning plan → tests) to make the mapping explicit. 5. Requirements Engineering (40%) - Evidence: - Use cases: `Planning/UseCases/*`, exported diagrams in `Planning/UseCases/exports/`. - Data model: `Momentum/Python/database/models.py` (dataclasses and `SCHEMA_SQL`). - `Planning/Technical Report/Contents.md` enumerates functional and non-functional sections. - Assessment: Strong coverage — documented use cases, data schema and detailed TOC are present. This area is well supported. - Recommendation: create a Requirements Traceability Matrix (RTM) file: `Planning/Technical Report/Requirements_Traceability.md` mapping requirement id → implementing module(s) → test(s) → status.

	6. Prototype (25%) - Evidence: - GUI and run scripts: `Momentum/Python/GUI/gui_game.py`, `Momentum/Python/GUI/launcher.py`, `Momentum/Python/GUI/gui_race_visual.py`. - Simulation engine: `Momentum/Python/RaceLogic/realtime_race_engine.py` and `simulation.py`. - DB and seed scripts: `scripts/generate_and_seed_db.py` and related scripts. - Tests: `Momentum/Python/tests/*` and `archive/basic-race-legacy/tests/*`. - Assessment: Covered — runnable prototype exists, but a short reproducible demo asset (screencast, test-run logs, or screenshots) would make it easier for an assessor to verify functionality quickly. - Recommendation: add `Planning/Midpoint Presentation/demo_evidence.md` with explicit run commands and expected outputs; include a screenshot and short MP4 demo. 7. Testing and Evaluation (supporting material) - Evidence: - Unit and integration tests in `Momentum/Python/tests/` and legacy archive. - `Planning/Testing/Plan.md` describes manual/automation testing approaches. - Playtesting notes: `Reflections/Reflection December.md`, logs in `logs/`. - Assessment: Partial — test code exists, and plans are documented. However, concise test result summaries, graphs or metric tables are not yet collated into a results appendix. - Recommendation: add `Planning/Technical Report/Test_Results.md` summarising key metrics (e.g., DNF

	counts per lap pre/post tuning, unit test pass rates). Export simple plots or tables (CSV + PNG) into `Planning/Technical Report/assets/`. 8. Reflective journals and supplementary materials - Evidence: `Reflections/Reflection December.md` and other notes in `Planning/`. - Assessment: Covered — reflective material exists for appendices. - Recommendation: ensure each reflection references the date and work performed and add them as appendices in the final report. 9. Diagrams and architecture - Evidence: draw.io/exports in `Planning/Technical Report/diagrams/` and `Planning/Design and Architecture/`. - Assessment: Covered — diagrams are present in editable forms and exports. - Recommendation: embed selected diagrams directly into the final `Technical Report` markdown files to ensure assessors can view them without opening draw.io. Actionable next steps (priority order) 1. Create a short one-page Executive Summary: `Planning/Technical Report/Executive_Summary.md` (high impact for assessors). 2. Add final slide deck and a 60–90s screencast of the Quick Race demo to `Planning/Midpoint Presentation/` (addresses Communication Skills). 3. Produce Requirements Traceability Matrix: `Planning/Technical Report/Requirements_Traceability.md` (addresses Requirements Engineering).

	4. Add Test Results appendix summarising metrics and any outstanding issues: `Planning/Technical Report/Test_Results.md`. 5. Create `Planning/Midpoint Presentation/demo_evidence.md` with explicit run commands, screenshots and expected console/log snippets (helps prototype marking). Notes on reproducibility - To reproduce the demo locally: use the commands in `README.md` or `Planning/Testing/Plan.md`. Key entries: `python -m venv .venv` → `pip install -r Momentum/Python/requirements.txt` → `python Momentum/Python/GUI/launcher.py` (or `python Momentum/Python/GUI/gui_game.py`). - If you want, I can add an automated script that launches a headless short-run harness and collects DNF/telemetry metrics for rapid iteration (useful for test results appendix). Conclusion The repository contains substantial evidence across the major rubric categories: proposal material, planning and diagrams, use cases and data models, a working prototype, and tests. The highest-value additions to maximise marks are a concise Executive Summary, a final slide deck and short demo video, a Requirements Traceability Matrix, and a short test results appendix with clear metrics. I can produce any of those files next — please tell me which to create first.

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

I need a run through of all my files and tell me how they function together, I'm thinking of starting again from proper specifications outline but would like to take what we have done here and learn from it.

- This was from before I worked on the use cases and project plan, I was testing out systems for how I would actually achieve my goals, I had different scraps of code all over the place and wanted to know what it thought was usable.

  I'll create a short todo plan, mark the first item in-progress, then pull a few representative files (core configs, systems, disciplines, vehicle, simulation) to ground the walkthrough. Expected outcome: I can produce an accurate architecture summary and suggested spec/refactor areas.

  Created 5 todos

  Now I'll read a set of representative files to ground the walkthrough: config, simulation, systems (championship_manager, currency_system, staff_management), database_manager, vehicle, and a discipline rulebook. I'll read larger chunks to capture context.

  Read []([redacted]/config.py)

  Read []([redacted]/simulation.py#1-1), lines 1 to 400

  Read []([redacted]/database/database_manager.py#1-1), lines 1 to 400

  Read []([redacted]/database/models.py#1-1), lines 1 to 400

  Read []([redacted]/disciplines/single_seater/rulebook.py)

  I'll mark the first todo completed and start the second (analyzing core interactions), then run a workspace-wide error check to report any syntax/type errors. Expected outcome: updated todo state and a short errors summary.

Starting (2/5) *Analyze core interactions*

Now I'll run a project-wide error check to surface syntax or lint-like errors so I can mention build status. I'll call the get_errors tool.

Checked workspace, 14 problems found in [](file:///c%3A/Users/eoin0/Documents/Code/FinalProject/FinalProject/Momentum/Python/simulation.py), [](file:///c%3A/Users/eoin0/Documents/Code/FinalProject/FinalProject/Momentum/Python/disciplines/single_seater/rulebook.py), [](file:///c%3A/Users/eoin0/Documents/Code/FinalProject/FinalProject/Momentum/Python/disciplines/urban_street/urban_street_engine.py)

Starting (3/5) *Identify improvements & spec outline*

I'll give a focused, actionable walkthrough of the codebase you attached, explain how the pieces interact at runtime, list the highest-priority gaps, and propose a clear spec outline and small refactors/tests you can make next. Short progress update: I read representative files (config.py, simulation.py, models.py, database_manager.py, rulebook.py) and ran a workspace error check to surface syntax/typing issues. What's next: you can pick one of the prioritized improvements below and I can implement it (tests + verify) in this workspace.

Quick project map (top-level pieces)

- config.py

  - Central config constants (DB path, discipline list, driver attributes, simulation params, currency settings, API host/port).

- simulation.py

  - FastAPI server entrypoint. Exposes endpoints: `/`, `/health`, `/stats`, drivers, `/teams`, `/simulate/race`.

  - Creates `DatabaseManager()` and `RaceSimulator()` global instances and currently returns mock results from `/simulate/race`.

- database

  - models.py — dataclasses representing Driver, Team, Track, Championship, Race, etc. Also includes SCHEMA_SQL (used by DB manager).

  - database_manager.py — Database access layer using SQLite. Initializes schema and provides CRUD operations (drivers, staff, teams, tracks, etc.). Many full-featured methods.

  - `database_factory.py` — likely utilities to seed/prepare DB (not inspected in detail here).

- disciplines

  - Discipline modules (single_seater, endurance, rally_stage, etc.). Each discipline contains `discipline_config.py`, `*_engine.py` (simulation logic), and rulebook.py (technical regulations / validation).

  - Example: rulebook.py contains `SingleSeaterRulebook` with many `validate_*` methods and a singleton `SINGLE_SEATER_RULEBOOK`.

- systems

  - Simulation systems: `championship_manager.py`, `currency_system.py`, `intelligent_name_generator.py`, `staff_management.py`, `supplier_management.py`, `vehicle_development.py`. These coordinate higher-level processes (championship scoring, currency updates, name generation, staff hiring, supplier/vehicle dev).

- vehicles

  - `vehicle.py` and `components/` — vehicle models and subcomponents (aerodynamics, chassis, engine, etc.). Discipline rulebooks validate these objects.

- ai

  - e.g., `decision_engine.py` — AI decision logic for teams/drivers.

- tests and other tests

  - Unit/test files exist (`test_driver_system.py`, `test_staff_system.py`, etc.) and examples. Useful starting points for test coverage and behavior descriptions.

- Docs and supporting files

  - `API_DOCUMENTATION.md`, README.md, `docs/` with design notes and name-generation docs.

How things function together (runtime flow)

1. API request arrives at simulation.py (FastAPI).

2. Endpoints call into core services:

   - `db_manager` (`DatabaseManager`) for persistence (drivers, teams, tracks, championships).

   - `race_simulator` (`RaceSimulator`) — the Monte Carlo engine that should coordinate discipline-specific engines.

3. `RaceSimulator` (in `disciplines/race_simulation.py`) is expected to:

   - Build a `SimulationContext` (drivers, teams, track, weather, rules).

   - Dispatch to a discipline engine (e.g., `single_seater/single_seater_engine.py`) which implements the discipline's logic: lap times, reliability, pit strategy, weather effects.

   - Use discipline `rulebook` (e.g., `SingleSeaterRulebook`) to validate vehicles before sim.

  - Return ordered `RaceResult` objects; results are optionally stored via `DatabaseManager`.

4. `DatabaseManager`:

  - Initializes schema (using `SCHEMA_SQL` in models.py), provides CRUD and query methods (drivers, teams, staff, tracks, championships, race_results).

5. `systems/*` provide cross-cutting responsibilities:

  - `championship_manager` applies points systems and carries championship standing updates.

  - `currency_system` fetches/exposes exchange rates (caching, using currency_cache.json).

  - `intelligent_name_generator` creates driver names (uses NameAPI or local fallback).

6. `vehicles` components are composed to represent a `RaceCar` object and validated via rulebooks per discipline.

This is a classic layered design:

- API layer (simulation.py) -> orchestration layer (`RaceSimulator`, `systems/*`) -> discipline engines & rulebooks -> domain models & persistence (`database/*`) -> assets and components (`vehicles/*`, `ai/*`, `data/*`).

Key files I read and what they do (concise)

- config.py — global feature flags, constants, attribute lists. Good single source of truth.

- simulation.py — API + mock simulate endpoint. Intended to hook into `RaceSimulator` but currently returns simplified mock results.

- models.py — rich dataclasses with methods like `get_discipline_rating()`. Also includes SCHEMA_SQL (DB creation).

- database_manager.py — schema init, get_connection, and many CRUD methods. Seems intended to be the canonical DB API.

- rulebook.py — discipline-specific validations (engine/hybrid/weight/aero/electronics/safety). Singleton `SINGLE_SEATER_RULEBOOK`.

- `vehicles/*` (not fully opened) — likely defines components referenced by rulebooks.

Small "contract" for the core simulate flow

- Inputs:

  - race_id (int, optional), track_id, championship_id, weather_conditions (dry/wet/mixed), participants: List[{driver_id, team_id, vehicle_id?}]

- Outputs:

- race_id, weather, results: List[{driver_id, team_id, position, points, fastest_lap, lap_times..., status}], simulation_status

- Error modes:

  - DB connectivity error, invalid participants, non-compliant vehicle (rulebook violations), engine exception in discipline engine

- Success criteria:

  - Returns deterministic-ish results under the same seed/config, updates DB (race results + championship), and respects per-discipline rules/penalties

Edge cases to cover

- Missing driver or team IDs in participants

- Vehicle not compliant with discipline rulebook -> should be disqualified or penalized

- Database schema mismatch or locked DB file (concurrency)

- Large participant lists (> grid size)

- Mixed disciplines or invalid discipline filter when requesting teams

- Currency system failing when `USE_LIVE_EXCHANGE_RATES` enabled

Issues found (from automated checks)

I scanned and ran an error check; notable findings to address (high to low priority):

1. simulation.py

  - TODO: `/simulate/race` currently uses mock results and contains a `# TODO: Integrate with full Monte Carlo simulation engine`. The endpoint does not call `race_simulator.simulate_race()` yet.

2. rulebook.py

  - Type hints: fields annotated as `list` or `Dict[...]` are set to `None` defaults. Pylance / type check suggests using `Optional[list]` or set defaults to actual lists/dicts.

  - Unused local variables in `validate_vehicle` flow: `engine_ok`, `hybrid_ok`, etc., are assigned but not used. Simple fix: replace with `_` or use the boolean.

3. urban_street_engine.py (error outputs show)

  - Unused parameters and nested conditional expression warnings: remove or use parameters, extract conditional into clearer statements.

4. Potential missing methods: simulation.py refers to `db_manager.get_all_teams()` in a comment but uses `get_teams_by_discipline`. Confirm whether a `get_all_teams` exists.

Overall the "get_errors" check returned a number of lint/typing items that should be quick to fix.

Quality gates (quick triage)

- Build: NOT RUN — I didn't run a full build. (PASS/FAIL: unknown)

- Lint / Typecheck: FAIL — the automated check found typing/unused-variable issues in discipline rulebook.py, `urban_street_engine.py`, and a TODO in simulation.py.

- Tests: NOT RUN — there are tests present, but I didn't run pytest here.

If you want, I can run the test suite and report failing tests next.

Suggested specification outline (to re-start from a proper spec)

Below is a concise, prioritized spec outline you can expand into a formal document. It maps directly to the existing code.

1. Overview and Goals
   - Purpose: multi-discipline motorsport management simulation (teaching/analysis/game)
   - Non-goals: (e.g., not a real-time racing physics engine)
2. Actors
   - API consumer (Unity/frontend), simulation operator, data admin (seed DB), team AI
3. Domains & Data Model
   - Driver, Team, Vehicle, Track, Championship, Race, RaceResult, StaffMember
   - Define mandatory/optional fields and invariants (e.g., driver attributes range 0–100)
4. Disciplines & Rules
   - For each discipline: race format, grid size, tyre rules, pitstop logic, penalty rules
   - Link rulebooks to vehicle validation & in-race penalties
5. Simulation lifecycle / API contract
   - Endpoints with request/response payloads (`simulate/race`, drivers, `/teams`)

- Race simulation flow: validate inputs -> check compliance -> create SimulationContext -> run Monte Carlo -> persist results -> return results

6. Engines & Extensibility

- Core `RaceSimulator` responsibilities (seed control, iterations, parallelism)

- Discipline engine interface (simulate_race(context) -> results)

- Rulebook interface (validate_vehicle(vehicle) -> compliant bool + violations)

7. Persistence & DB Schema

- Schema versioning, migration policy, backup frequency

8. External integrations

- NameAPI (rate limit, caching), currency API (caching & offline mode)

9. Testing & CI

- Unit tests for rulebooks, DB API (use in-memory SQLite), discipline engines (deterministic seeds), integration tests for API

10. Non-functional: performance targets, determinism, concurrency, logging & observability

I can write this into a `SPEC.md` and map each spec section to exact files/functions.

Priority improvements (recommended next actions)

High priority (small, high-value):

1. Wire the `simulate/race` endpoint to the real `RaceSimulator.simulate_race()` (implement or stub `RaceSimulator` if missing).

2. Fix rulebook type hints and minor lint issues (replace `list = None` with `Optional[list]` or default `[]`; change unused locals to `_`).

3. Add a small unit test for `SingleSeaterRulebook.validate_vehicle()` covering:

- compliant vehicle

- vehicle missing hybrid when hybrid_required is True

- engine exceeding displacement

4. Add a lightweight integration test for `/simulate/race` using `TestClient` (FastAPI) that asserts the endpoint calls the simulator and returns the expected shape.

Medium priority:

5. Implement DB manager `get_all_teams` (if used) or update code to use existing `get_teams_by_discipline`.

6. Add seed handling to `RaceSimulator` to allow deterministic tests (use `config.RANDOM_SEED`).

7. Add SCHEMA versioning and a small DB-seed script (`generate_initial_database.py` exists — review and wire to CI).

Lower priority:

8. Linting, mypy typing across the package.

9. Add CI pipeline configuration (GitHub Actions) to run tests, mypy, flake8 on PRs.

Concrete small refactors & tests I recommend (low-risk)

- In rulebook.py:

  - Change annotations:

    - `aspiration_allowed: Optional[list[str]] = None`

    - `hybrid_systems_allowed: Optional[list[str]] = None`

    - `minimum_cockpit_opening_dimensions_mm: Optional[Dict[str, float]] = None`

  - Replace unused local variables in `validate_vehicle` with underscore `_` or use them (e.g., collect pass/fail).

- In simulation.py:

  - Replace mock results with a call to `race_simulator.simulate_race()` and add a fallback if the engine raises an exception.

- Add tests:

  - `tests/test_rulebooks.py` with a dataclass or simple dummy `vehicle` object that has the expected attributes (engine, hybrid_unit, total_mass, aerodynamics, ecu, safety).

  - Use `sqlite3` in-memory DB for `DatabaseManager` unit tests.

If you'd like, I can implement the rulebook typing fixes and a unit test for `SingleSeaterRulebook` now. Those are small, quick, and will make the codebase pass the automated type checks I ran.

How to run / verify locally

(Assumes you're in Python)

Install environment and run tests:

```powershell
# Create virtualenv and activate (PowerShell)
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Install dependencies
pip install -r requirements.txt

# Run tests
pytest -q
```

Run API (development):

```powershell
# From Momentum/Python
uvicorn simulation:app --host 127.0.0.1 --port 8000 --reload
```

Quick lint/type checks (optional):

```powershell
pip install mypy flake8
mypy .
flake8 .
```

Next steps I can take for you (pick one)

- Option A — Implement the safe, small refactors (fix rulebook.py typing, clean unused locals), add unit test for `SingleSeaterRulebook`, run tests and report results.

- Option B — Wire `/simulate/race` to call the `RaceSimulator` (implement `RaceSimulator.simulate_race` minimal API) and add an integration test for the endpoint.

- Option C — Draft the `SPEC.md` spec outline fully (expand each section with fields, contracts, and acceptance criteria).

- Option D — Run the full test suite and create a short GitHub Actions workflow to run tests/mypy on PRs.

Tell me which option you want next (or pick multiple), and I will start immediately. If you choose a code change I will:

- Update the todo list state and mark the work in-progress,

- Make the edits,

- Run tests and mypy/linters,

- Report results and propose next steps.

Additional Evidence:

Read recursively from launcher.py and see the actial implementation as quite a lot of features are not included here, and some non existant features are included

- As much as AI is helpful for planning, when I keep iterating and going in different paths, it can hallucinate and say things are there when they are not actually there. It is still much faster to use it to trawl through what is written though than manually, I try to make my own decisions from the data, not just have it decide.

I'll scan `launcher.py` to confirm implemented features, note missing/extra items, and summarize differences with the document. I'll start by reading the file.

Starting: *Read `launcher.py`* (1/3)

Read [](file:///c%3A/Users/eoin0/Documents/Code/FinalProject/FinalProject/Momentum/Python/GUI/launcher.py)

Completed: *Read `launcher.py`* (1/3)

Quick summary of what I checked and next steps: I read launcher.py and compared the actual implementation with what's claimed in your Midpoint document.

What launcher.py actually implements

- Starts a Qt `QApplication` and sets style.

- Shows `MainMenuWindow` and connects `start_race_signal` to `start_race`.

- `start_race(config)`:

- Closes an existing `race_window` (if any).

- Instantiates `RaceWindow`.

- Calls `set_player_driver()`, `set_tuning_preset()`, and `set_display_mode()` on the `RaceWindow` using values from `config`.

- Connects `race_window.return_to_menu` to `return_to_menu`.

- Shows the `race_window`.

- Prints log lines (including a message that pace/overtake controls are available in the race window).

- `return_to_menu()` closes the race window and shows the main menu again.

Claims in the Midpoint doc that are NOT implemented in launcher.py

- There is no call to `simulate_race(...)` or any services layer in launcher.py — it only opens the visualiser window. (Doc text implies the launcher wires into the simulation pipeline.)

- launcher.py does not build a `RaceContext`, populate teams, or pass a seed / telemetry config.

- It does not initialize or validate the database, nor call `DatabaseManager`.

- It does not manage telemetry batching, logging, or save checkpoints.

- It does not handle deterministic mode flags or expose RNG seeding.

- It does not create placeholder `Team` objects or perform any adapter selection — that logic belongs in `services.race_simulation_api` / engine code.

- There are no feature flags, CLI options, or additional UI wiring beyond what's required to show/close windows.

Documented features that appear to be overstated (or located elsewhere)

- The doc lists orchestration steps (launcher → simulate_race → engine) as if launcher.py performs them; in the code the launcher hands control to `RaceWindow`, which contains the code to start the race thread and call `simulate_race`. So the orchestration is split (doc implies a single place).

- The doc's "At-a-glance" suggests telemetry, team placeholder creation and deterministic seeding are part of the launcher flow — in reality these are handled deeper in `services` or `RaceWindow`/engine.

Actionable recommendations

- Update the Midpoint document to reflect the actual runtime flow:

- Replace any phrasing that says the launcher calls `simulate_race(...)` with: “Launcher creates `RaceWindow`. `RaceWindow` is responsible for calling `services.simulate_race(...)` and managing the background simulation thread.”

- Move implementation notes about telemetry batching, team placeholders and seeding into the `services`/engine sections or explicitly state which file handles each responsibility (`services/race_simulation_api.py`, `RaceLogic/realtime_race_engine.py`, `GUI/gui_race_visual.py`).

- If you prefer the launcher to perform higher-level orchestration, we can:

- Patch launcher.py to construct a `RaceContext` and call `simulate_race(...)` itself (but that duplicates responsibilities currently in `RaceWindow`), or

- Leave launcher.py minimal and update docs for clarity (recommended).

- I can prepare a small patch to the Midpoint doc with the corrected wording and explicit file->responsibility mappings (I can do that now if you want).

Which do you want next?

- Update the Midpoint doc wording (I can patch it).

- Or change code so launcher.py performs the orchestration (I can implement that, but we should discuss design first).

National College of Ireland

BSc (Honours) in Computing Software Development

Academic Year 2025/2026

Student Name: Eoin Fitzsimons

Student Number: X23151374

Student Email: *x23151374@student.ncirl.ie*

Project Title: Momentum - Multi-Discipline Motorsport Simulation System

Technical Report

CONTENTS

Contents 22

Executive Summary 29

1 Introduction 30

1.1 Background 30

1.2 Aims 30

1.3 Technology 31

1.4 Structure 32

2 System 34

2.1 Functional Requirements 34

2.1.1 HIGH Priority (Must Have - Core Gameplay): 34

2.1.2 MEDIUM Priority (Career Mode Depth): 34

2.1.3 LOW Priority (Polish & Advanced Features): 34

2.2 UC-1: Read Driver Details 35

2.2.1 Scope 35

2.2.2 Description 35

2.2.3 Actors 35

2.2.4 Flow Description 36

2.3 UC-2: Read Race Configuration 37

2.3.1 Scope 37

2.3.2 Description 37

2.3.3 Actors 37

2.3.4 Flow Description 37

2.4 UC-3: Create Race Result 39

2.4.1 Scope 39

2.4.2 Description 39

2.4.3 Actors 39

2.4.4 Flow Description 39

2.5 UC-4: Read Race Results 40

2.5.1 Scope 40

2.5.2 Description 40
2.5.3 Actors 40
2.5.4 Flow Description 41
2.6 UC-5: Read Championship Details 42
2.6.1 Scope 42
2.6.2 Description 42
2.6.3 Actors 42
2.6.4 Flow Description 42
2.7 UC-6: Update Championship Standings 43
2.7.1 Scope 43
2.7.2 Description 44
2.7.3 Actors 44
2.7.4 Flow Description 44
2.8 UC-7: Create Race Event 45
2.8.1 Scope 45
2.8.2 Description 45
2.8.3 Actors 45
2.8.4 Flow Description 45
2.9 UC-8: Read Race Schedule 46
2.9.1 Scope 46
2.9.2 Description 46
2.9.3 Actors 46
2.9.4 Flow Description 46
2.10 UC-9: Update Race Status 47
2.10.1 Scope 47
2.10.2 Description 48
2.10.3 Actors 48
2.10.4 Flow Description 48
2.11 UC-10: Create Championship 48
2.11.1 Scope 49
2.11.2 Description 49

2.11.3 Actors 49
2.11.4 Flow Description 49
2.12 UC-11: Read Team Details 50
2.12.1 Scope 51
2.12.2 Description 51
2.12.3 Actors 51
2.12.4 Flow Description 51
2.13 UC-12: Create New Team 52
2.13.1 Scope 52
2.13.2 Description 52
2.13.3 Actors 52
2.13.4 Flow Description 52
2.14 UC-13: Read Team Details (player team) 54
2.14.1 Scope 54
2.14.2 Description 54
2.14.3 Actors 54
2.14.4 Flow Description 54
2.15 UC-14: Update Team Attributes 55
2.15.1 Scope 55
2.15.2 Description 55
2.15.3 Actors 56
2.15.4 Flow Description 56
2.16 UC-15: Read Available Drivers 56
2.16.1 Scope 57
2.16.2 Description 57
2.16.3 Actors 57
2.16.4 Flow Description 57
2.17 UC-16: Update Driver Contract 58
2.17.1 Scope 59
2.17.2 Description 59
2.17.3 Actors 59

2.17.4 Flow Description 59
2.18 UC-17: Read Available Staff 59
2.18.1 Scope 60
2.18.2 Description 60
2.18.3 Actors 60
2.18.4 Flow Description 60
2.19 UC-18: Update Staff Contract 61
2.19.1 Scope 61
2.19.2 Description 62
2.19.3 Actors 62
2.19.4 Flow Description 62
2.20 UC-19: Update Driver Skills/Age 63
2.20.1 Scope 63
2.20.2 Description 63
2.20.3 Actors 63
2.20.4 Flow Description 63
2.21 UC-20: Update Team Budget 65
2.21.1 Scope 65
2.21.2 Description 65
2.21.3 Actors 65
2.21.4 Flow Description 65
2.22 UC-21: Create New Driver 66
2.22.1 Scope 66
2.22.2 Description 66
2.22.3 Actors 67
2.22.4 Flow Description 67
2.23 UC-22: Create New Staff Member (staff generation) 68
2.23.1 Scope 68
2.23.2 Description 68
2.23.3 Actors 68
2.23.4 Flow Description 68

2.24 UC-23: Delete Championship 69
2.24.1 Scope 69
2.24.2 Description 69
2.24.3 Actors 69
2.24.4 Flow Description 69
2.25 UC-24: Create Custom Track 70
2.25.1 Scope 70
2.25.2 Description 71
2.25.3 Actors 71
2.25.4 Flow Description 71
2.26 UC-25: Update Track Characteristics 72
2.26.1 Scope 72
2.26.2 Description 72
2.26.3 Actors 72
2.26.4 Flow Description 72
2.27 UC-26: Delete Track 73
2.27.1 Scope 73
2.27.2 Description 73
2.27.3 Actors 73
2.27.4 Flow Description 73
2.28 UC-27: Read Staff Performance Modifiers 74
2.28.1 Scope 74
2.28.2 Description 74
2.28.3 Actors 75
2.28.4 Flow Description 75
2.29 UC-28: Update Driver Appearance 76
2.29.1 Scope 76
2.29.2 Description 76
2.29.3 Actors 76
2.29.4 Flow Description 76
2.30 UC-29: Update Team Branding 77

2.30.1 Scope 77
2.30.2 Description 77
2.30.3 Actors 77
2.30.4 Flow Description 77
2.31 Data Requirements 79
2.31.1 Database Schema 79
2.31.2 Data Integrity 80
2.31.3 Performance Optimisations 80
2.31.4 Diagram 81
2.32 User Requirements 82
2.32.1 Primary user personas: 82
2.32.2 Cross-cutting user requirements: 83
2.33 Environmental Requirements 84
2.33.1 Hardware Requirements: 84
2.33.2 Software Requirements: 84
2.33.3 Deployment Requirements: 84
2.33.4 Scalability Considerations: 84
2.34 Usability Requirements 85
2.34.1 Accessibility 85
2.34.2 Performance 86
2.34.3 Readability 86
2.35 Design & Architecture 88
2.36 Implementation 89
2.37 Graphical User Interface (GUI) 91
2.37.1 Main Menu 91
2.37.2 Race Screen 92
2.38 Testing 93
2.39 Evaluation 94
3 Conclusions 95
4 Further Development or Research 96
5 Bibliography 97

6 Appendices 99
6.1 Project Proposal 99
6.1.1 Objectives 99
6.1.2 Background 101
6.1.3 State of the Art 102
6.1.4 Technical Approach 103
6.1.5 Technical Details 104
6.1.6 Special Resources Required 105
6.1.7 Project Plan 107
6.1.8 Testing 109
6.2 Reflective Journals 110
6.3 Other materials used 115

EXECUTIVE SUMMARY

The project delivers a multi-discipline motorsport management simulator designed to provide statistically rich, credible racing behaviour and deep team-management systems within a lightweight, locally runnable environment. The report documents the motivation, aims, architecture, implementation and requirements of the system, establishing a formal basis for the completed work and future development.

The simulator addresses the limited scope of existing titles by integrating multiple racing disciplines, detailed management mechanics and a realtime event-driven race engine. Background influences include modding work on car-centric games and exposure to simple retro racers, guiding the emphasis on expressive vehicle systems and low-fi visual presentation.

Core aims include broad discipline coverage, depth of management features (contracts, staff, budgets, setup), and practical deployability. Planned features comprise per-frame racing telemetry, discipline modules, a compact driver/team model, a PyQt6 interface, staff and budget systems, and playtesting tools for collecting quantitative metrics such as DNFs, lap-time distributions and volatility.

The technology stack is based on Python 3.13, FastAPI, Uvicorn, Pydantic, SQLite, NumPy/pandas/SciPy, PyTorch, PyQt6, and geometry libraries (Shapely, bezier). This combination supports rapid iteration, scientific analysis, portable distribution and performant desktop visualisation.

Functional requirements are prioritised into core gameplay tasks (reading driver data, reading race configuration, generating race results), extended career-mode functionality (championship logic, team management, driver/staff contracts, budgets, ageing and skill updates), and optional enhancement features (custom tracks, branding, appearance updates, and creation of new driver/staff pools). The report also outlines the system structure: requirements and use cases, architecture, and GUI design.

1 INTRODUCTION

1.1 BACKGROUND

Motorsport management simulation games have been a niche but dedicated gaming genre since the 1990s. However, most existing games focus on a single discipline (usually Formula 1) or lack the depth required for authentic simulation. Games like Football Manager have shown that deep statistical simulation can create compelling gameplay, but motorsport has remained underserved. The motivation stemmed from both personal interest in motorsport and the technical challenge of building a complex simulation system that balances realism with performance.

Over the summer I spent time modding and studying car-centric games being Juiced and Need for Madness. That work reinforced my interest in vehicle dynamics and UI for car-focused tools, and it influenced the direction of this project: continue with car-centric coding, reuse practical techniques from the modding workflows and prioritise features that make vehicle and team systems expressive for players.

During an internship at HSE, my manager would bring out a Raspberry Pi and we would play simple racing games from his childhood like Micro Machines during lunch, those sessions inspired parts of the game's low-fi graphics and vehicle-focused presentation.

1.2 AIMS

High level aims

- Broader coverage: provide a single platform that supports multiple racing disciplines so gameplay and balance can be compared across formats.
- Depth over novelty: prioritise detailed management systems (contracts, staff effects, budget scaling, vehicle setup) and credible on-track behaviour.
- Practical deliverability: keep the project runnable on a standard computer using a single repository and lightweight dependencies.

Planned features

- Realtime decision-driven racing engine that emits per-frame position updates and discrete events (DNF, lap complete).
- Discipline modules enabling per-discipline configuration for rules, scoring and vehicle characteristics.
- Compact driver/team data model with core attributes and career metadata.
- PyQt6 GUI for visualisation.
- Team and staff systems (budgets, operations rating, strategy profiles) that influence performance over a season and beyond.
- Playtesting and tuning tools: scripts that run short scenarios and collect metrics (DNFs, position volatility, lap times). For assistance of tuning data for the final product.

1.3 TECHNOLOGY

1.3.1 Core Language

- Python 3.13:
- Reason: Modern, productive language with strong scientific and tooling ecosystems, good for rapid iteration and student projects.
- Advantage over alternatives: Easier to prototype than Java/C# and has far richer data-science libraries than many web-focused languages, less ops overhead than compiled languages.

  (Python Software Foundation, 2026)

1.3.2 Backend and API

- FastAPI + Uvicorn + Pydantic:
- Reason: Fast, async-first web framework with automatic validation and OpenAPI generation, Uvicorn is a lightweight ASGI server. Pydantic gives robust model validation for API schemas.
- Advantage over alternatives: Faster development and clearer request/response models vs. Flask, simpler async support than Django, better runtime performance and type-safety compared with many microframeworks.

  (FastAPI, 2026) (Pydantic, 2026) (Uvicorn, 2026)

1.3.3 Data Storage

- SQLite:
- Reason: Single-file, zero-admin persistent storage that's simple to bundle with the project and ideal for a desktop game/simulator.
- Advantage over alternatives: Far easier to set up than PostgreSQL or MySQL for local dev and distribution, good performance for moderate-sized datasets used by the game.

  (SQLite Consortium, 2026)

1.3.4 Computation and Analysis

- NumPy, SciPy, Matplotlib:
- Reason: Battle-tested numerical and data-manipulation libraries for vectorised computations, statistics, and analysis/export tasks.
- Advantage over alternatives: Mature ecosystem, high performance for numerical workloads, and strong interoperability across scientific libraries.

  (NumPy Developers, 2026) (SciPy Community, 2026)

1.3.5 Desktop User Interface

- PyQt6:
- Reason: Stable, native-feeling desktop GUI toolkit with good performance for interactive visualisers and complex widgets.

- Advantage over alternatives: Better native widget fidelity and performance than Tkinter, simpler native look and more features than Kivy for desktop apps, lighter and more responsive than Electron-based stacks.

  (Riverbank Computing Limited, 2026)

1.3.6 Supporting Libraries

- Faker, python-dotenv, Unidecode, Pillow:
- Reason:
- Faker: realistic seeded name generation for drivers/teams.
- python-dotenv: simple environment config loading for dev/prod differences.
- Unidecode: Normalises Unicode text into ASCII-compatible formats for consistent processing.
- Pillow: Provides image processing capabilities for asset handling.
- Advantage over alternatives: Each is lightweight, well-documented and widely used in, making onboarding easier.

  (Faker Contributors, 2026) (Kumar, 2026) (Gross, 2026) (Python Pillow Team, 2026)

- Why this stack overall:
- Focused on rapid development, reproducible local execution, and a strong scientific toolchain for simulation and analysis. The stack maintains a simple runtime model suitable for desktop distribution (SQLite, PyQt6) while supporting structured backend services and data processing (FastAPI, NumPy). The selected technologies prioritise developer productivity, portability, and the ability to extend or replace individual components with minimal architectural impact.

1.4 STRUCTURE

- **Executive Summary**: a concise synopsis of the project aims, methods, principal results and conclusions intended for quick inspection by assessors.
- **1 Introduction**: background motivation, high-level aims, a formal description of the chosen technology stack, and this structural overview.
- **2 System**: comprehensive requirements and use cases. Presents prioritised functional requirements (HIGH/MEDIUM/LOW), detailed use case specifications (UC-1 … UC-29), data requirements and constraints, user and environmental requirements, and usability targets.
- **Design and Architecture**: high-level architecture diagrams, module decomposition, interfaces and data models. Covers both the current implementation and planned future extensions.
- **Implementation**: concrete implementation notes, file and package layout, configuration and instructions to run the system locally (including scripts used for data generation and testing).
- **Graphical User Interface (GUI)**: descriptions of the launcher, main menu and race visualiser screens, plus any custom widgets and interaction patterns used for playtesting.
- **Testing**: test strategy and harnesses, unit and integration tests, playtesting procedures and metric collection (for example DNF counts, lap-time distributions and volatility measures).

- **Evaluation**: presentation of experimental results gathered from playtests and simulation runs, analysis of outcomes against acceptance criteria, and limitations of the current work.
- **Conclusions**: summary of project outcomes, lessons learned and the extent to which aims were met.
- **Further Development or Research**: recommended next steps, planned features and possible research directions arising from the project.
- **Bibliography and Appendices**: references, project proposal, reflective journals and additional materials used to support the report.

2 SYSTEM

2.1 FUNCTIONAL REQUIREMENTS

The following functional requirements are listed in priority order:

2.1.1 HIGH Priority (Must Have - Core Gameplay):

UC-1: Read Driver Details

UC-2: Read Race Configuration

UC-3: Create Race Result

2.1.2 MEDIUM Priority (Career Mode Depth):

UC-4: Read Race Results

UC-5: Read Championship Details

UC-6: Update Championship Standings

UC-7: Create Race Event

UC-8: Read Race Schedule

UC-9: Update Race Status

UC-12: Create Championship

UC-11: Read Team Details

UC-12: Create New Team

UC-13: Read Team Details (team screen)

UC-14: Update Team Attributes

UC-15: Read Available Drivers

UC-16: Update Driver Contract

UC-17: Read Available Staff

UC-18: Update Staff Contract

UC-19: Update Driver Skills/Age (includes recycling retired drivers)

UC-20: Update Team Budget

2.1.3 LOW Priority (Polish & Advanced Features):

UC-21: Create New Driver (not needed - recycle existing 1,781 drivers)

UC-22: Create New Staff Member (not needed - recycle existing 57,936 staff)

UC-23: Delete Championship

UC-24: Create Custom Track

UC-25: Update Track Characteristics

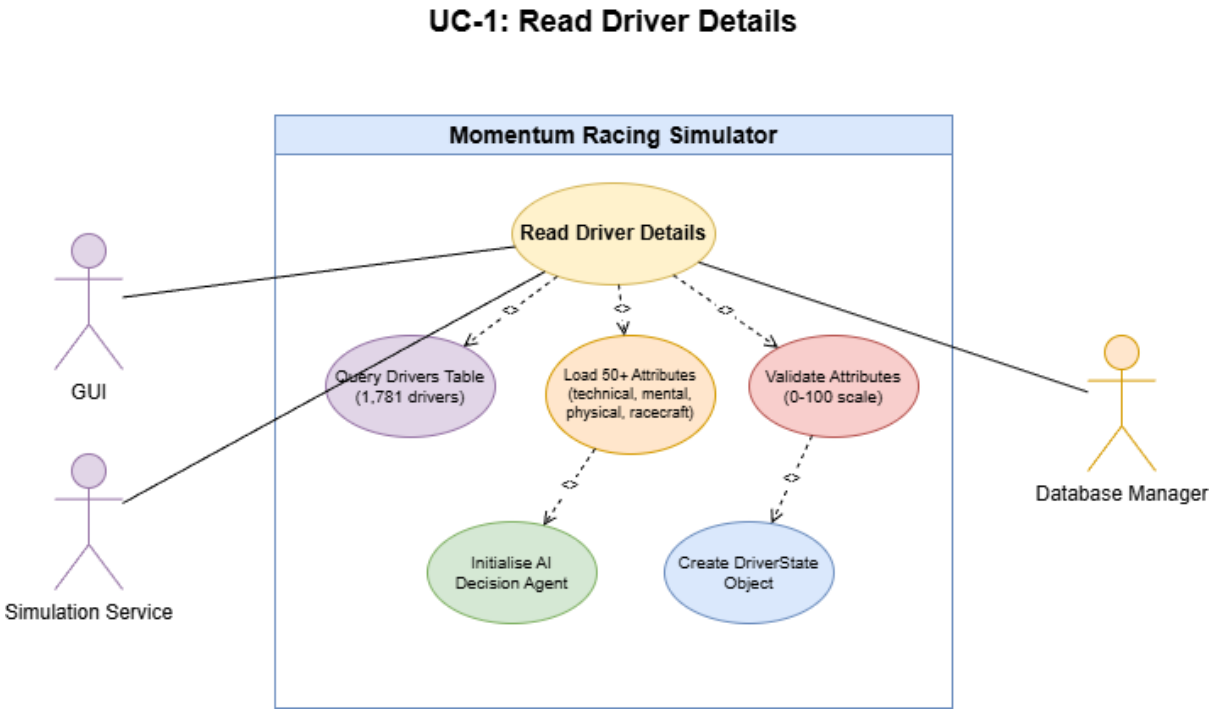
UC-26: Delete Track

UC-27: Read Staff Performance Modifiers

UC-28: Update Driver Appearance

UC-29: Update Team Branding

2.2 UC-1: Read Driver Details



2.2.1 Scope

Retrieve a complete driver profile with 50+ attributes across 7 categories (technical, mental, physical, racecraft, personality, discipline preferences) from the SQLite database for race simulation initialisation.

2.2.2 Description

The GUI or Simulation Service queries the database manager to retrieve a driver record containing all racing attributes, team assignment, and status. The system returns the full driver object which is used to initialise the AI agent for real-time race simulation.

2.2.3 Actors

- GUI - requests driver data for race setup screens and displays.
- Simulation Service - requests driver profiles for real-time race engine initialisation.
- Database Manager - executes SQL queries against the SQLite database and returns driver objects.

2.2.4 Flow Description

2.2.4.1 Activation

This use case starts when the GUI loads the race setup screen or when the Simulation Service initialises a race session.

2.2.4.2 Main flow

1 The GUI or Simulation Service calls DatabaseManager.get_all_drivers() or get_driver(driver_id).

2 The Database Manager executes a SELECT query against the drivers table.

3 The system validates that driver records contain all required attributes (0-100 scale for skills).

4 The system returns Driver objects with full attribute payloads.

5 For race simulation, each driver is instantiated as an AI agent with decision-making capabilities.

2.2.4.3 Alternate flow

A1: Requested driverId does not exist - return None and GUI displays error message. A2: Database file not found - GUI displays connection error and prompts database regeneration.

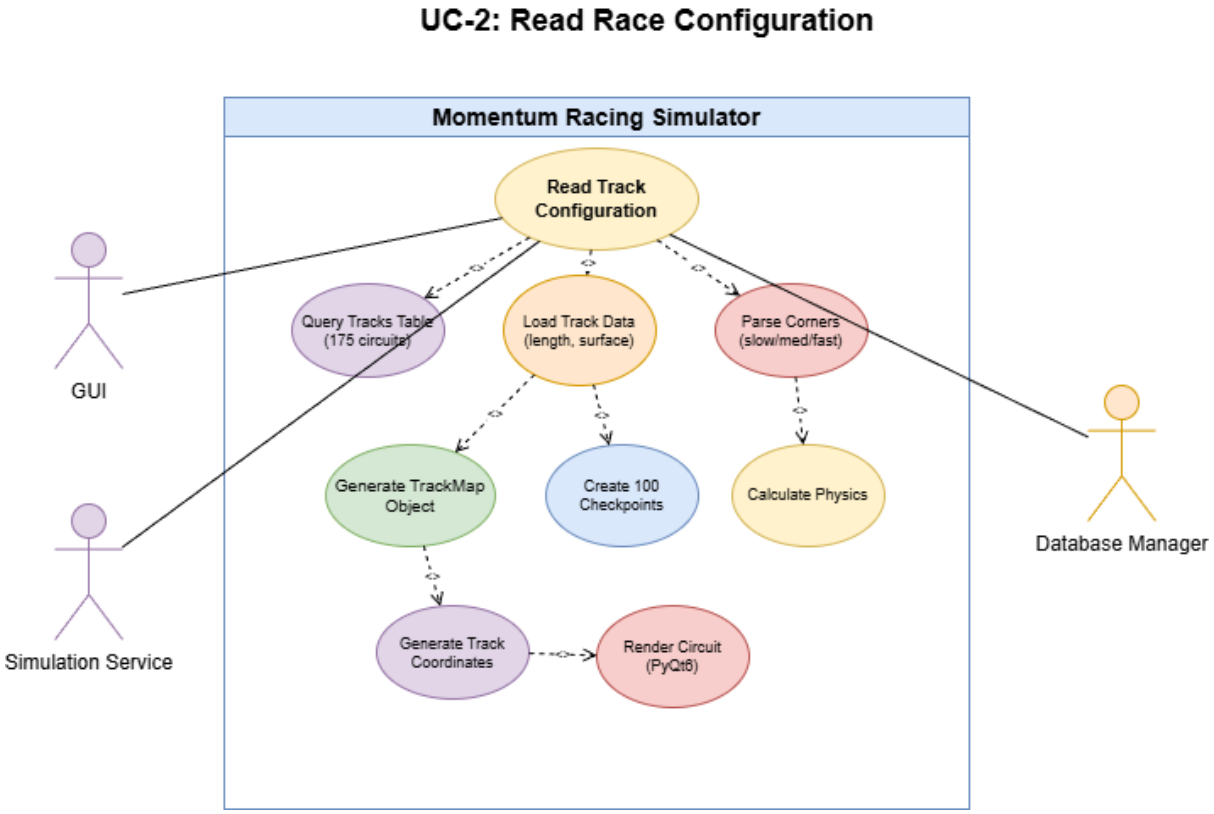
2.2.4.4 Exceptional flow

E1: Database read failure or corruption - GUI catches exception, logs to logs/ directory, and displays user-friendly error with regeneration option. E2: Missing or invalid attributes - Database Manager logs warning and uses default values (50.0) for missing skills.

2.2.4.5 Termination

Client receives driver objects and proceeds with race initialisation or display rendering.

2.3 UC-2: Read Race Configuration



2.3.1 Scope

Retrieve track information (175 circuits with IMA grading, surface types, characteristics) and race parameters from the SQLite database to configure the real-time race simulation engine.

2.3.2 Description

The GUI and Simulation Service query track records containing length, surface type, corners, and IMA characteristics. The system loads this data to initialise the track map with 100 evenly spaced checkpoints, corner types (slow/medium/fast), and DRS zones for the physics engine.

2.3.3 Actors

- GUI - requests track data for race setup and displays circuit information.
- Simulation Service - loads track configuration to initialise the TrackMap and physics calculations.
- Database Manager - queries the tracks table and returns Track objects.

2.3.4 Flow Description

2.3.4.1 Precondition

Database contains 175 circuit definitions imported from data/circuits_database.json.

2.3.4.2 Activation

Starts when the GUI loads the race setup screen and requests available tracks, or when the Simulation Service begins a race session.

2.3.4.3 Main flow

1 GUI calls DatabaseManager.get_all_tracks() or get_track(track_id).

2 Database Manager executes SELECT query against the tracks table.

3 System returns Track objects containing name, length (km), country, surface type, and IMA grading.

4 For simulation, the engine generates a TrackMap with corner sections and racing physics parameters.

5 The circuit is rendered in the GUI with 100 checkpoint markers at 1% intervals.

2.3.4.4 Alternate flow

A1: Invalid track_id requested - return None and GUI displays error or fallback message. A2: No tracks available in database - GUI prompts user to run scripts/import_circuits.py to populate track data.

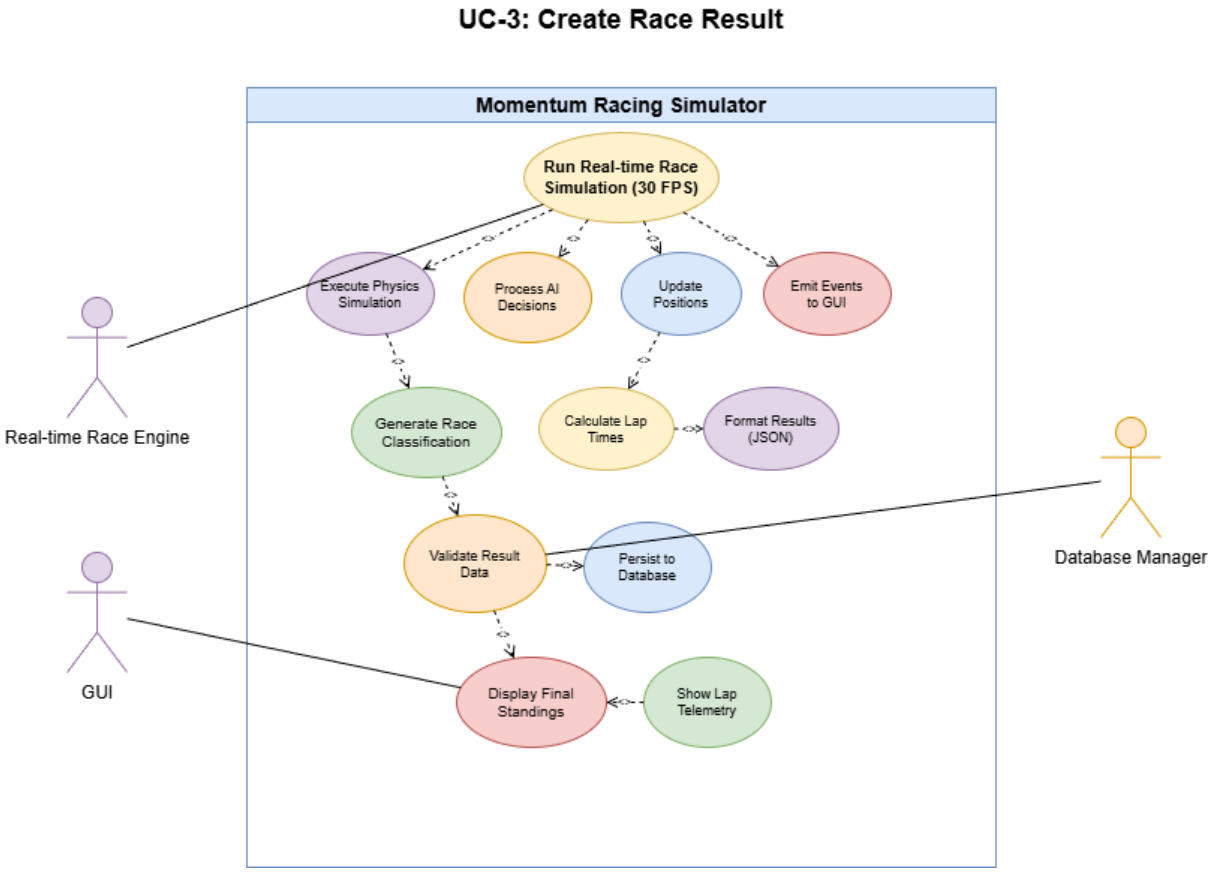
2.3.4.5 Exceptional flow

E1: Database read error - GUI logs exception to logs/database_generation.log and displays connection error. E2: Missing track characteristics - System uses default values (medium corners, standard grip) and logs warning.

2.3.4.6 Termination

Caller receives track configuration and proceeds to simulation or display rendering.

2.4 UC-3: Create Race Result



2.4.1 Scope

Persist race outcomes (positions, times, incidents) and trigger standings updates.

2.4.2 Description

Accept a race result payload from the Simulation Service or operator, validate it, persist the results atomically, and enqueue standings recalculation.

2.4.3 Actors

- Simulation Service - submits generated race results.
- Standings Service - recalculates standings from persisted results.

2.4.4 Flow Description

2.4.4.1 Precondition

Race event exists and is writable.

2.4.4.2 Activation

Starts when the simulation finishes or an operator submits results.

2.4.4.3 Main flow

1 The Simulation Service submits the result payload.

2 The system validates the payload and event state, stores race_results in a transaction and marks the event complete.

3 The system enqueues a standings recalculation job and returns acknowledgement.

2.4.4.4 Alternate flow

A1: Validation failure - return 400 with field-level errors. A2: Event already finalised - return 409 Conflict and provide adjustment options.

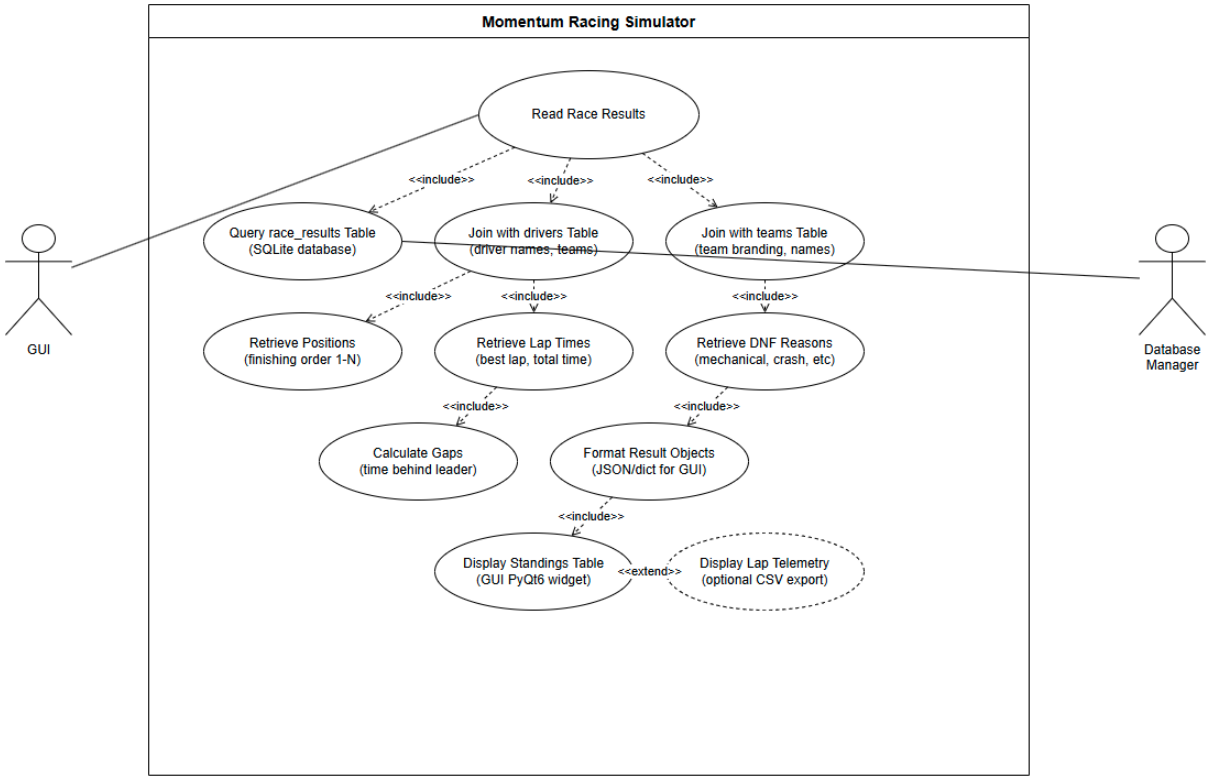
2.4.4.5 Exceptional flow

E1: Database write failure - return 500 and include diagnostic id.

2.4.4.6 Termination

Event status updated, standings update job queued.

2.5 UC-4: Read Race Results



2.5.1 Scope

Retrieve historical race results from the database for display in the GUI, including final classification, lap times, and DNF information.

2.5.2 Description

The system queries completed race records to display historical results. Results include finishing positions, lap times, DNF reasons, and incident details for analysis and comparison.

2.5.3 Actors

- GUI - requests and displays historical race results.

- Database Manager - queries the race_results table and joins with driver/team data.

2.5.4 Flow Description

2.5.4.1 Precondition

Race results have been persisted to the database from previous simulations.

2.5.4.2 Activation

Starts when user navigates to race history screen or requests results for a specific event.

2.5.4.3 Main flow

1. GUI calls DatabaseManager.get_race_results(race_id) or get_championship_results(championship_id).
2. Database Manager executes SELECT query joining race_results, drivers, and teams tables.
3. System returns ordered list of results with driver names, positions, times, and DNF reasons.
4. GUI displays results in standings table format.

2.5.4.4 Alternate flow

A1: No results found for requested race - GUI displays "No results available" message. A2: Partial results due to incomplete race - GUI displays available data and marks as "Race Incomplete".

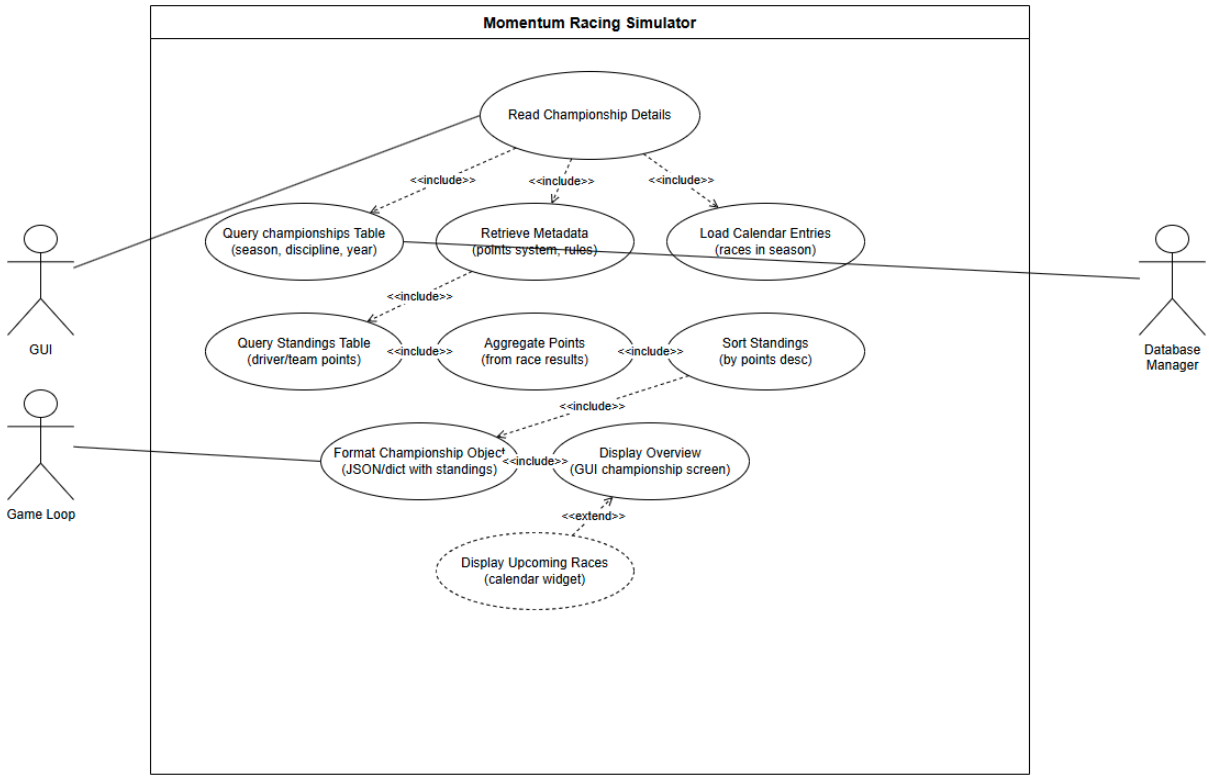
2.5.4.5 Exceptional flow

E1: Database read failure - GUI logs error and displays cached results if available from previous session. E2: Corrupt result data - System logs validation errors and displays partial data with warning.

2.5.4.6 Termination

GUI displays historical results and awaits user navigation.

2.6 UC-5: READ CHAMPIONSHIP DETAILS



2.6.1 Scope

Retrieve championship information including season parameters, calendar of events, current standings, and points allocations across 7 racing disciplines.

2.6.2 Description

The system maintains championship records tracking season-long competitions. Each championship contains a calendar of races, points standings, and discipline-specific rules for the simulation.

2.6.3 Actors

- GUI - displays championship standings, calendar, and season progress.
- Game Loop - manages season progression and championship state (planned feature).
- Database Manager - queries championship, standings, and calendar data.

2.6.4 Flow Description

2.6.4.1 Precondition

Championship has been created with discipline, season year, and initial calendar.

2.6.4.2 Activation

Starts when user navigates to championship screen or game loop initialises a season.

2.6.4.3 *Main flow*

1 GUI or Game Loop requests championship data via DatabaseManager.get_championship(championship_id).

2 System retrieves championship metadata (discipline, year, points system).

3 System queries calendar entries and aggregates standings from race results.

4 GUI displays championship overview with standings table and upcoming races.

2.6.4.4 *Alternate flow*

A1: Championship has no races yet - display empty calendar with option to schedule events.
A2: Standings unavailable - calculate standings from existing race results or display zero points.

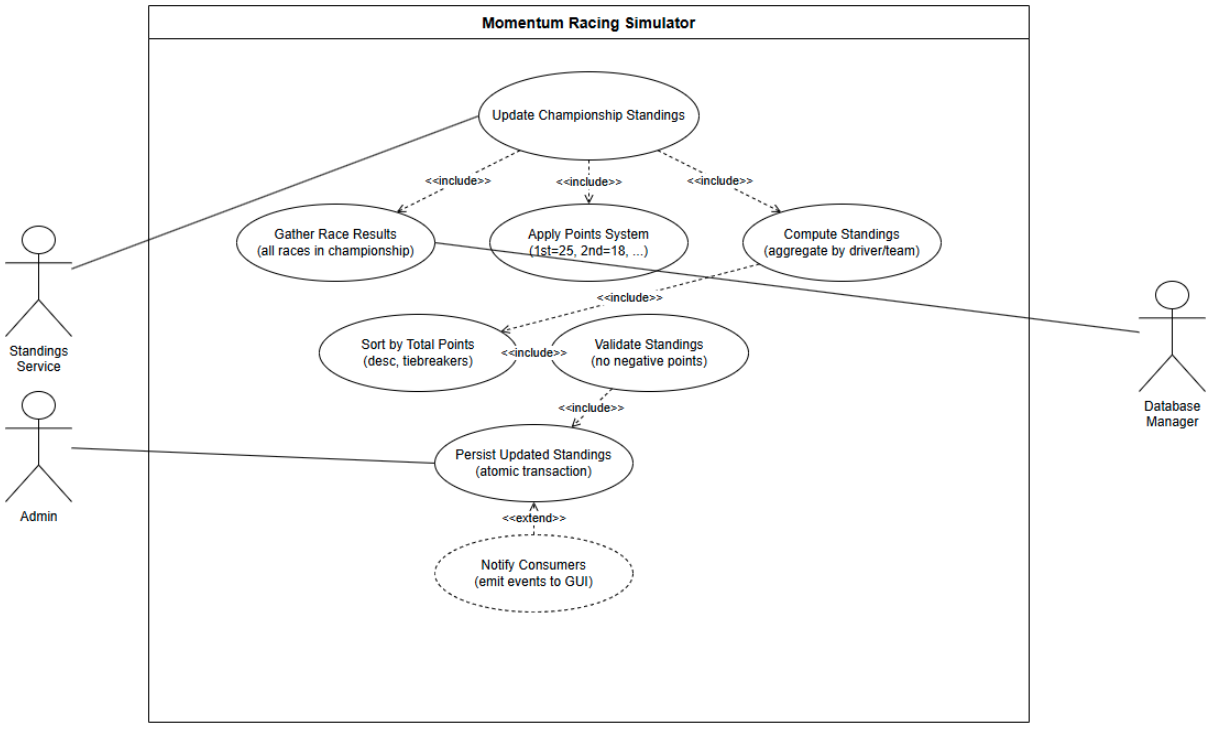
2.6.4.5 *Exceptional flow*

E1: Database query failure - GUI displays error and offers retry or cached data. E2: Invalid championship structure - System logs diagnostic and prompts database verification.

2.6.4.6 *Termination*

GUI displays championship view with standings and calendar, system awaits user interaction.

2.7 UC-6: UPDATE CHAMPIONSHIP STANDINGS



2.7.1 Scope

Recalculate and persist championship standings after results are recorded.

2.7.2 Description

2.7.3 Actors

- Standings Service - triggers recalculation and consumes updated standings.
- Admin - may invoke recalculation manually.

2.7.4 Flow Description

2.7.4.1 Precondition

Championship and results exist.

2.7.4.2 Activation

Starts when results are created or adjusted.

2.7.4.3 Main flow

1 Gather results for the championship.

2 Apply points system and compute standings.

3 Persist updated standings and notify consumers.

2.7.4.4 Alternate flow

A1: Partial update requested - return 202 Accepted with job id for async completion. A2: Concurrent updates - return 409 Conflict with current version.

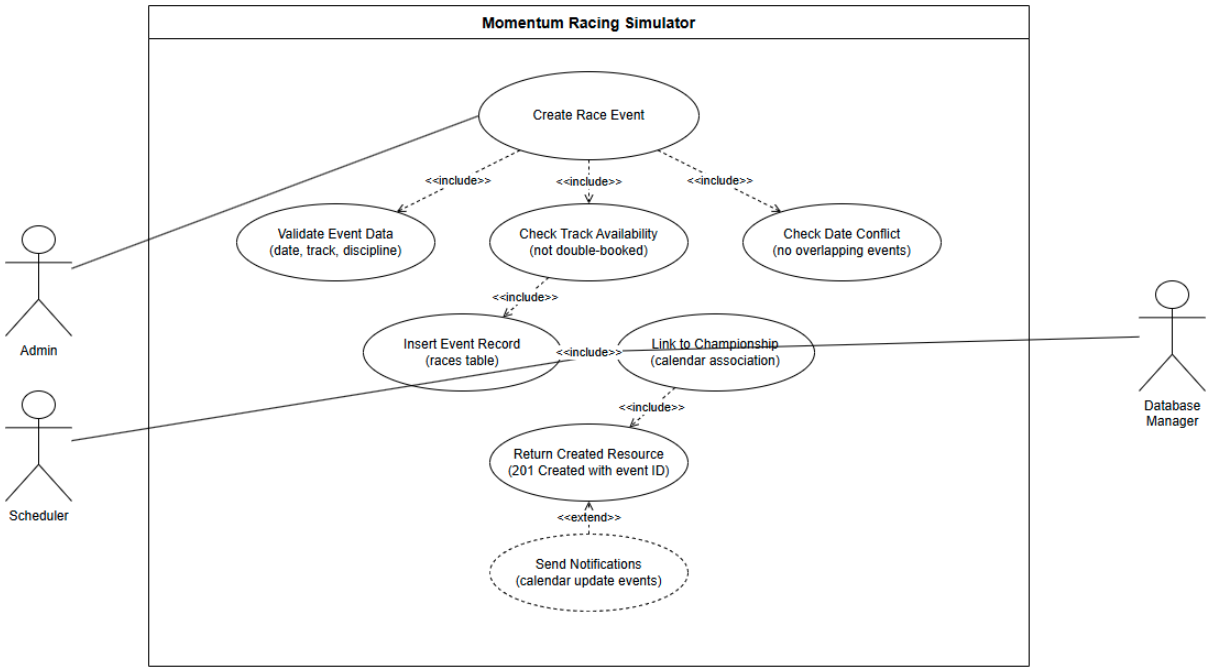
2.7.4.5 Exceptional flow

E1: Points rules missing - return 500 and log for admin attention. E2: Persistence error - roll back and queue retry.

2.7.4.6 Termination

Standings persisted.

2.8 UC-7: CREATE RACE EVENT



2.8.1 Scope

Add a race event to a championship calendar (track, date, configuration).

2.8.2 Description

An admin or scheduler submits an event, the backend validates and persists the event record.

2.8.3 Actors

- Admin - creates and manages race events.
- Scheduler - schedules events programmatically.

2.8.4 Flow Description

2.8.4.1 Precondition

Championship exists and is editable.

2.8.4.2 Activation

Starts when an admin posts new event details.

2.8.4.3 Main flow

1 Validate dates and track availability.

2 Insert event record into races.

3 Return created resource.

2.8.4.4 Alternate flow

A1: Date/track conflict - return 409 Conflict with conflicts and alternatives. A2: Validation warning - return 200 with warnings and allow override.

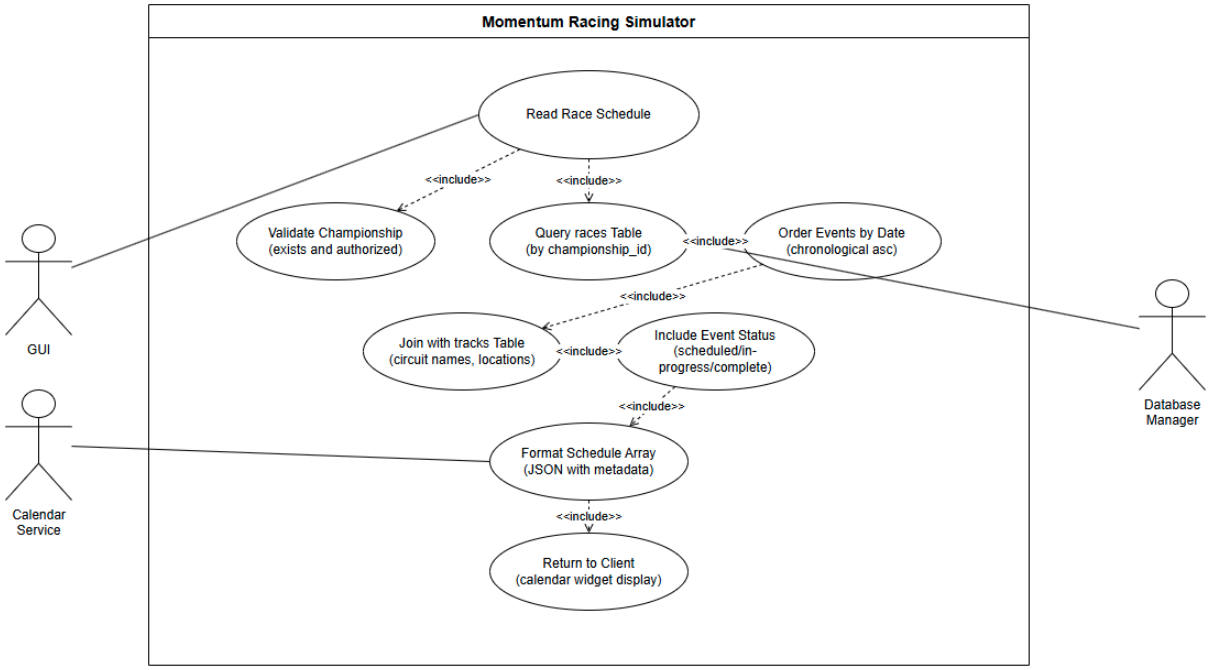
2.8.4.5 Exceptional flow

E1: Database write failure - return 500 and persist request to retry queue. E2: External sync failure - create event in draft and return 202 Accepted with sync pending.

2.8.4.6 Termination

Calendar updated, notifications may be sent.

2.9 UC-8: Read Race Schedule



2.9.1 Scope

Retrieve championship calendar/schedule for UI display.

2.9.2 Description

Return ordered list of events with dates, tracks and statuses.

2.9.3 Actors

- User / UI - requests the schedule for display.
- Calendar Service - synchronises events externally when configured.

2.9.4 Flow Description

2.9.4.1 Precondition

Championship or season exists.

2.9.4.2 Activation

Starts when client opens the calendar screen.

2.9.4.3 Main flow

1 Validate championship/season.

2 Retrieve ordered events and metadata.

3 Return schedule to caller.

2.9.4.4 Alternate flow

A1: Championship not found - return 404 Not Found with guidance to create a championship first. A2: Championship exists but has no events - return empty schedule with a message suggesting adding events.

2.9.4.5 Exceptional flow

E1: Database read error - return 503 Service Unavailable with diagnostic id, offer cached schedule if available. E2: Permission or tenancy error - return 403.

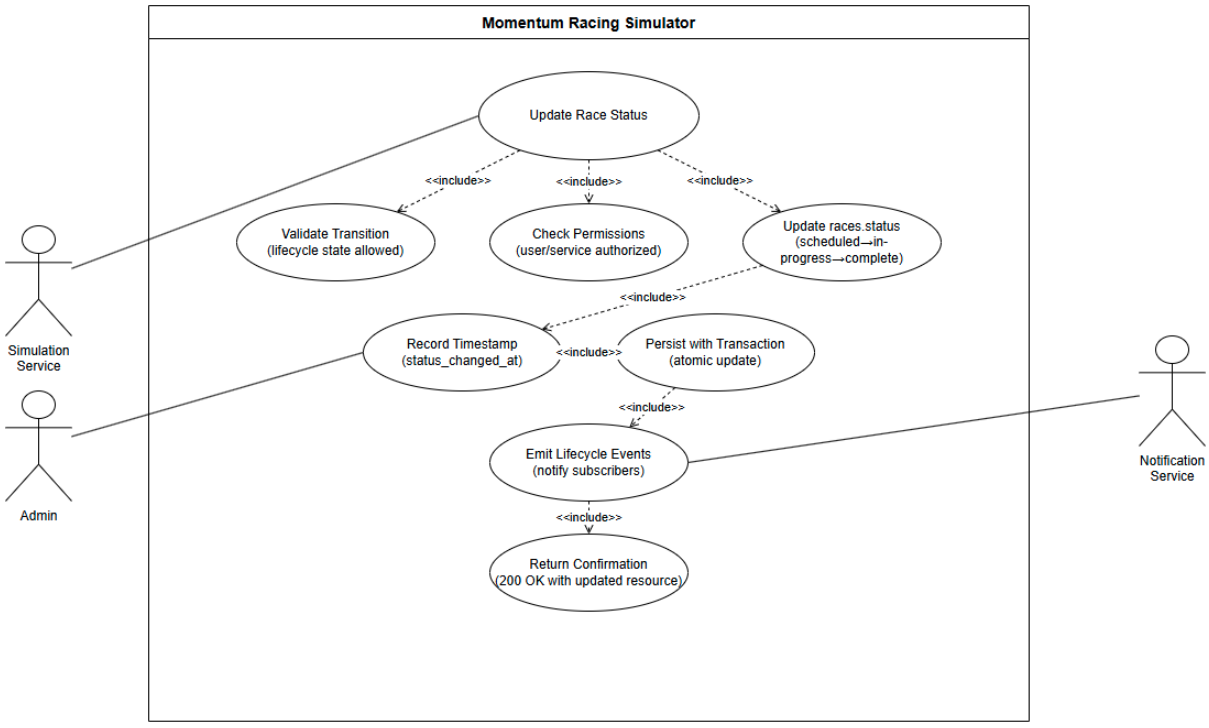
2.9.4.6 Termination

Client displays schedule.

2.9.4.7 Post condition

System awaits next request.

2.10 UC-9: UPDATE RACE STATUS



2.10.1 Scope

Change a race event status (scheduled → in-progress → completed → cancelled).

2.10.2 Description

Update event lifecycle state and notify dependent systems.

2.10.3 Actors

- Simulation/Service - updates status during simulation.
- Admin - may manually change status.
- Notification Service - receives lifecycle events.

2.10.4 Flow Description

2.10.4.1 Precondition

Event exists and caller has permission.

2.10.4.2 Activation

Starts when a service or admin posts a status update.

2.10.4.3 Main flow

1 Validate transition is allowed.

2 Update races.status accordingly.

3 Emit notifications/events.

2.10.4.4 Alternate flow

A1: Invalid lifecycle transition (e.g., completed → scheduled) - return 400 Bad Request with allowed transitions and current state. A2: Caller requests cancellation during in-progress - return 202 Accepted and initiate graceful abort procedures (safety checks, rollback attempts).

2.10.4.5 Exceptional flow

E1: Database write error or optimistic lock fail - return 500 and include current event state, schedule a background reconciliation job. E2: Notification subsystem failure - update status but record notification failure for later retry, return 200 with notificationsPending: true.

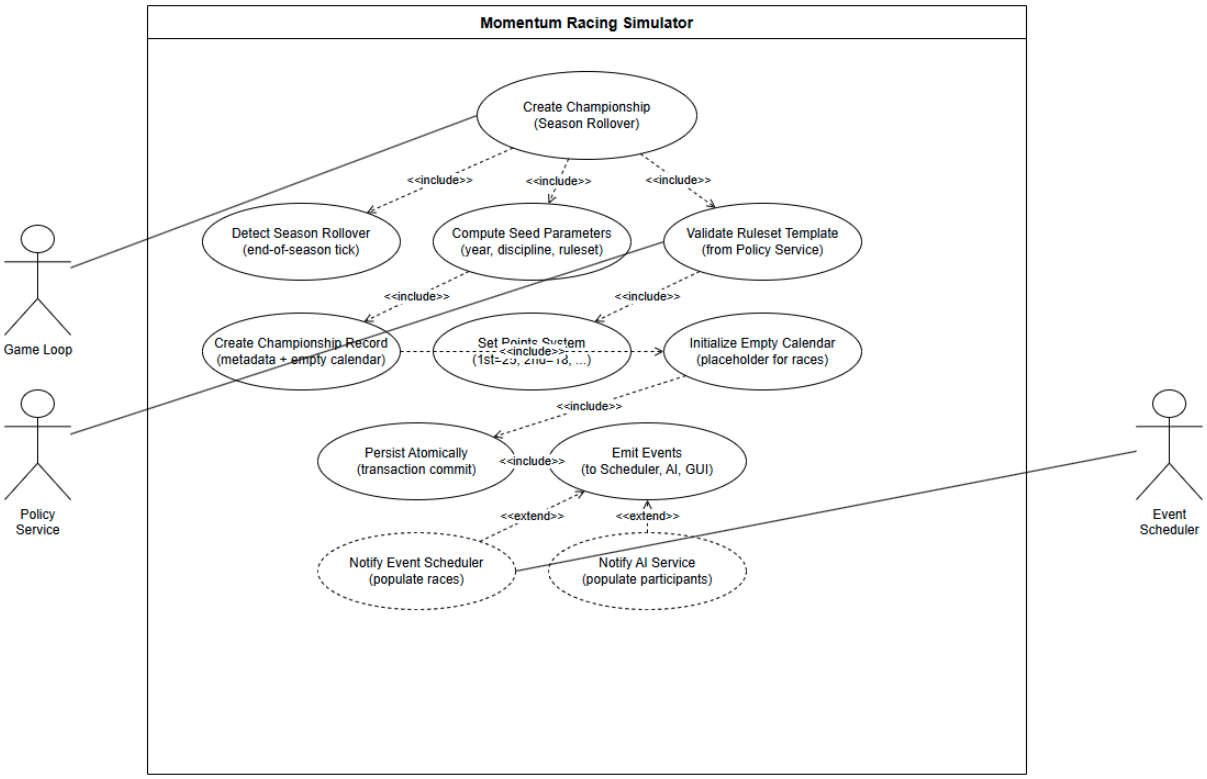
2.10.4.6 Termination

Event status updated and notifications sent.

2.10.4.7 Post condition

System awaits next lifecycle event.

2.11 UC-10: CREATE CHAMPIONSHIP



2.11.1 Scope

Create a new championship/season automatically on season rollover, with base rules and an empty calendar.

2.11.2 Description

On season rollover the Game Loop computes the next season parameters, instantiates a championship record with a ruleset and calendar placeholder, persists it, and notifies downstream systems (scheduling, AI, UI) that a new season is available.

2.11.3 Actors

- Game Loop / Scheduler - triggers automatic season rollover and championship creation.
- Policy Service - validates or supplies rulesets.
- Event Scheduler - consumes the new championship to populate events.

2.11.4 Flow Description

2.11.4.1 Precondition

End-of-season tick reached, ruleset templates and calendar seeding configuration are available.

2.11.4.2 Activation

Triggered automatically by the Game Loop on season rollover (no manual admin intervention required).

2.11.4.3 Main flow

1 Game Loop determines season rollover has occurred and computes seed parameters (year, discipline, ruleset template).

2 System requests/validates the ruleset from Policy Service.

3 System creates the championship record with initial metadata and an empty calendar.

4 System persists the championship atomically and emits events to Event Scheduler, AI and UI.

2.11.4.4 Alternate flow

A1: Ruleset validation fails - select fallback template and continue or mark championship as requiresReview and emit admin notification.

2.11.4.5 Exceptional flow

E1: Database write failure - rollback and schedule a retry, emit diagnostic id to monitoring and place a recovery job in the maintenance queue.

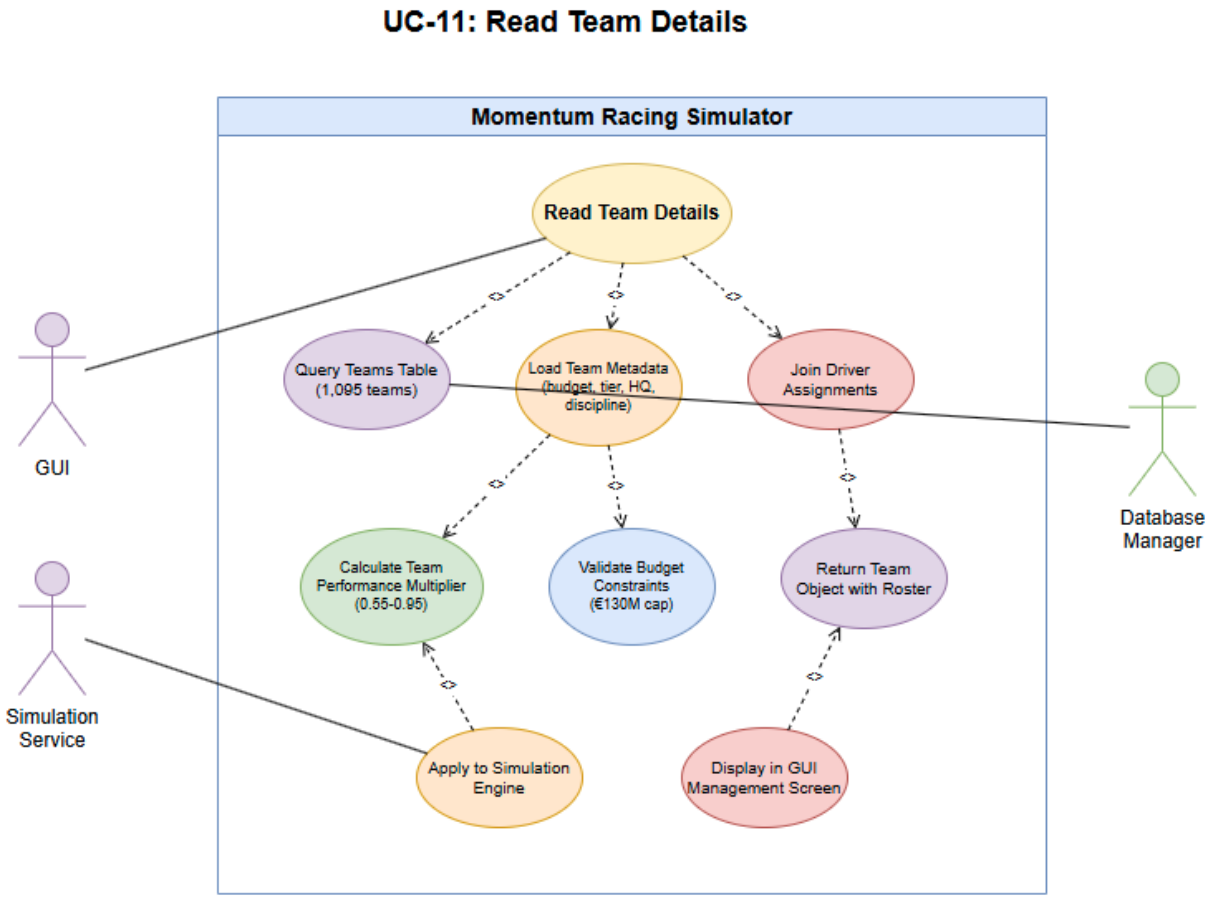
2.11.4.6 Termination

Championship record created and downstream systems notified.

2.11.4.7 Post condition

Event Scheduler and AI begin populating events and participants for the new season.

2.12 UC-11: Read Team Details



2.12.1 Scope

Retrieve complete team profile including budget (€130M cap for top-tier Single Seater), tier classification, headquarters, discipline specialisation, and assigned drivers.

2.12.2 Description

The system maintains 1,095 team records across 7 racing disciplines. Each team has budget constraints, tier classification (tier1/tier2/tier3), and performance multipliers that affect simulation outcomes.

2.12.3 Actors

- GUI - displays team information on management screens.
- Simulation Service - requests team data to calculate performance multipliers for race simulation.
- Database Manager - queries the teams table and joins with driver assignments.

2.12.4 Flow Description

2.12.4.1 Precondition

Team records exist in database with complete attribute sets.

2.12.4.2 Activation

Starts when GUI loads team management screen or simulation engine initialises a race.

2.12.4.3 Main flow

1. GUI or Simulation Service calls DatabaseManager.get_team(team_id) or get_teams_by_discipline(discipline).
2. Database Manager executes SELECT query against teams table.
3. System retrieves team metadata including budget, tier, headquarters, and discipline.
4. For simulation, engine calculates team performance multiplier (0.55 to 0.95) based on tier and budget.
5. GUI displays team profile or simulation applies multipliers to driver performance.

2.12.4.4 Alternate flow

A1: Invalid team_id - return None and GUI displays "Team not found" error. A2: Team has no assigned drivers - display team info with empty roster.

2.12.4.5 Exceptional flow

E1: Database read failure - GUI logs error to logs/ and displays connection error with retry option. E2: Missing team attributes - System uses default values (tier2, €100M budget) and logs warning.

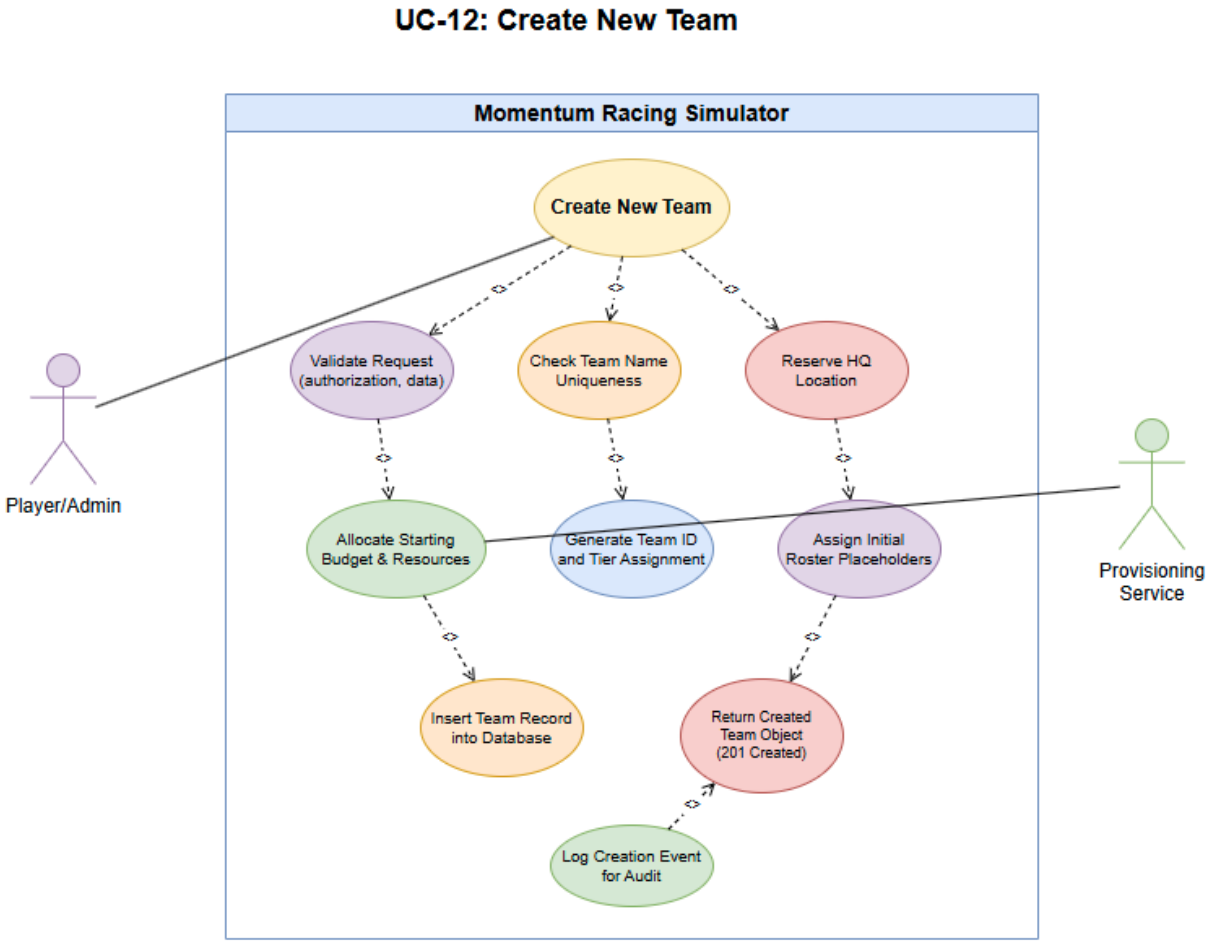
2.12.4.6 Termination

GUI displays team screen or simulation proceeds with team performance calculations.

2.12.4.7 Post condition

System awaits next team query or user action.

2.13 UC-12: Create New Team



2.13.1 Scope

Create a new team record (initial roster, budget, HQ).

2.13.2 Description

Player or admin creates a team, the system allocates starting assets or placeholders.

2.13.3 Actors

- Player/Admin - initiates team creation.
- Provisioning Service - reserves starting resources.

2.13.4 Flow Description

2.13.4.1 Precondition

Requester authorised, required seed data available.

2.13.4.2 Activation

Starts when player/admin submits team creation.

2.13.4.3 Main flow

1 Validate request and reserve resources.

2 Insert team record and assign initial roster/placeholders.

3 Return created team.

2.13.4.4 Alternate flow

A1: Resource conflict (e.g., team name or HQ occupied) - return 409 Conflict with suggested alternatives and actions to resolve. A2: Missing seed data - return 422 Unprocessable Entity with details on required fields.

2.13.4.5 Exceptional flow

E1: Database transaction failure - return 500 and log the full request for replay. E2: External provisioning failure (e.g., storage) - create the team in pending state and notify admin.

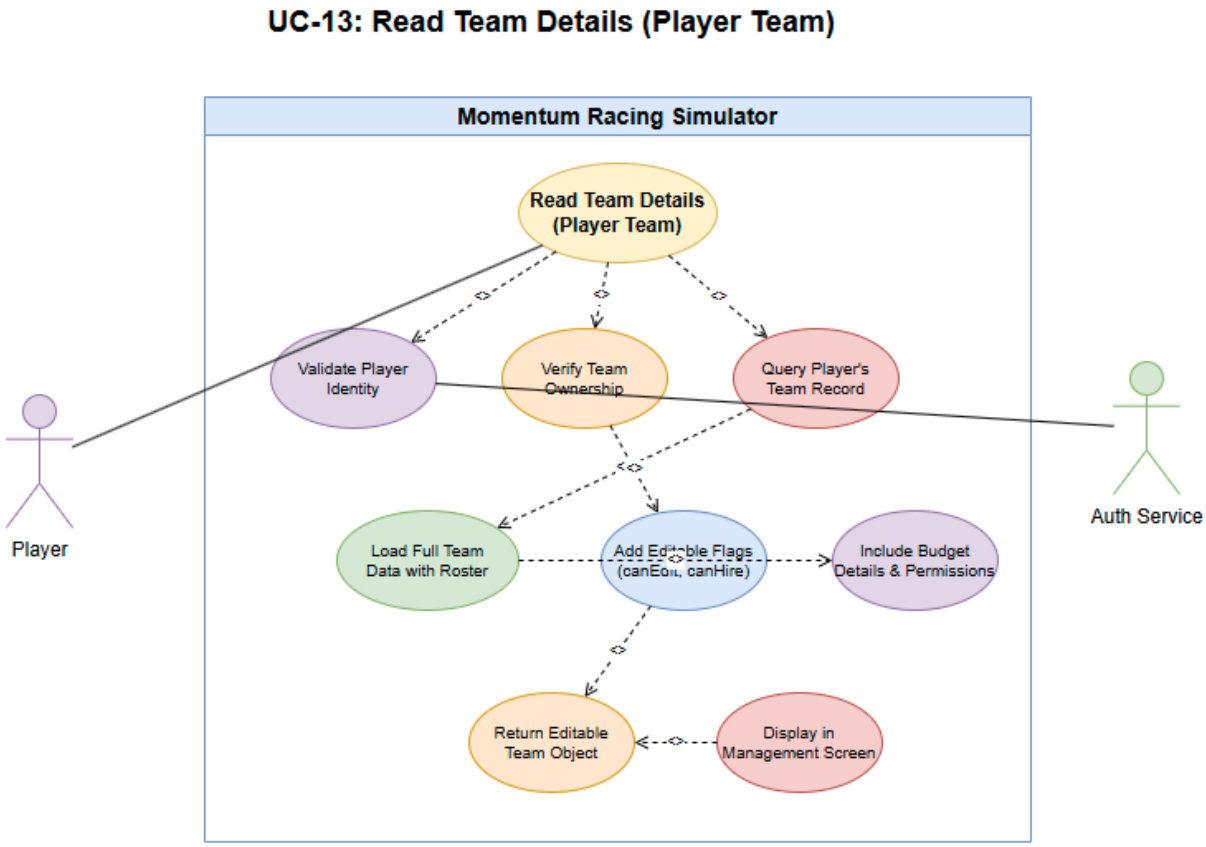
2.13.4.6 Termination

Team visible in lists.

2.13.4.7 Post condition

System awaits next management action.

2.14 UC-13: Read Team Details (Player Team)



2.14.1 Scope

Return player's own team with editable flags and affordances.

2.14.2 Description

Provide editable team data for the player's management screen.

2.14.3 Actors

- Player - owner of the team and primary editor.
- Auth Service - validates ownership and permissions.

2.14.4 Flow Description

2.14.4.1 Precondition

Player has a team.

2.14.4.2 Activation

Starts when player opens their team management screen.

2.14.4.3 Main flow

1 Validate player's identity and team ownership.

2 Return team data with editable flags.

2.14.4.4 Alternate flow

A1: Player has no team - return 404 Not Found with guidance to create or join a team and links to relevant endpoints.

2.14.4.5 Exceptional flow

E1: Database read error - return 503 Service Unavailable and include a diagnostic id. E2: Authorisation check failure - return 403.

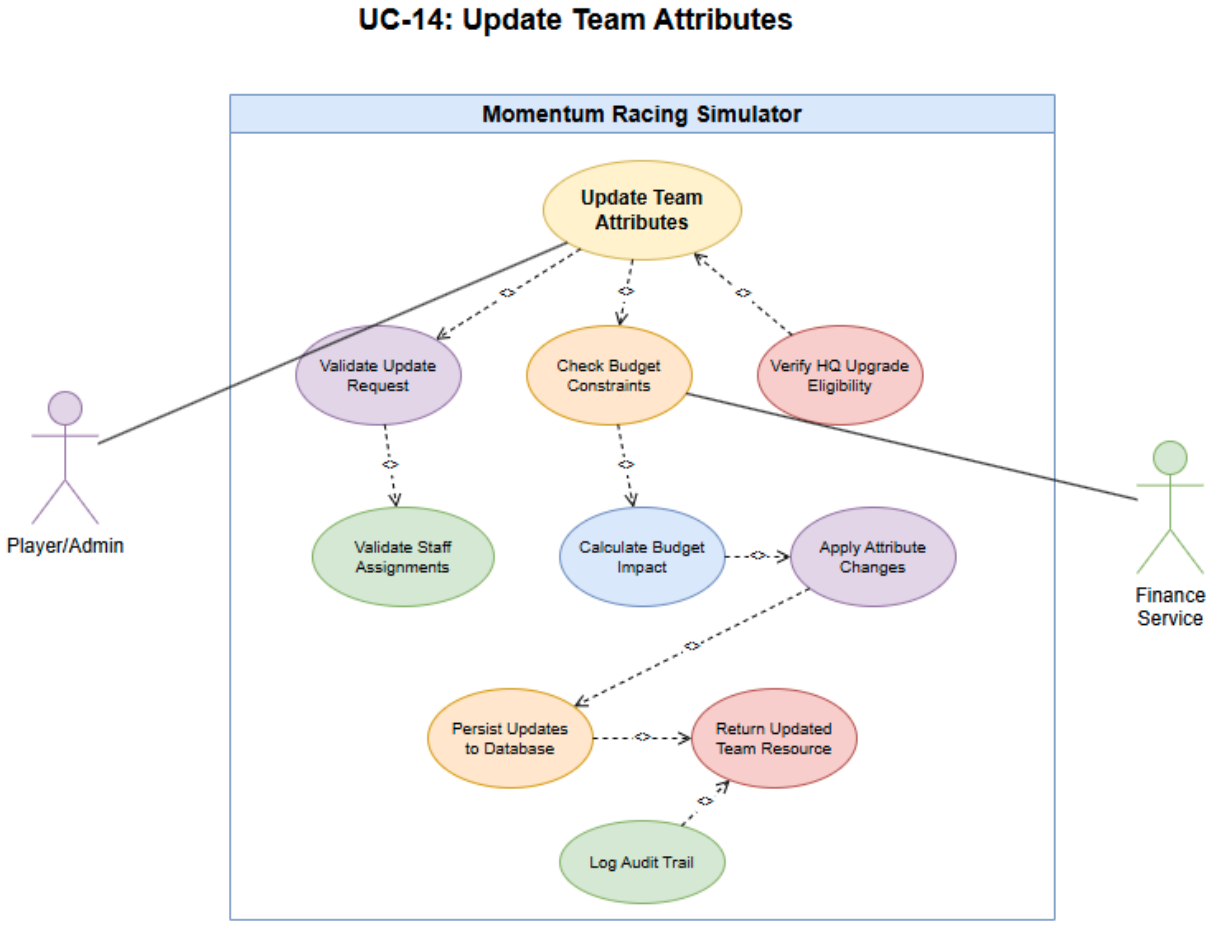
2.14.4.6 Termination

Player views editable team screen.

2.14.4.7 Post condition

System awaits further edits.

2.15 UC-14: Update Team Attributes



2.15.1 Scope

Modify team attributes (budget allocation, HQ upgrades, staff assignments).

2.15.2 Description

Player or admin updates team settings which are validated and persisted.

2.15.3 Actors

- Player/Admin - submits updates.
- Finance Service - validates budget changes.

2.15.4 Flow Description

2.15.4.1 Precondition

Requester authorised and has sufficient resources.

2.15.4.2 Activation

Starts when client submits team updates.

2.15.4.3 Main flow

1 Validate changes and budget.

2 Persist updates.

3 Return updated resource.

2.15.4.4 Alternate flow

A1: Insufficient funds detected - return 422 Unprocessable Entity with required amount and suggestions (sponsor, loan, reduce spend). A2: Concurrent update to budget - return 409 Conflict with current balance and retry instructions.

2.15.4.5 Exceptional flow

E1: Database write/commit failure - return 500 and enqueue the update for manual reconciliation. E2: Audit/logging service failure - persist the change but flag audit missing, return 200 with auditPending: true.

2.15.4.6 Termination

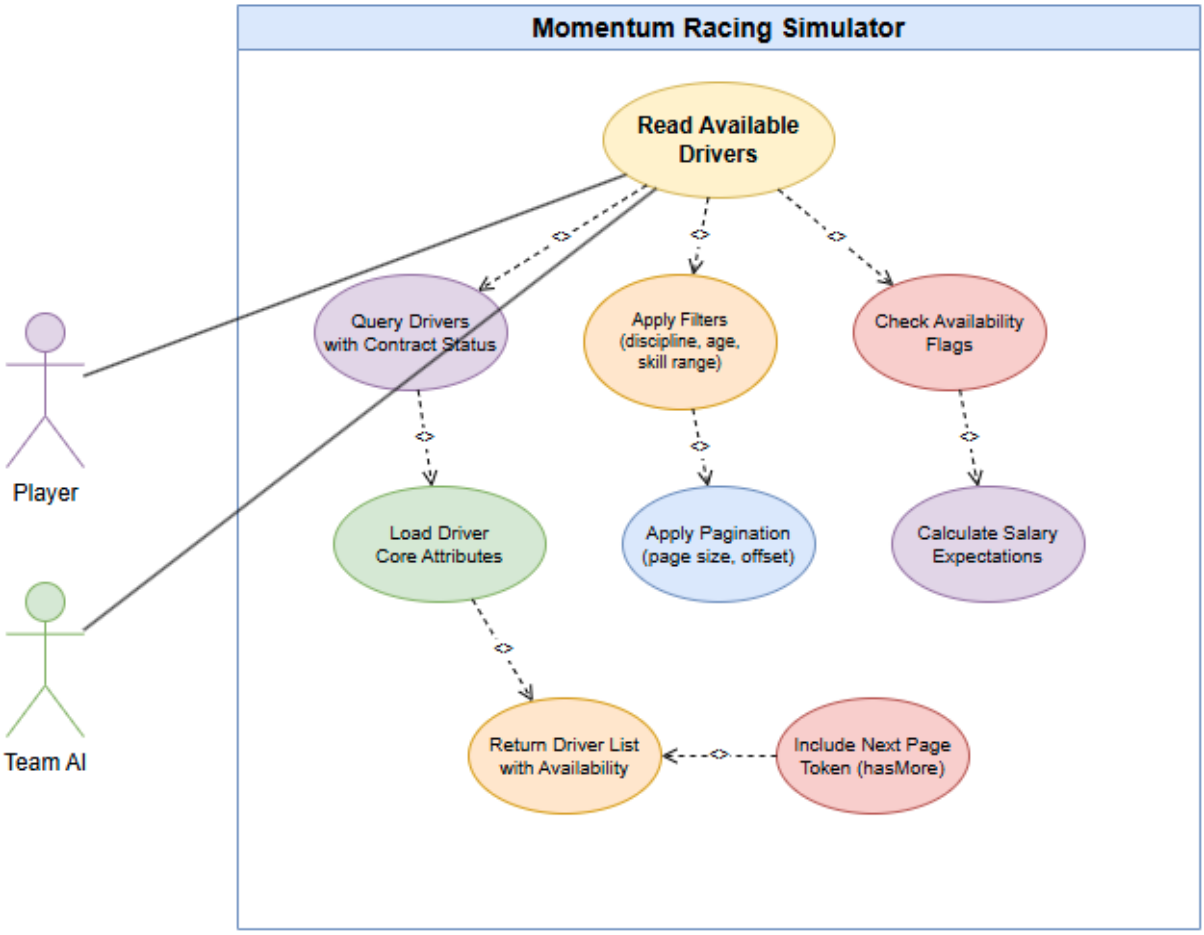
Changes reflected on subsequent reads.

2.15.4.7 Post condition

System awaits next request.

2.16 UC-15: Read Available Drivers

UC-15: Read Available Drivers



2.16.1 Scope

Return list of uncontracted or transfer-available drivers for recruitment.

2.16.2 Description

Provide filtered and paginated driver pool for the market UI.

2.16.3 Actors

- Player - browses and filters available drivers.
- Team AI - automated consumer of driver listings.

2.16.4 Flow Description

2.16.4.1 Precondition

Driver pool exists.

2.16.4.2 Activation

Starts when user opens driver market.

2.16.4.3 Main flow

1 Apply filters/pagination.

2 Return driver list with availability flags.

2.16.4.4 Alternate flow

A1: Filter parameters return no matches - return 200 OK with empty list and suggestions to relax filters. A2: Partial data due to pagination limits - include hasMore: true and next-page token.

2.16.4.5 Exceptional flow

E1: Database query failure - return 503 Service Unavailable with diagnostic id, if cached pool exists return cached results marked stale: true. E2: Indexing/search service failure - fall back to simple DB query and note reduced fidelity.

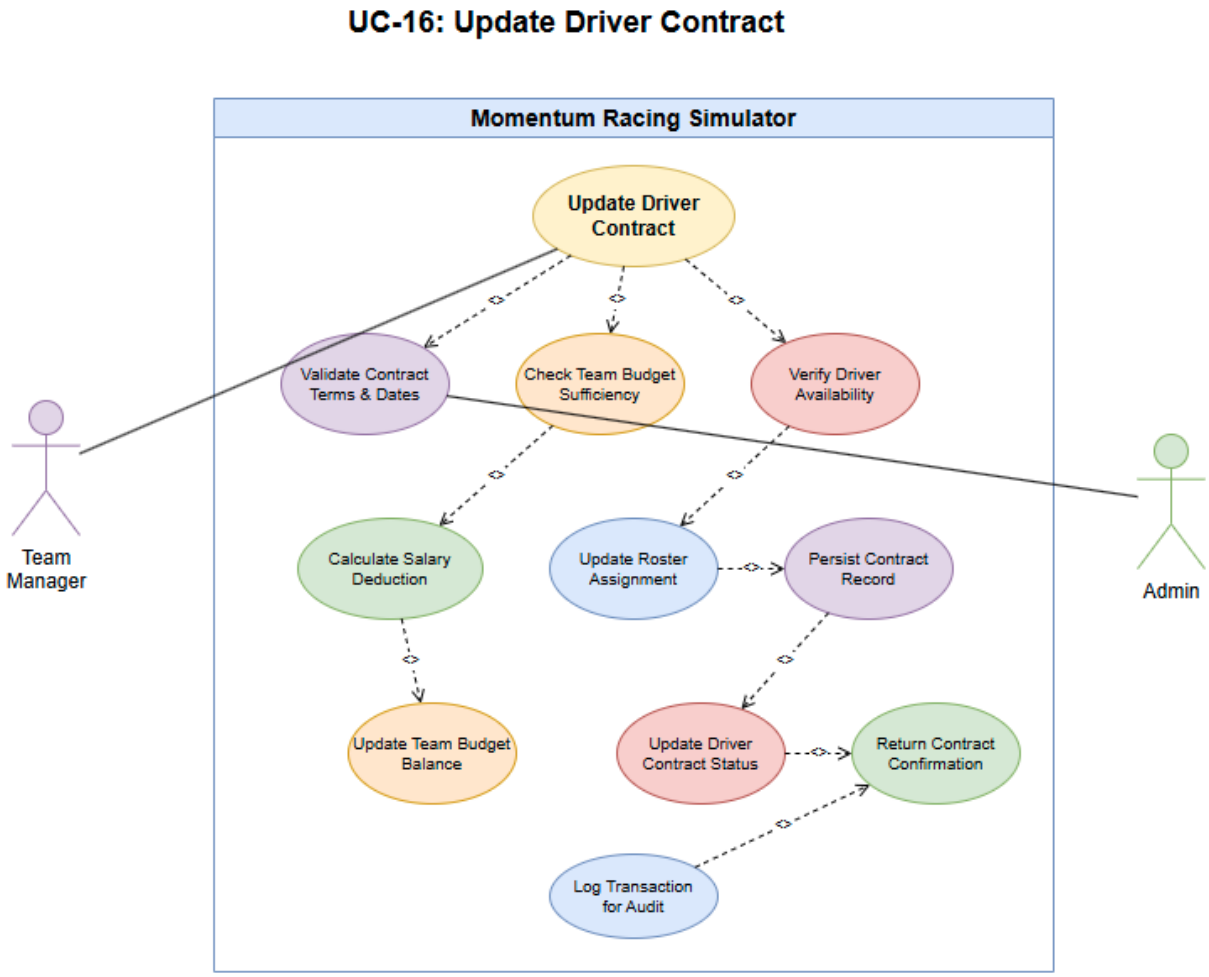
2.16.4.6 Termination

Client displays candidate drivers.

2.16.4.7 Post condition

System awaits next query.

2.17 UC-16: Update Driver Contract



2.17.1 Scope

Sign, renew or terminate driver contracts and update team roster and budget.

2.17.2 Description

Persist contract terms and adjust availability and team budgets.

2.17.3 Actors

- Team Manager - negotiates and confirms driver contracts.
- Admin - enforces or modifies contract rules when required.

2.17.4 Flow Description

2.17.4.1 Precondition

Driver and team exist, negotiation outcome determined.

2.17.4.2 Activation

Starts when contract terms are finalised and submitted.

2.17.4.3 Main flow

1 Validate budget and contract rules.

2 Persist contract and update roster/budget.

3 Return confirmation.

2.17.4.4 Alternate flow

A1: Budget shortfall during contract update - return 422 with required funding and options (defer, partial contract, renegotiate). A2: Contract already signed by another party - return 409 Conflict with current contract details.

2.17.4.5 Exceptional flow

E1: Database write error - rollback, return 500 and include reference id for support. E2: Payment gateway or external finance service failure - abort contract update, return 502 Bad Gateway and queue retry.

2.17.4.6 Termination

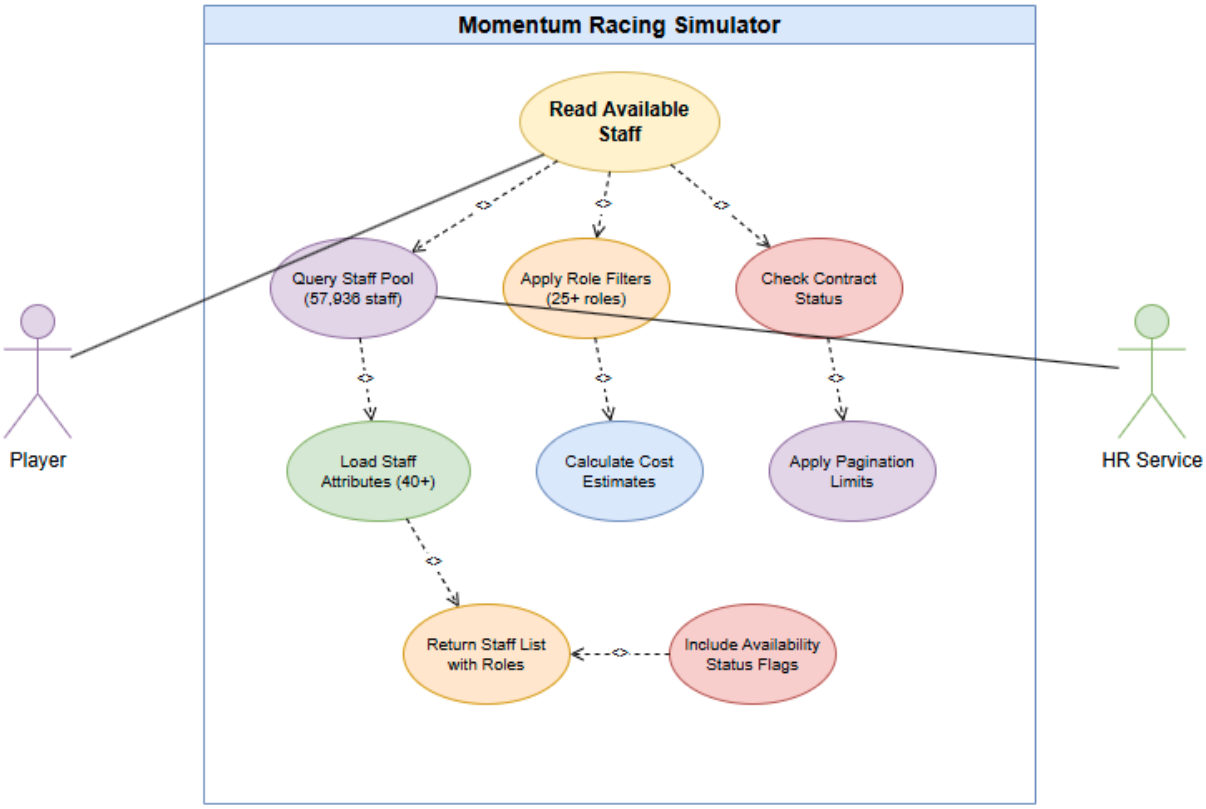
Driver assigned and availability updated.

2.17.4.7 Post condition

System awaits next contract action.

2.18 UC-17: Read Available Staff

UC-17: Read Available Staff



2.18.1 Scope

Return list of available staff for hiring with roles and modifiers.

2.18.2 Description

Provide paginated staff pool for recruitment screens.

2.18.3 Actors

- Player - requests staff listings.
- HR Service - provides staff filters and pools.

2.18.4 Flow Description

2.18.4.1 Precondition

Staff pool exists.

2.18.4.2 Activation

Starts when player opens staff hiring screen.

2.18.4.3 Main flow

1 Apply filters and paginate.

2 Return staff list with core stats and cost.

2.18.4.4 Alternate flow

A1: No staff matching filters - return 200 OK with empty list and filter suggestions. A2: Partial availability (some staff on contract) - return available subset with availability: limited flag.

2.18.4.5 Exceptional flow

E1: Database or search index failure - return 503 Service Unavailable and provide cached results if available. E2: Data corruption in staff records detected - return 500 and set a maintenance flag.

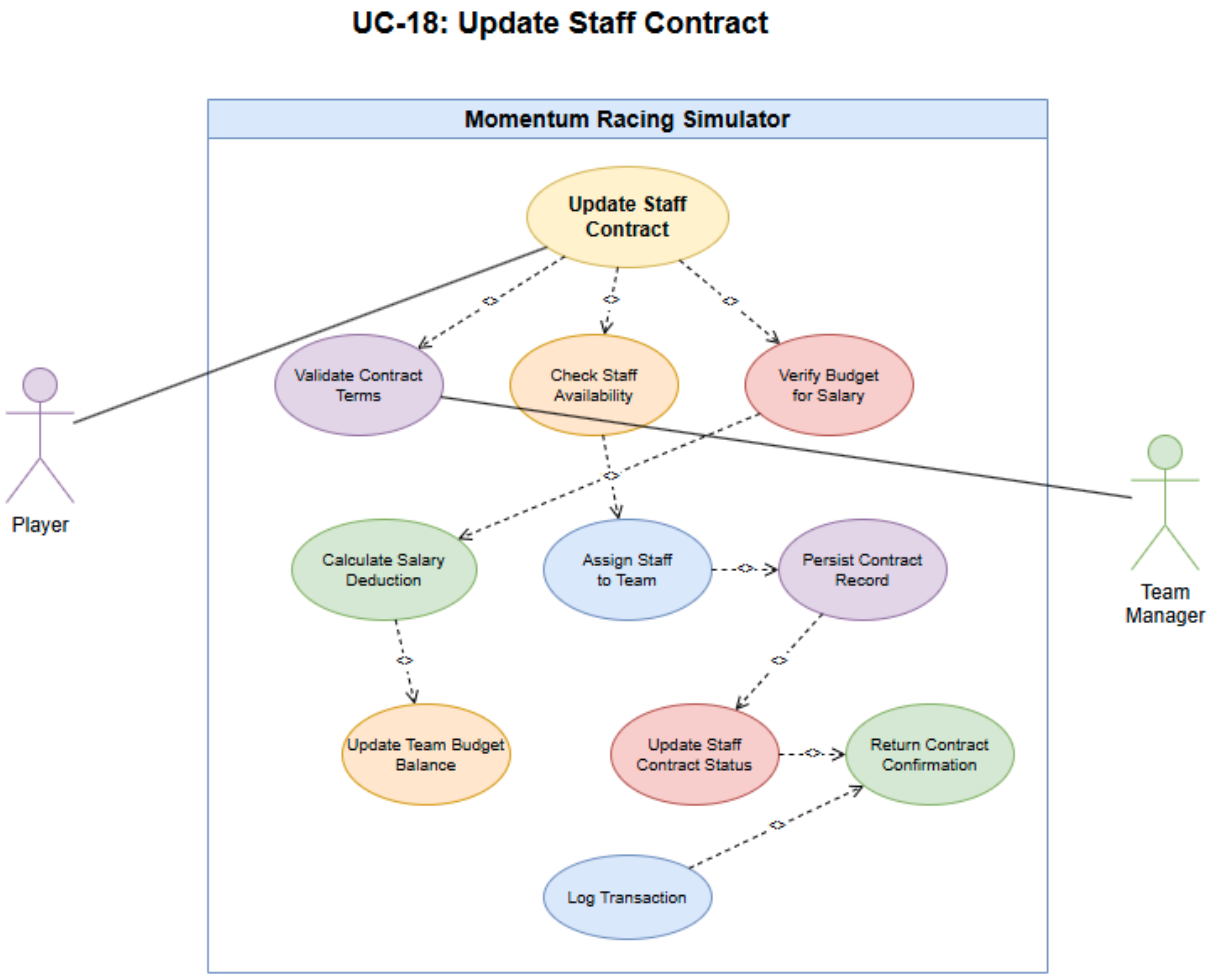
2.18.4.6 Termination

Client displays staff candidates.

2.18.4.7 Post condition

System awaits next request.

2.19 UC-18: Update Staff Contract



2.19.1 Scope

Hire, fire or renegotiate staff contracts and persist changes to team resources.

2.19.2 Description

Assign staff to teams, record salary and update budgets.

2.19.3 Actors

- Player - initiates hiring/renegotiation.
- Team Manager - manages staff assignments.

2.19.4 Flow Description

2.19.4.1 Precondition

Staff profile exists and is available.

2.19.4.2 Activation

Starts when hiring/renegotiation action is confirmed.

2.19.4.3 Main flow

1 Validate terms and budget.

2 Persist contract and update team state.

3 Return confirmation.

2.19.4.4 Alternate flow

A1: Insufficient funds - return 422 Unprocessable Entity with required funding detail and rollback any provisional assignments. A2: Candidate no longer available at confirmation - return 409 Conflict and provide alternatives.

2.19.4.5 Exceptional flow

E1: Database write failure - rollback and return 500 with support reference. E2: Concurrent contract update detected - return 409 with current contract state and instructions to retry.

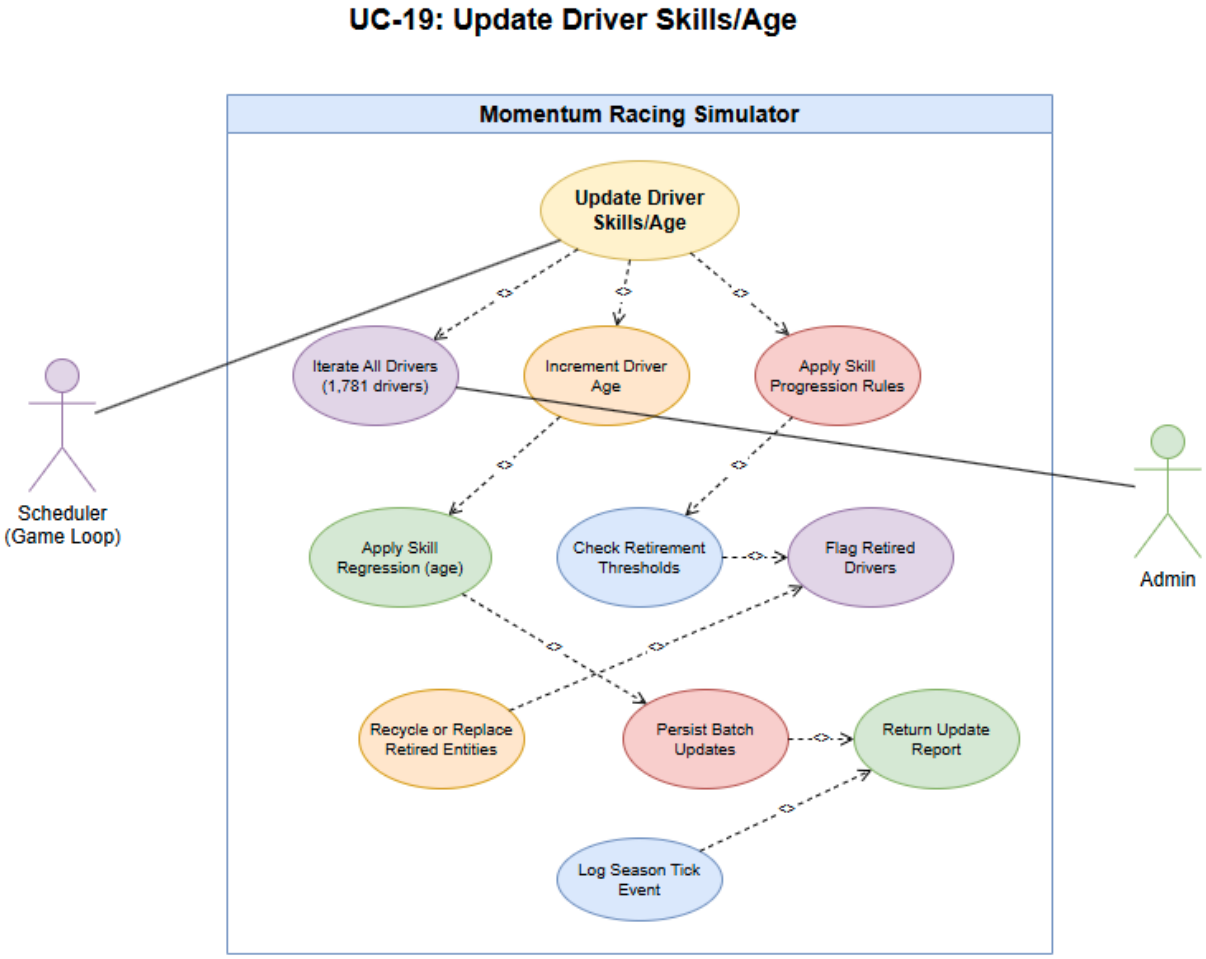
2.19.4.6 Termination

Staff assigned and salary deducted from budget.

2.19.4.7 Post condition

System awaits further HR actions.

2.20 UC-19: Update Driver Skills/Age



2.20.1 Scope

Periodic update of driver attributes (ageing, progression/regression) and handle retirements/recycling.

2.20.2 Description

Perform end-of-season or scheduled updates to driver skills and ages, mark retirees and optionally regenerate/recycle entities.

2.20.3 Actors

- Scheduler (Game Loop) - triggers seasonal maintenance jobs.
- Admin - may invoke manual progression runs.

2.20.4 Flow Description

2.20.4.1 Precondition

Driver population present.

2.20.4.2 Activation

Starts at season end or scheduled tick.

2.20.4.3 Main flow

1 System iterates drivers and adjusts skills and ages.

2 Flag retired drivers and recycle or replace per policy.

3 Persist updates.

2.20.4.4 Alternate flow

A1: Emergency update due to injury or incident - apply immediate decrement to affected driver(s), mark as underReview, and notify teams. A2: Player-triggered training boost - apply progression rules and mark a cooldown period.

2.20.4.5 Exceptional flow

E1: Batch update failure during season tick - return 500, log affected driver ids and schedule a retry job. E2: Constraint violation (e.g., negative attribute) - abort update for affected records and flag for manual review.

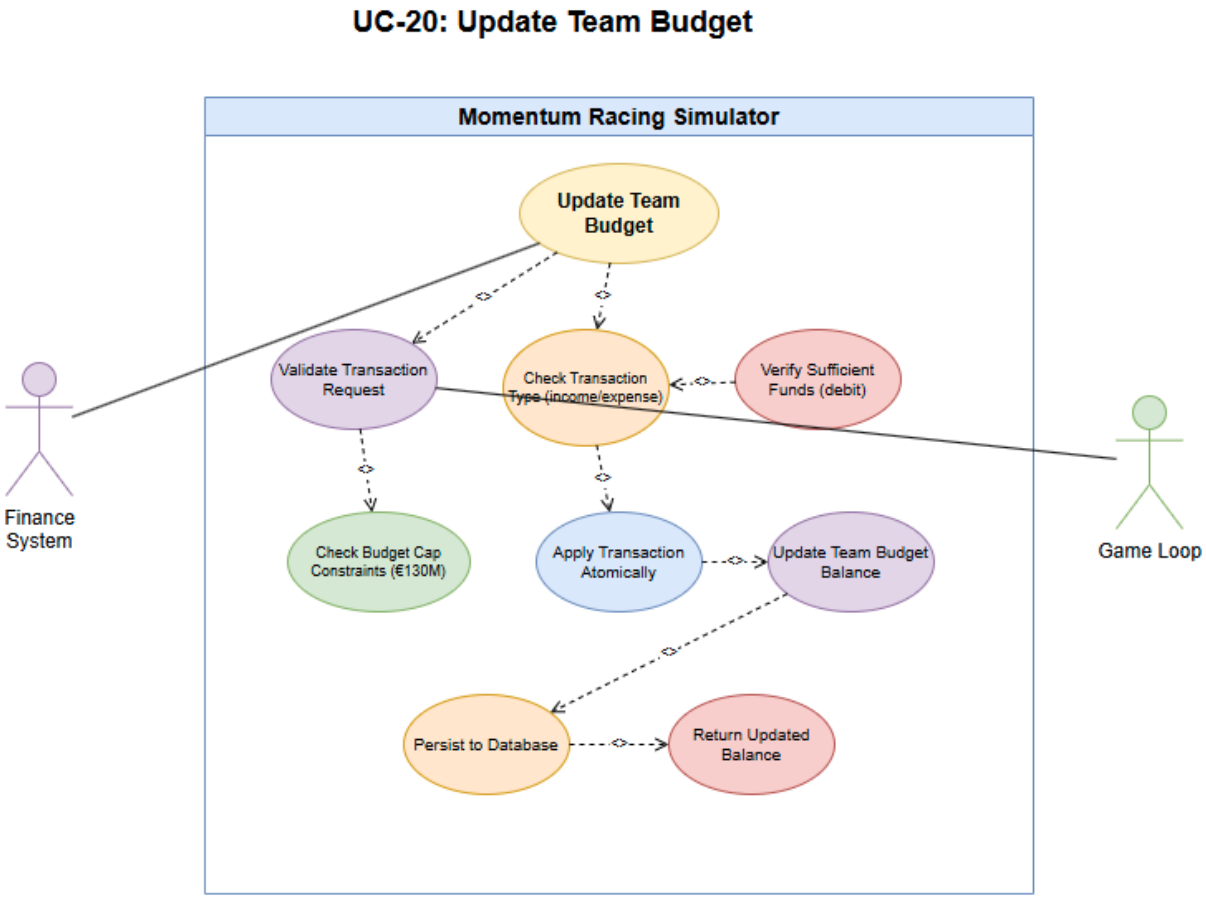
2.20.4.6 Termination

Driver records updated for next season.

2.20.4.7 Post condition

System awaits next scheduled update.

2.21 UC-20: Update Team Budget



2.21.1 Scope

Adjust team budget for income/expenses, sponsor payouts and race fees.

2.21.2 Description

Apply transactions affecting a team's budget and persist them.

2.21.3 Actors

- Finance System - posts transactions affecting team budgets.
- Game Loop - triggers periodic income/expense events.

2.21.4 Flow Description

2.21.4.1 Precondition

Team exists.

2.21.4.2 Activation

Starts when a transaction affecting budget occurs.

2.21.4.3 Main flow

1 Validate transaction and atomicity.

2 Update team budget.

3 Return updated balance.

2.21.4.4 Alternate flow

A1: Insufficient funds for debit - reject the transaction with 402 Payment Required (or 422) and suggest corrective actions (sponsor, loan, adjust expenses). A2: Pending transactions that affect available balance - return 409 Conflict with pending transaction list.

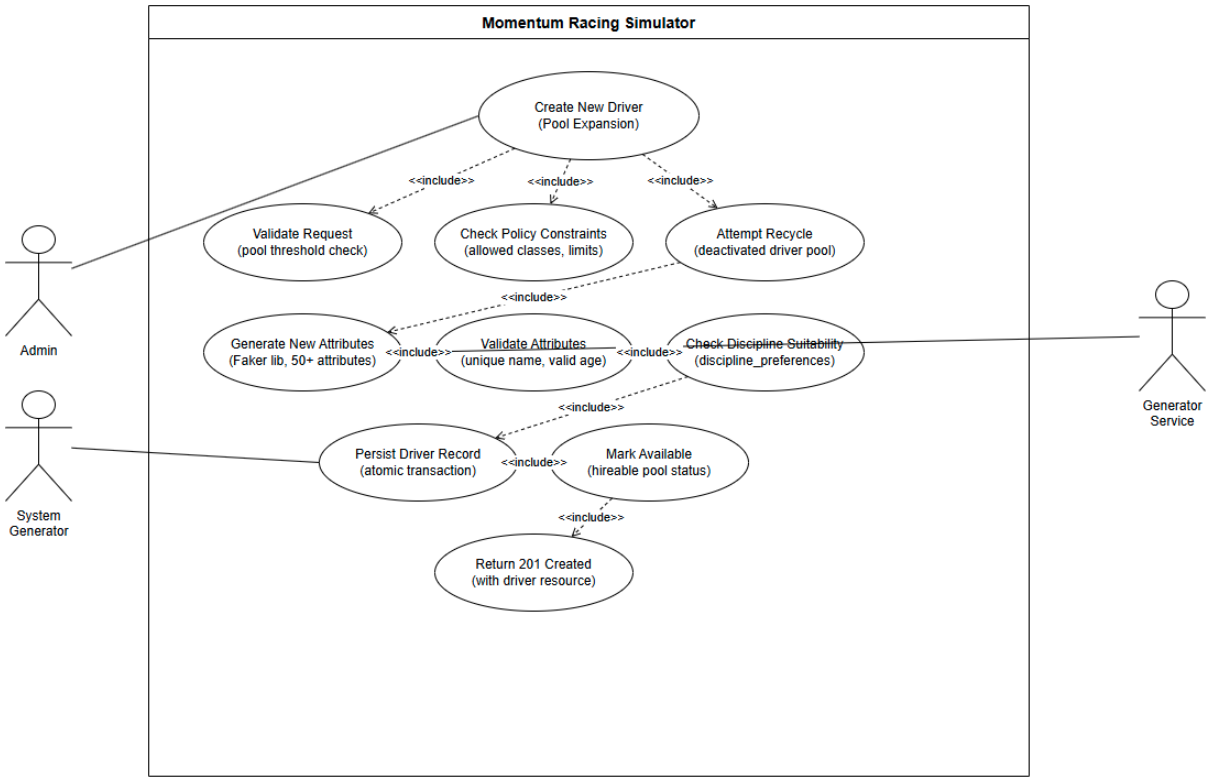
2.21.4.5 Exceptional flow

E1: Database write failure - return 500 and enqueue transaction for replay.

2.21.4.6 Termination

Team budget updated and awaiting next transaction.

2.22 UC-21: Create New Driver



2.22.1 Scope

Add a driver to the hireable pool by recycling an existing placeholder or generating a new driver profile when pool thresholds require expansion.

2.22.2 Description

Maintain and expand the hireable driver pool to support roster growth, replacement of retired drivers, and content variety. The system prefers recycling deactivated driver records when suitable, otherwise, the Generator Service creates a new driver profile validated against uniqueness and configured policy constraints.

2.22.3 Actors

- Admin - requests manual creation.
- System - scheduled or policy-driven generator.
- Generator Service - produces seeded attribute payloads for new drivers.

2.22.4 Flow Description

2.22.4.1 Precondition

Driver pool is below configured threshold or an explicit admin request to create a driver is received, seeding and generation configuration are available.

2.22.4.2 Activation

Triggered by POST /api/drivers/generate from an admin or by an automated roster expansion job.

2.22.4.3 Main flow

1. Validate request and policy constraints (pool thresholds, allowed classes).
2. Attempt to select a recycle candidate from the deactivated pool, if none available, call Generator Service to create attributes.
3. Validate generated attributes (unique name, valid age, discipline suitability).
4. Persist the driver record inside a transaction and mark availability.
5. Return 201 Created with the driver resource.

2.22.4.4 Alternate flow

A1: Recycle candidate unavailable - generate new attributes and continue. A2: Minor validation warnings - persist with a warnings field and return 201 Created. A3: Name or identity conflict - mutate the attribute (rename) and retry up to configured attempts, on exhaustion return 409 Conflict with guidance.

2.22.4.5 Exceptional flow

E1: Database write failure - rollback and return 500 with diagnostic id for support. E2: Generator Service unavailable - return 503 Service Unavailable and schedule a retry job. E3: Policy or configuration violation - return 422 Unprocessable Entity with policy details.

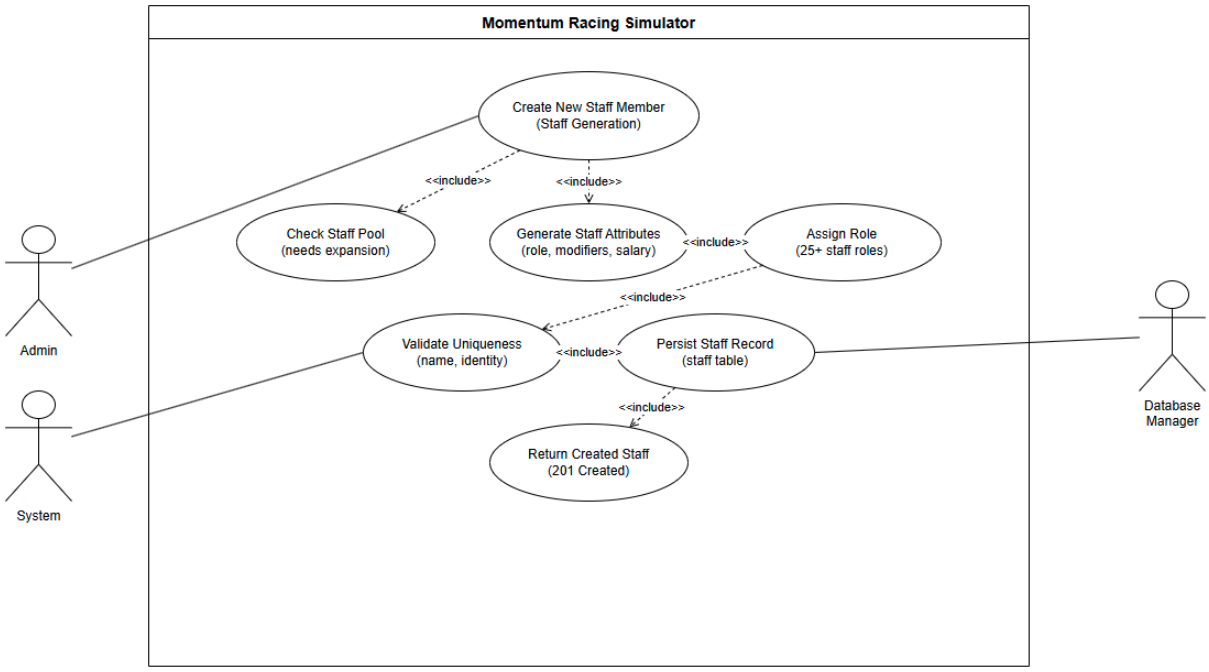
2.22.4.6 Termination

Driver record created and visible in the hireable pool.

2.22.4.7 Post condition

Driver returned in subsequent listing endpoints and system awaits next generation task.

2.23 UC-22: CREATE NEW STAFF MEMBER (STAFF GENERATION)



2.23.1 Scope

Generate new staff when required (optional).

2.23.2 Description

Create or recycle staff records for hireable pool expansion.

2.23.3 Actors

- Admin/System - triggers staff generation or pool maintenance.

2.23.4 Flow Description

2.23.4.1 Precondition

Staff pool needs expansion.

2.23.4.2 Activation

Triggered by admin or scheduled regeneration.

2.23.4.3 Main flow

1 Generate staff attributes and persist.

2 Return created staff record.

2.23.4.4 Alternate flow

A1: Attribute conflict or duplicate identity - alter generated attributes (name, hometown) and retry generation automatically. A2: Pool constraints prevent generation - return 422 with explanation (e.g., max drivers reached).

2.23.4.5 Exceptional flow

E1: Database write failure - return 500 and record the attempted generation for retry. E2: Resource exhaustion (temp files, quota) - return 503 and back off generation.

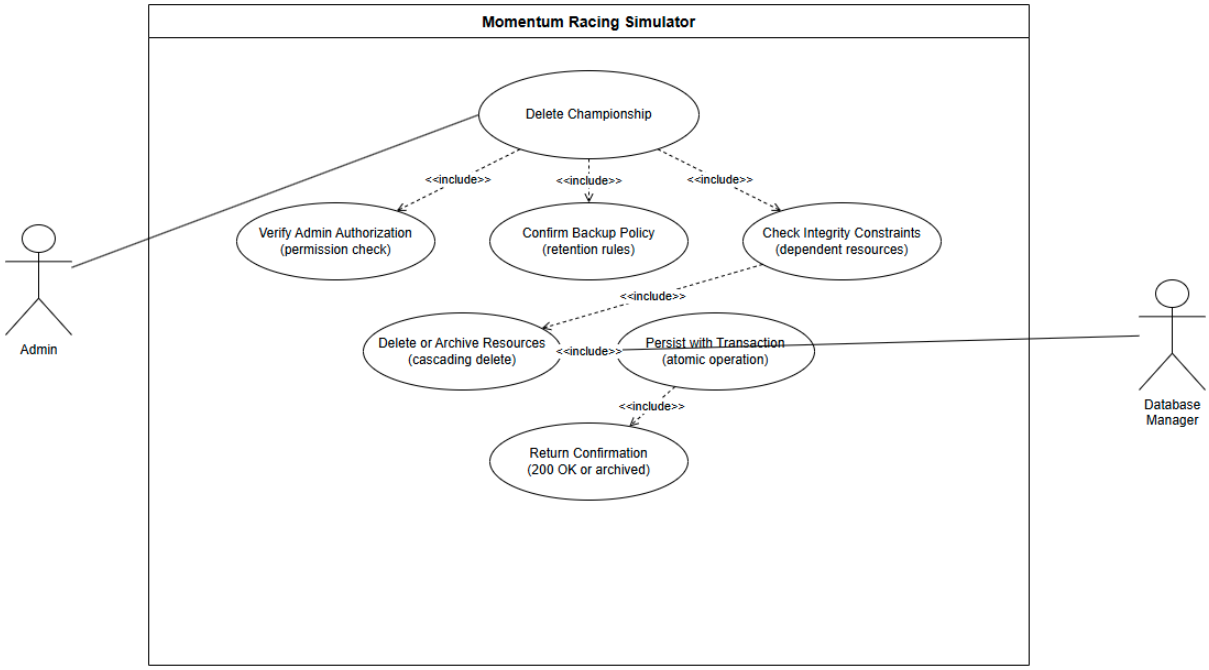
2.23.4.6 Termination

Staff visible in hiring pool.

2.23.4.7 Post condition

System awaits next generation.

2.24 UC-23: DELETE CHAMPIONSHIP



2.24.1 Scope

Remove or archive a championship (admin only).

2.24.2 Description

Delete or archive championship and related resources according to retention policy.

2.24.3 Actors

- Admin - authorised to delete or archive championships.

2.24.4 Flow Description

2.24.4.1 Precondition

Admin authorised and backup policy confirmed.

2.24.4.2 Activation

Starts when admin confirms deletion.

2.24.4.3 Main flow

1 Perform integrity checks.

2 Delete or archive resources.

3 Return confirmation.

2.24.4.4 Alternate flow

A1: Referential integrity prevents deletion - return 409 Conflict with list of dependent resources and suggested cleanup steps. A2: Admin chooses archive instead of delete - archive resources and return 200 OK with archived: true flag.

2.24.4.5 Exceptional flow

E1: Database cascade failure - return 500 and halt operation, create a rollback plan and notify admin. E2: Backup/restore service failure during deletion - abort and recommend manual intervention.

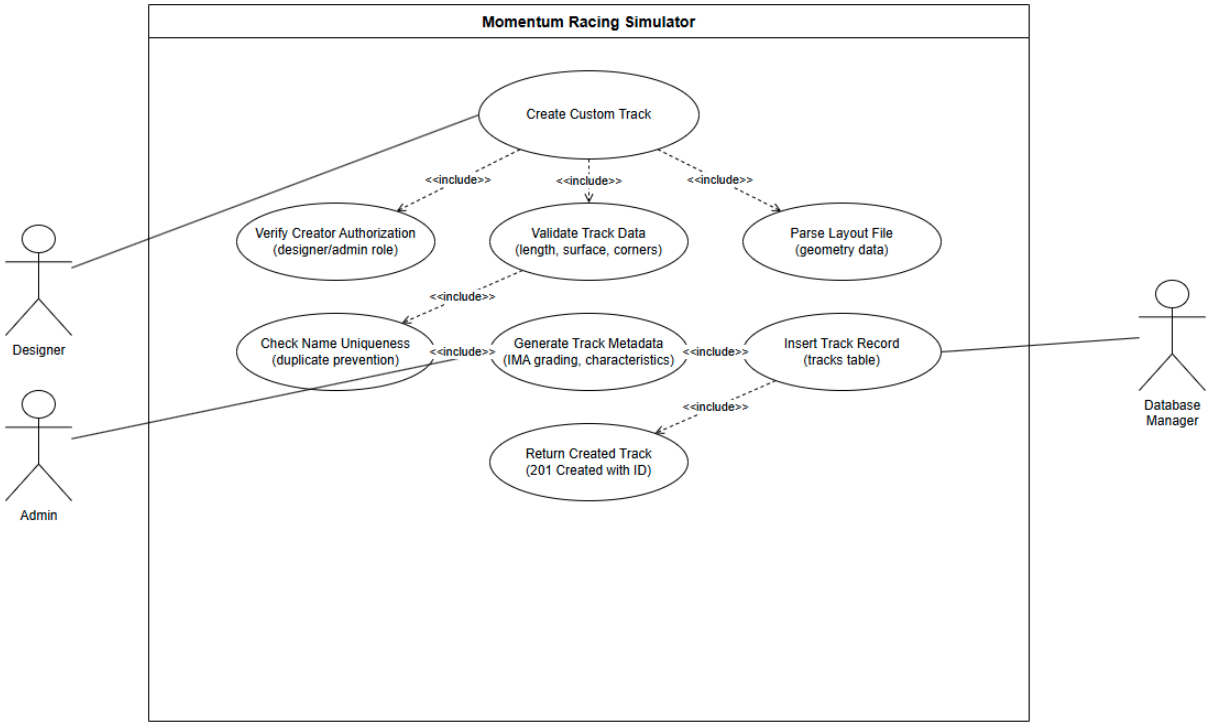
2.24.4.6 Termination

Championship removed or archived.

2.24.4.7 Post condition

System awaits next admin action.

2.25 UC-24: CREATE CUSTOM TRACK



2.25.1 Scope

Add a custom track with characteristics for use in events.

2.25.2 Description

Designer or admin submits track metadata and layout which becomes available for events.

2.25.3 Actors

- Designer/Admin - submits track definitions and layout files.

2.25.4 Flow Description

2.25.4.1 Precondition

Creator authorised.

2.25.4.2 Activation

Starts when track creation form is submitted.

2.25.4.3 Main flow

1. Validate track data.
2. Insert track record.
3. Return created track resource.

2.25.4.4 Alternate flow

A1: Invalid or incomplete track layout submitted - return 400 Bad Request with field-level errors and validation hints. A2: Duplicate track name or coordinates - return 409 Conflict with resolution options.

2.25.4.5 Exceptional flow

E1: Database write failure - return 500 and log the payload for manual replay. E2: Geometry parsing failure - return 422 Unprocessable Entity with parsing error details.

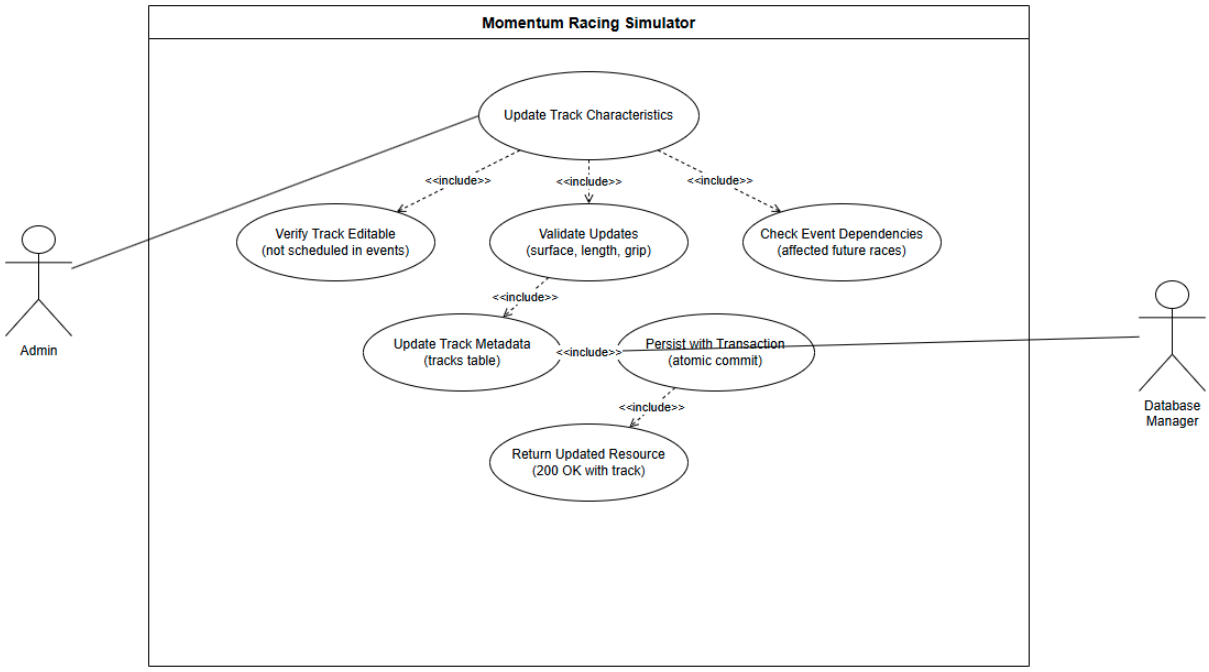
2.25.4.6 Termination

Track available for event creation.

2.25.4.7 Post condition

System awaits next track management action.

2.26 UC-25: UPDATE TRACK CHARACTERISTICS



2.26.1 Scope

Modify track parameters (surface, length, grip) for balancing.

2.26.2 Description

Admin updates track metadata and future events use revised parameters.

2.26.3 Actors

- Admin - submits track updates.

2.26.4 Flow Description

2.26.4.1 Precondition

Track exists and is editable.

2.26.4.2 Activation

Starts when admin submits updates.

2.26.4.3 Main flow

1 Validate and persist updates.

2 Return updated resource.

2.26.4.4 Alternate flow

A1: Track is scheduled in upcoming events - return 409 Conflict and offer archive or force options with explicit confirmation. A2: Minor characteristic change allowed - apply change with warnings indicating affected events.

2.26.4.5 Exceptional flow

E1: Database write failure - return 500 and queue the request for admin review. E2: Constraint violation due to dependent events - return 409 and detail the blocking events.

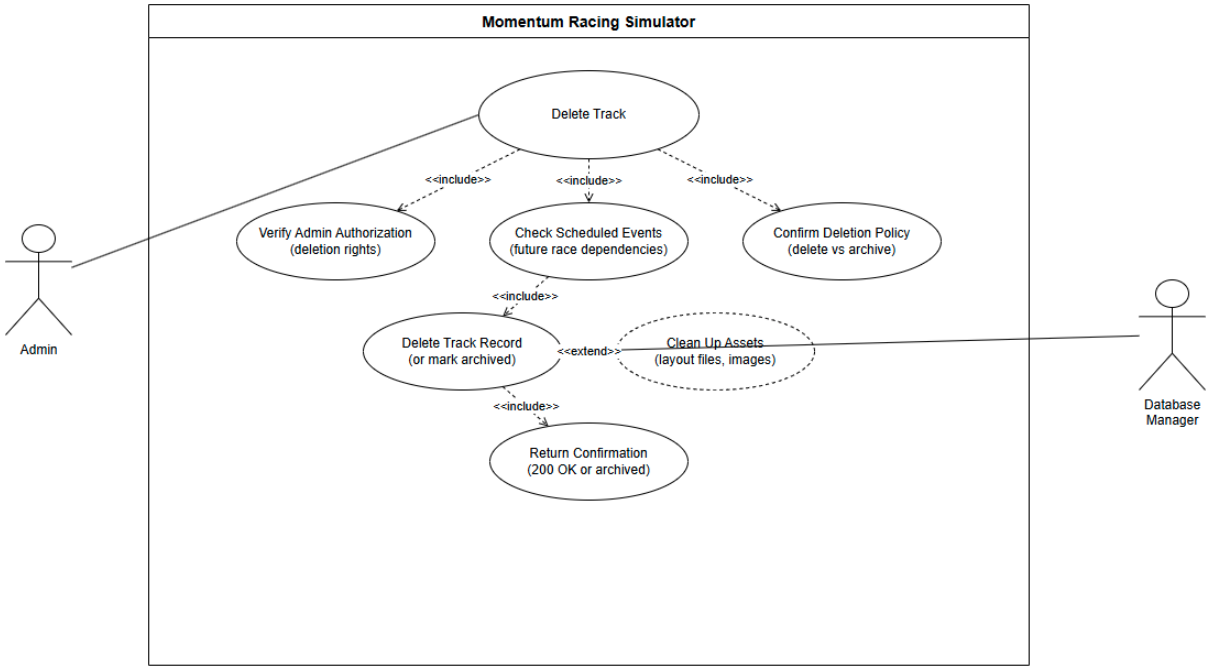
2.26.4.6 Termination

Track updated for future events.

2.26.4.7 Post condition

System awaits next management action.

2.27 UC-26: DELETE TRACK



2.27.1 Scope

Remove custom or unused tracks with checks for scheduled use.

2.27.2 Description

Admin removes track or archives it if scheduled in future.

2.27.3 Actors

- Admin - initiates deletion or archiving of tracks.

2.27.4 Flow Description

2.27.4.1 Precondition

Admin confirmed deletion policy.

2.27.4.2 Activation

Starts when deletion requested.

2.27.4.3 Main flow

1 Check for scheduled events.

2 If none, delete, else archive or return 409.

2.27.4.4 Alternate flow

A1: Track has scheduled events - automatically archive and return 200 OK with archived: true when admin confirms or return 409 if admin cancels.

2.27.4.5 Exceptional flow

E1: DB transaction failure during delete/archive - return 500 and log for manual reconciliation.
E2: File storage deletion error (assets) - mark record archived and flag cleanup pending.

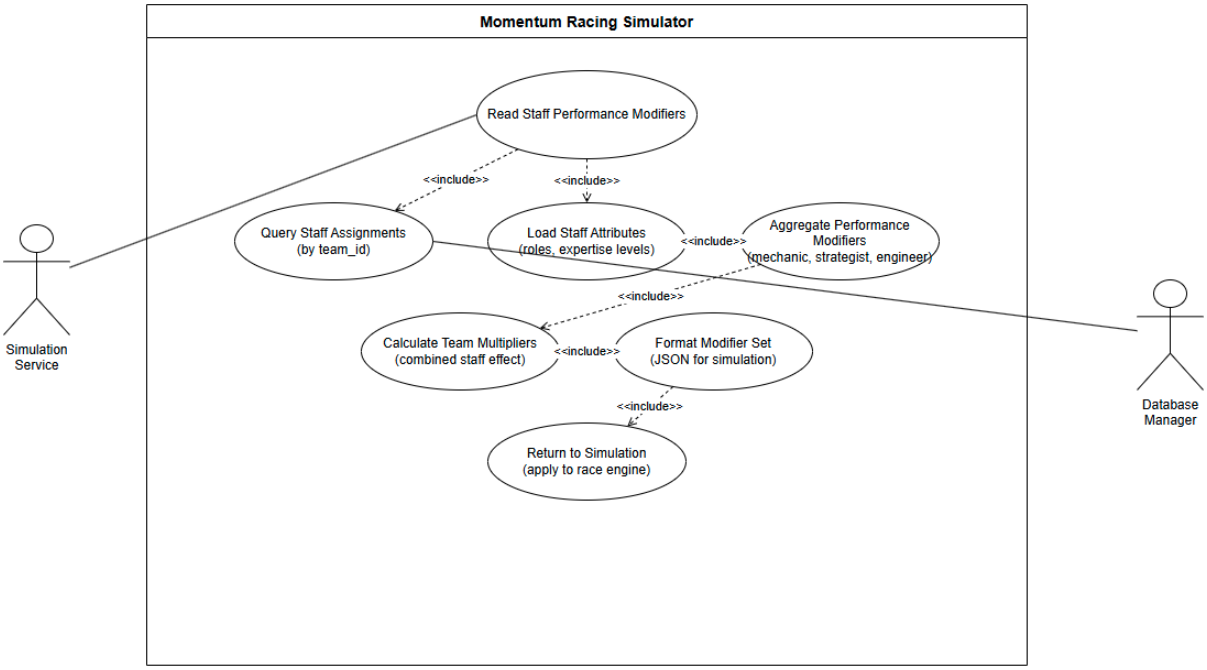
2.27.4.6 Termination

Track removed or archived.

2.27.4.7 Post condition

System awaits further track management.

2.28 UC-27: READ STAFF PERFORMANCE MODIFIERS



2.28.1 Scope

Return computed staff modifiers used by the simulation for team performance.

2.28.2 Description

Provide staff-based modifiers (mechanic efficiency, strategist value) to the Simulation Service.

2.28.3 Actors

- Simulation Service - requests computed modifiers for a team.

2.28.4 Flow Description

2.28.4.1 Precondition

Staff members assigned or available.

2.28.4.2 Activation

Starts when simulation requests modifiers for a team.

2.28.4.3 Main flow

1 Aggregate staff attributes for team.

2 Return modifier set to simulator.

2.28.4.4 Alternate flow

A1: Missing or partial staff assignments - return default modifiers and mark incomplete: true in response. A2: Requested team has no staff assigned - return 200 OK with defaults and a recommendation to hire staff for full modifiers.

2.28.4.5 Exceptional flow

E1: Database read failure - return 503 Service Unavailable and include diagnostic id. E2: Aggregation error while computing modifiers - return 500 and provide fallback default values.

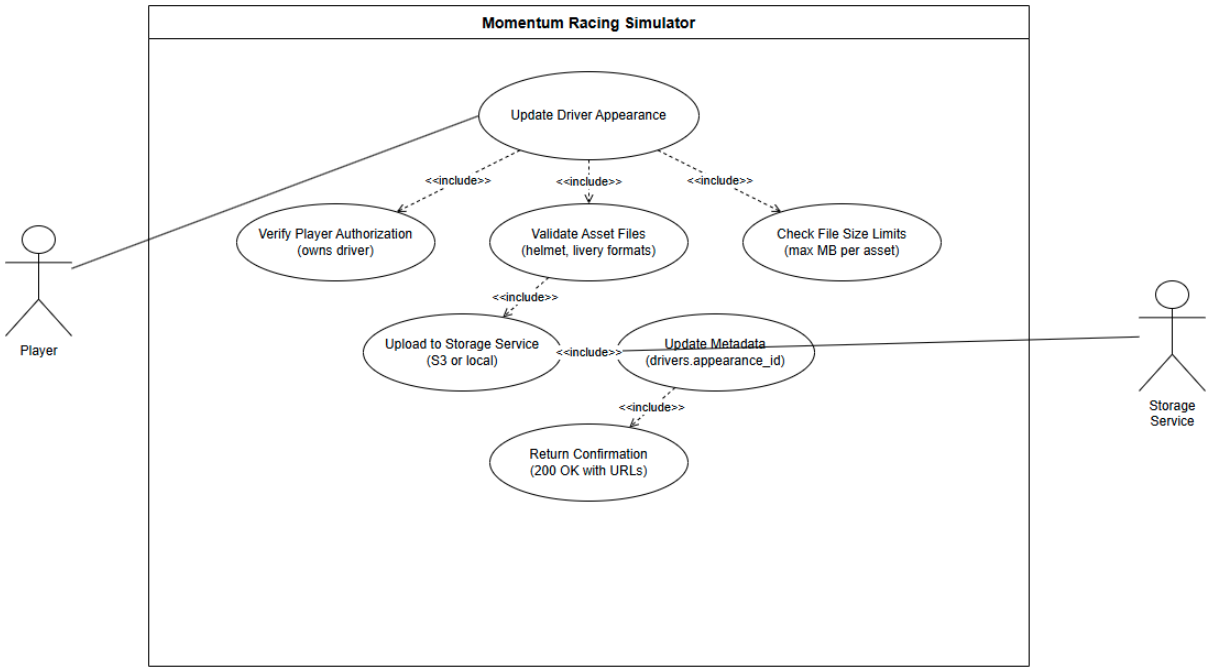
2.28.4.6 Termination

Simulation applies modifiers.

2.28.4.7 Post condition

System awaits next modifier request.

2.29 UC-28: UPDATE DRIVER APPEARANCE



2.29.1 Scope

Change cosmetic appearance for a driver (helmet, livery) - non-gameplay affecting.

2.29.2 Description

Player edits and saves driver appearance assets which are stored and served to UI.

2.29.3 Actors

- Player - creates or edits appearance assets.
- Storage Service - stores and serves asset files.

2.29.4 Flow Description

2.29.4.1 Precondition

Player owns or is authorised to edit driver appearance.

2.29.4.2 Activation

Starts when player submits appearance changes.

2.29.4.3 Main flow

1 Validate assets.

2 Persist appearance metadata and files.

3 Return confirmation.

2.29.4.4 Alternate flow

A1: Invalid asset format or too large - return 400 Bad Request with accepted formats and size limits. A2: Asset conflicts (same filename) - return 409 Conflict and provide renaming options.

2.29.4.5 Exceptional flow

E1: File storage error (S3/local) - return 500 and queue asset upload for retry, return a reference id to the client. E2: Virus/validation rejection - return 422 Unprocessable Entity with reason.

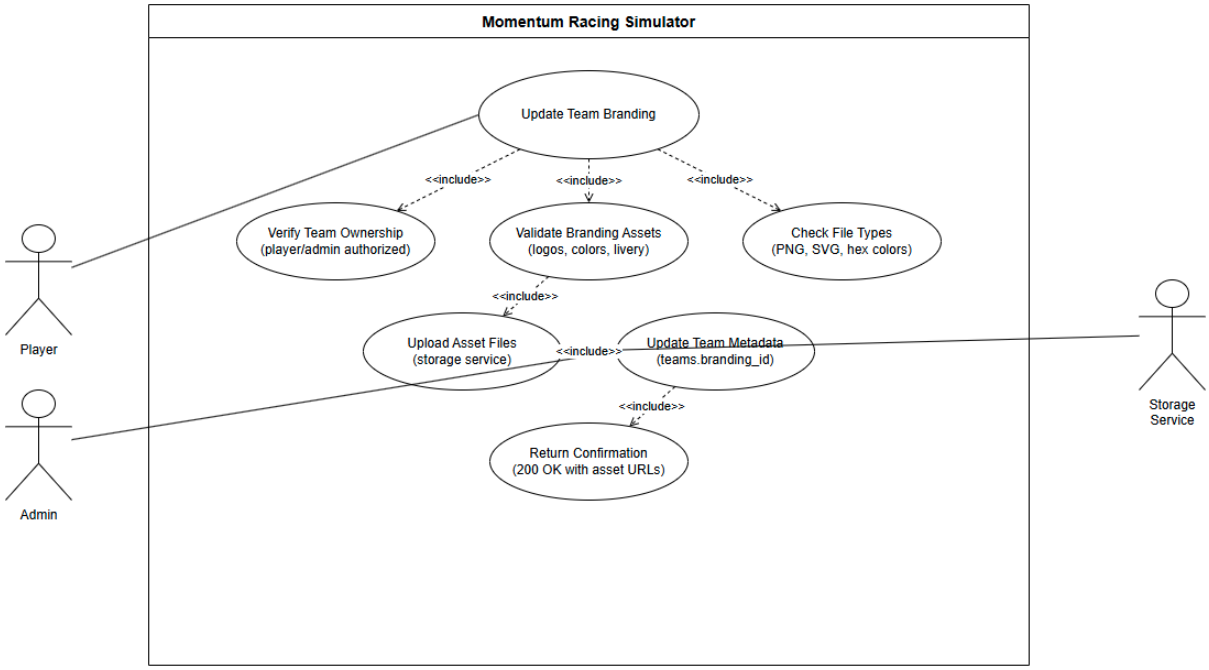
2.29.4.6 Termination

Appearance updated for reads.

2.29.4.7 Post condition

System awaits further edits.

2.30 UC-29: UPDATE TEAM BRANDING



2.30.1 Scope

Update team visual branding assets and metadata (logos, colours).

2.30.2 Description

Player or admin uploads branding assets and updates team metadata for display in UI.

2.30.3 Actors

- Player/Admin - uploads branding assets.
- Storage/DB - stores branding files and metadata.

2.30.4 Flow Description

2.30.4.1 Precondition

Requester authorised, assets valid.

2.30.4.2 Activation

Starts when branding upload is submitted.

2.30.4.3 Main flow

1 Validate assets and metadata.

2 Persist files and update team record.

3 Return confirmation.

2.30.4.4 Alternate flow

A1: Invalid branding assets - return 400 Bad Request with clear validation errors and accepted file types. A2: Asset name conflicts - return 409 Conflict with suggested renames.

2.30.4.5 Exceptional flow

E1: File store or DB failure - return 500 and persist metadata change to a pending queue. E2: Permission or quota error - return 403 or 429 as appropriate with remediation steps.

2.30.4.6 Termination

Branding served to UI.

2.30.4.7 Post condition

System awaits next update.

2.31 Data Requirements

This section documents the project's persistent database schema and related data requirements. The database uses a lightweight SQLite schema (SCHEMA_SQL) with seven primary tables and a small set of indexes.

2.31.1 Database Schema

The database contains the following tables (principal columns shown):

2.31.1.1 Drivers

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Biographical: name (TEXT NOT NULL), nationality (TEXT NOT NULL), birth_date (TEXT NOT NULL)
- Performance attributes (REAL, 0.0–100.0 with CHECK where applicable): speed, racecraft, consistency, fitness, reactions, pressure, setup_knowledge, fuel_management, tyre_management, feedback, adaptability
- Career statistics: career_points (INTEGER), career_podiums (INTEGER), career_starts (INTEGER), career_dnf (INTEGER), seasons_active (INTEGER)
- Discipline wins: single_seater_wins, touring_wins, rally_wins, street_wins, drag_wins, rallycross_wins, endurance_wins (INTEGER)
- Career progression & metadata: peak_performance_age (INTEGER), retirement_age (INTEGER), learning_rate (REAL), form_trend (REAL), biography (TEXT), career_highlights (TEXT), preferred_discipline (TEXT), career_phase (TEXT)
- Team/contract: current_team_id (INTEGER), previous_teams (TEXT JSON-string), contract_length (INTEGER), salary (REAL), name_locale (TEXT)
- Foreign key: current_team_id references teams(id)

2.31.1.2 Teams

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Fields: name (TEXT NOT NULL UNIQUE), nationality (TEXT NOT NULL), discipline (TEXT NOT NULL), tier (TEXT NOT NULL), budget (REAL NOT NULL)
- Optional: founded_year, team_principal, history (TEXT), total_wins, total_podiums, championships (INTEGER)
- Performance attributes: operations_rating (REAL DEFAULT 50.0, CHECK 0.0–100.0), strategy_profile (TEXT)

2.31.1.3 Tracks

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Fields: name (TEXT NOT NULL), country (TEXT NOT NULL), primary_discipline (TEXT NOT NULL)
- Physical/auxiliary: is_point_to_point (INTEGER 0/1), joker_enabled (INTEGER 0/1), instructions_json (TEXT), geometry_json (TEXT), pit_entry_index, pit_exit_index
- Note: additional track geometry and physical attributes are modelled in the Track dataclass and can be persisted in JSON fields when required.

2.31.1.4 Championships

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Fields: name (TEXT NOT NULL UNIQUE), discipline (TEXT NOT NULL), season_length (INTEGER NOT NULL), points_system (TEXT NOT NULL), current_season (INTEGER NOT NULL)

2.31.1.5 Races

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Foreign Keys: championship_id (REFERENCES championships(id)), track_id (REFERENCES tracks(id))
- Fields: season (INTEGER), round_number (INTEGER), race_date (TEXT NOT NULL), weather_conditions (TEXT), laps (INTEGER), status (TEXT DEFAULT 'scheduled')

2.31.1.6 RaceResults

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Foreign Keys: race_id (REFERENCES races(id)), driver_id (REFERENCES drivers(id)), team_id (REFERENCES teams(id))
- Fields: position (INTEGER NOT NULL), fastest_lap (INTEGER 0/1 DEFAULT 0), points (INTEGER DEFAULT 0), dnf_reason (TEXT)
- Expanded persisted fields: time_seconds, grid_position, laps_completed, best_lap_time, average_lap_time, pitstops, tyre_strategy, penalties_seconds, penalty_reason, gap_to_winner_seconds, status (TEXT DEFAULT 'FINISHED'), recorded_at

2.31.1.7 StaffMembers

- Primary Key: id (INTEGER PRIMARY KEY AUTOINCREMENT)
- Fields: name (TEXT NOT NULL), role (TEXT NOT NULL), rating (REAL DEFAULT 50.0), current_team_id (INTEGER REFERENCES teams(id))

2.31.2 Data Integrity

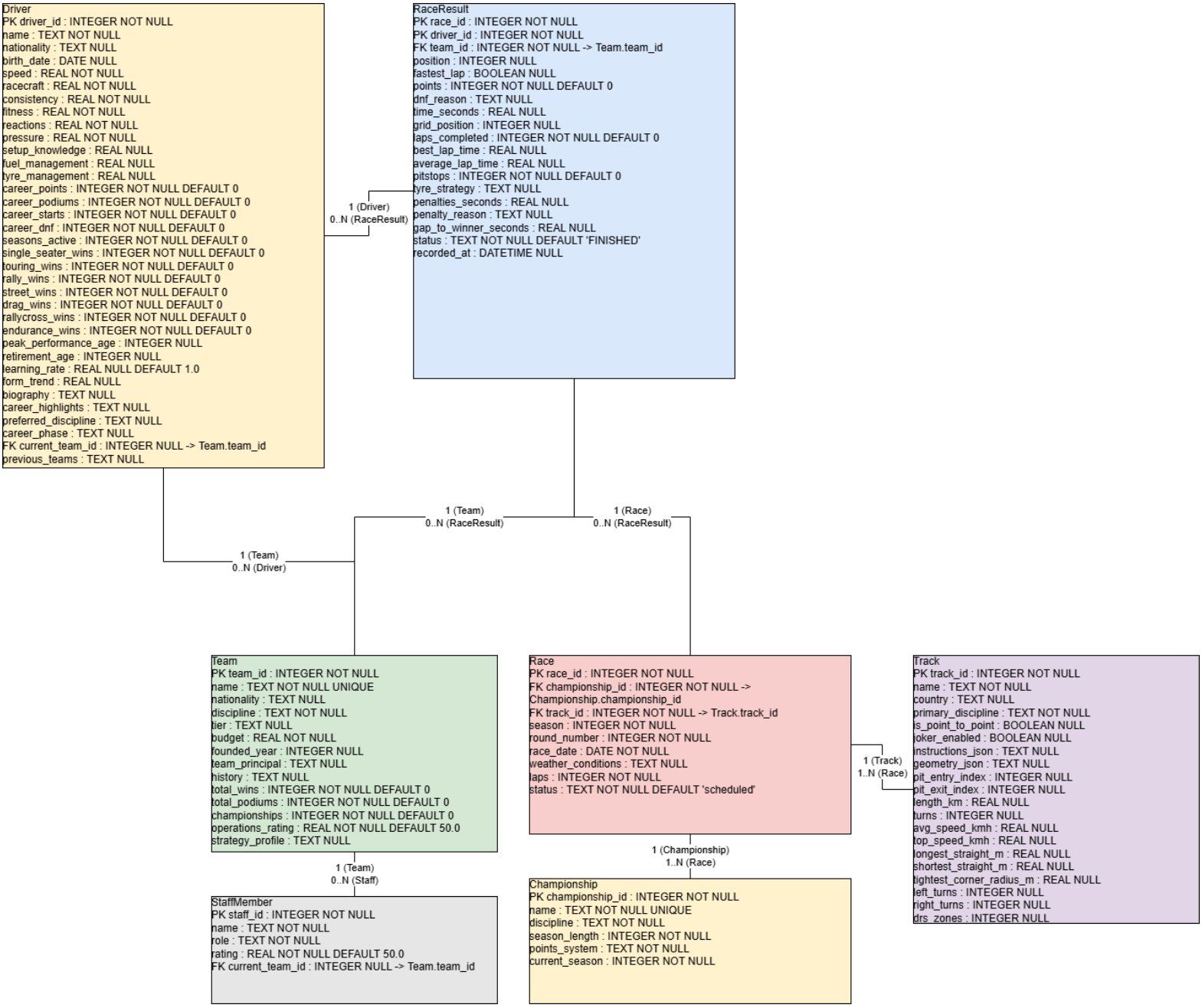
- Foreign key constraints: Implemented for drivers.current_team_id -> teams(id), races.championship_id -> championships(id), races.track_id -> tracks(id), race_results.(race_id,driver_id,team_id) -> races/drivers/teams, and staff_members.current_team_id -> teams(id).
- CHECK constraints: Core driver and team numeric attributes (e.g., speed, operations_rating) include CHECK(range) clauses in SCHEMA_SQL to enforce 0.0–100.0 where appropriate.
- UNIQUE constraints: teams.name and championships.name are declared UNIQUE to prevent duplicates.
- NOT NULL: Critical fields use NOT NULL (e.g., driver name, nationality, birth_date; team name, discipline, budget; race race_date).

2.31.3 Performance Optimisations

- Indexes: The schema creates the following indexes: idx_drivers_team on drivers(current_team_id), idx_teams_discipline on teams(discipline), and idx_race_results_driver on race_results(driver_id).

- Query patterns: Query patterns favour small, targeted queries and joins; larger or optional payloads are stored as JSON/text to avoid wide tables.
- Composite lookups are supported to efficiently query races by championship, season and round, and to fetch results by race.

2.31.4 Diagram



2.32 USER REQUIREMENTS

2.32.1 Primary user personas:

Personal	Specific	Measurable	Achievable	Relevant	Time-bound
Casual Race Fans	Start a single race using default driver and configuration	Complete a race within 5 minutes, HUD text legible, top-line stats visible without extra menus	Default presets and simplified HUD provided	Fast, entertaining access for casual players	Within 5 minutes of app launch
Management / Career Mode Players	Save/load careers, manage teams and drivers, adjust contracts	Save/load completes without data loss, standings recalculation finishes reliably, contract/transfer changes reversible via UI	Backed by SQLite persistence, background job queue, and UI validation	Supports deep career management and championship progression	Operations completed within session, save/load <30 seconds
Modders / Content Creators	Import/export tracks, drivers, or balance changes using JSON/CSV	Custom track import succeeds and visible, names can be added/modified	Tools provided with documented asset pipeline, RNG seeds hidden	Enables content creation and community engagement	Import/export completes within 60 seconds for standard files

2.32.2 Cross-cutting user requirements:

Requirement	Specific	Measurable	Achievable	Relevant	Time-bound
Local-first	Game runs fully offline	All core features accessible offline	SQLite included, no mandatory online services	Ensures usability in offline environments	Instant availability on app launch
Replay and Investigation	Races record telemetry and events	Telemetry stored, replay visualisation accessible	Batched IO writes minimise overhead, simulation non-deterministic	Allows analysis of AI and player performance	Telemetry available immediately after race completion
Low Friction Installer/Run	Install and run via Python or packaged binary	Instructions execute successfully, binary launches app without additional setup	PyInstaller binary provided, minimal Python dependencies documented	Reduces barriers for non-technical users	Installation and launch completed within 10 minutes

2.33 Environmental Requirements

2.33.1 Hardware Requirements:

- **Minimum:** 2GB RAM, 500MB disk space, 1GHz dual-core CPU
- **Recommended:** 4GB RAM, 1GB disk space, 2GHz quad-core CPU

2.33.2 Software Requirements:

- **Operating System:** Windows 10+, macOS 10.14+, Linux (Ubuntu 20.04+)
- **Python:** Version 3.8 or higher
- **Database:** SQLite 3.7+

2.33.3 Deployment Requirements:

- **Development:** Local uvicorn server on localhost:8000
- **Production:** ASGI-compatible server (uvicorn, gunicorn)
- **Database:** File-based SQLite (momentum.db)

2.33.4 Scalability Considerations:

- API can run multiple worker processes
- Database can migrate to PostgreSQL/MySQL for multi-user scenarios
- Stateless design enables horizontal scaling

2.34 Usability Requirements

2.34.1 Accessibility

2.34.1.1 Colour and contrast

2.34.1.1.1 Specifics

All informational colour pairs must meet WCAG 2.1 contrast ratios: at least 4.5:1 for normal text and 3:1 for large text/icons. (World Wide Web Consortium, 2025) Provide a colourblind-safe palette option (deuteranopia/protanopia/tritanopia safe) and ensure graphs can be read without colour alone (use patterns, shapes, or icons). Implement a high-contrast mode. (World Wide Web Consortium, 2025)

2.34.1.1.2 Acceptance Criteria

All text and icons meet the required contrast ratios. Colourblind-safe palette is available and testable. High-contrast mode can be toggled and preserves readability.

2.34.1.2 Colourblind support

2.34.1.2.1 Specifics

Avoid using red/green as the sole differentiator. Provide shape/icon overlays and tooltips. Provide a toggle to preview UI using common colour-blind filters for validation.

2.34.1.2.2 Acceptance Criteria

Colourblind filters correctly display all UI elements without reliance on colour alone. Tooltips and overlays convey the same information as colour cues.

2.34.1.3 Text & scaling

2.34.1.3.1 Specifics

Support UI scaling (100% / 125% / 150%) and a minimum font size option. Ensure dynamic resizing of panels preserves usability.

2.34.1.3.2 Acceptance Criteria

UI remains fully usable at all supported scaling levels. No content overlaps or truncation occurs.

2.34.1.4 Keyboard and input

2.34.1.4.1 Specifics

Arrow keys move focus. Enter activates the focused item. Escape returns to the previous screen. Tab and Shift+Tab cycle focus. All focusable elements display an outline. Spacebar toggles pause or resume. A increases push intensity. Z decreases push intensity. X toggles overtaking aggression mode. R restarts the race when paused and after a confirmation dialogue. All race controls must be remappable. Conflict detection must prevent duplicate bindings. Remapped bindings must persist across sessions.

2.34.1.4.2 Acceptance Criteria

The full interface must be operable using keyboard only. Spacebar toggles pause unless remapped. Remapped bindings take effect immediately. All focusable elements show a visible outline during keyboard navigation.

2.34.2 Performance

2.34.2.1 Frame rate targets

2.34.2.1.1 Specifics

Target GUI/simulation visual update rate: 30 FPS. Minimum acceptable visual responsiveness: 15 FPS during heavy load. Simulation logic can run decoupled from rendering, logic ticks target 30 ticks/sec.

2.34.2.1.2 Acceptance Criteria

GUI maintains target frame rate under normal load. Low-fidelity mode maintains minimum responsiveness under heavy load.

2.34.2.2 Responsiveness

2.34.2.2.1 Specifics

UI actions (menu navigation, opening panels) respond within 150–300 ms. Long-running tasks run as background jobs with progress indicators.

2.34.2.2.2 Acceptance Criteria

All UI interactions respond within specified limits. Background tasks do not block the UI.

2.34.2.3 Resource usage

2.34.2.3.1 Specifics

Provide a low-memory mode that reduces telemetry retention and drops non-essential animations.

2.34.2.3.2 Acceptance Criteria

Low-memory mode is functional and maintains core usability.

2.34.3 Readability

2.34.3.1 Clear language

2.34.3.1.1 Specifics

Use plain English for labels, tooltips, and help text.

2.34.3.1.2 Acceptance Criteria

All UI text is understandable by users without technical background.

2.34.3.2 Tooltips and help

2.34.3.2.1 Specifics

All non-trivial controls must have explanatory tooltips and an optional in-app short tutorial.

2.34.3.2.2 Acceptance Criteria

Tooltips provide sufficient context. Tutorial covers all non-obvious controls.

2.34.3.3 Localisation

2.34.3.3.1 Specifics

Design UI strings for easy translation. Support UTF-8 locales and date/currency formatting.

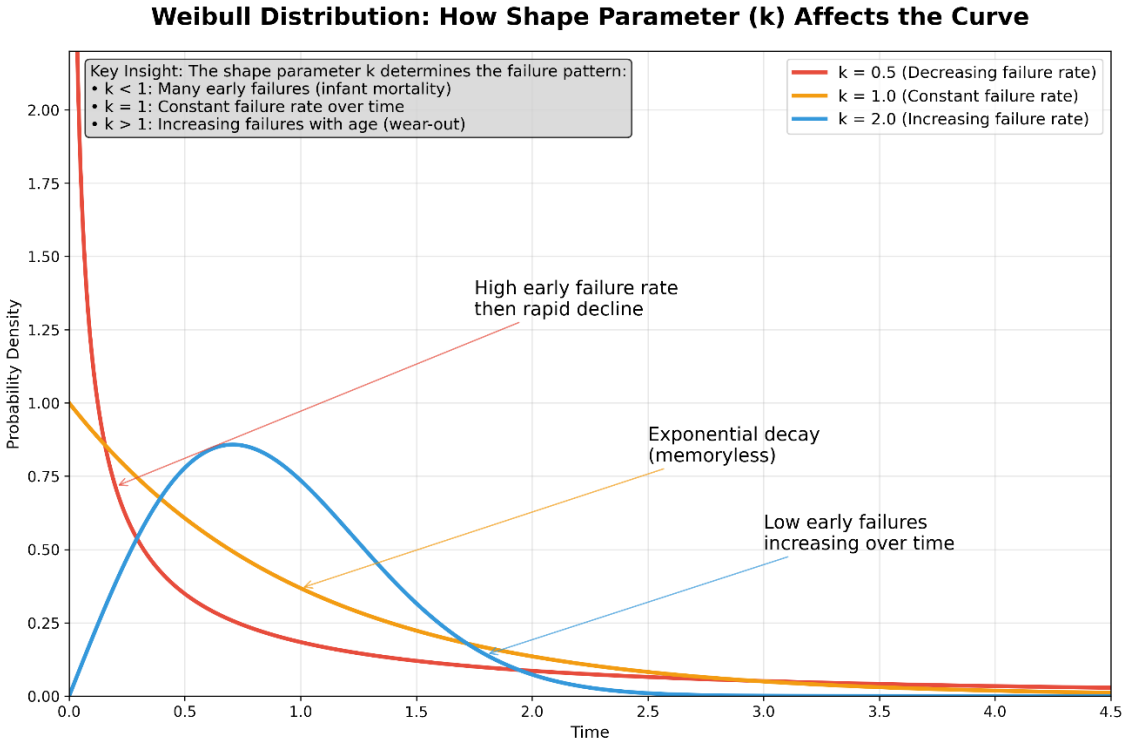
2.34.3.3.2 Acceptance Criteria

All UI text can be translated without breaking layouts. Locale-specific formatting displays correctly.

2.35 Design & Architecture

2.36 IMPLEMENTATION

The Concise Guide to Weibull Distribution



P:TEXSIMAX -1 43 43

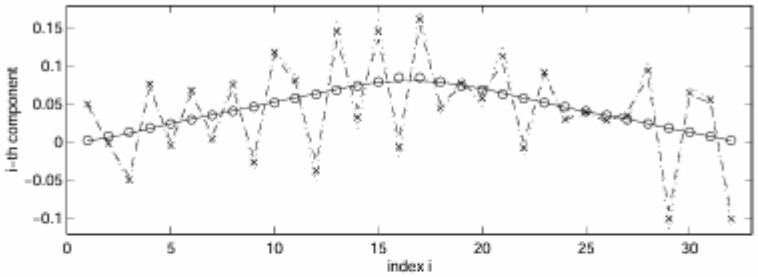


FIG. 1. *Regularized solutions to* (3) *for various values of the regularization parameter* λ; *see Table* 1 *for details.*

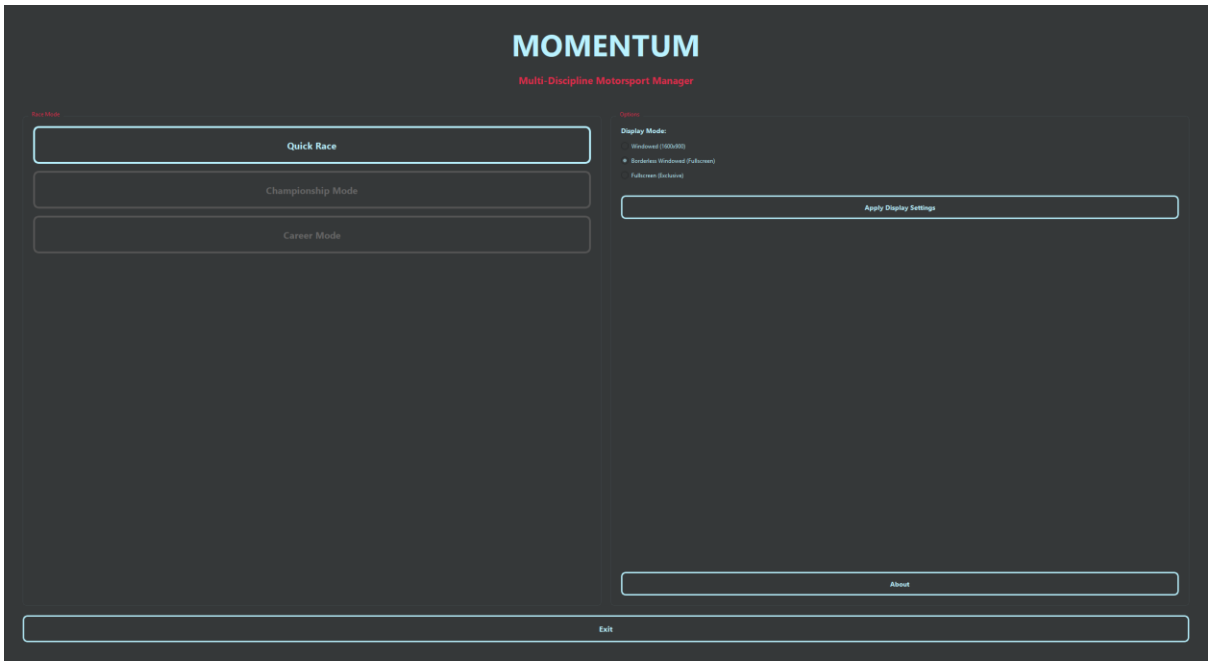
TABLE 1
The solutions shown in Figure 1.

Solution	λ	Solution norm	Symbol
x_λ	10^{-5}	0.289	solid line
x_{LS}	0	0.385	dashed line
x_λ	$-\frac{1}{2}\bar{\sigma}^2_{n+1}$	0.408	crosses
x_{TLS}	$-\bar{\sigma}^2_{n+1}$	0.443	dotted line
x^{exact}	–	0.288	circles

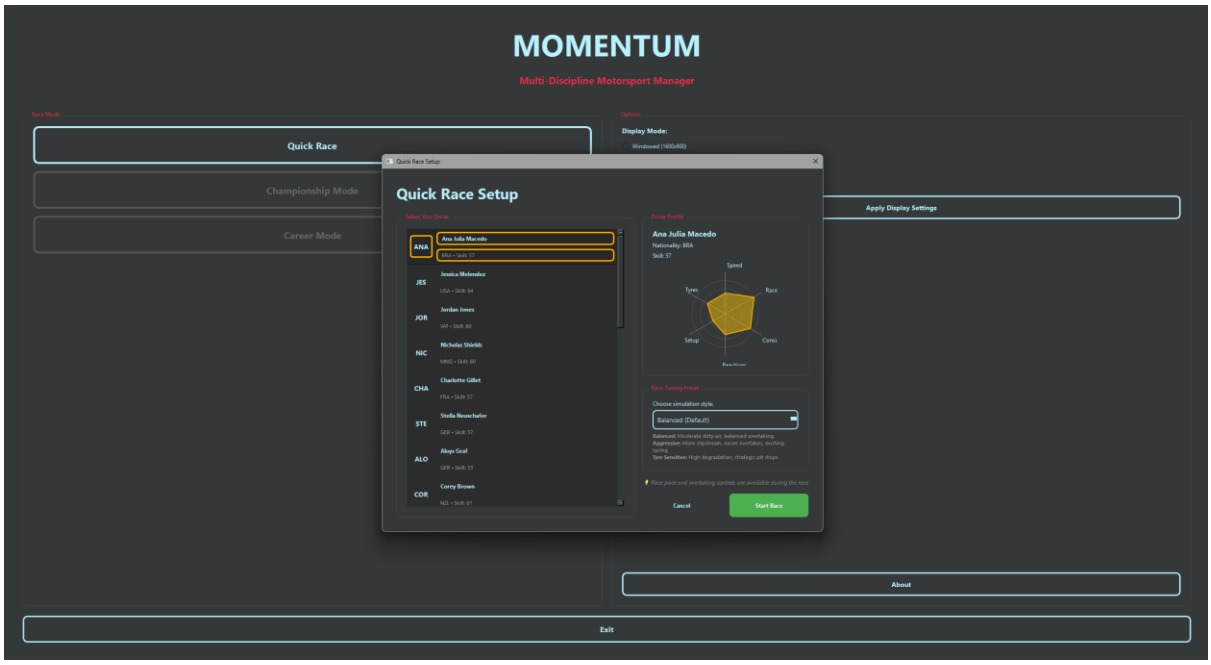
2.37 Graphical User Interface (GUI)

2.37.1 Main Menu

2.37.1.1 Starting Page

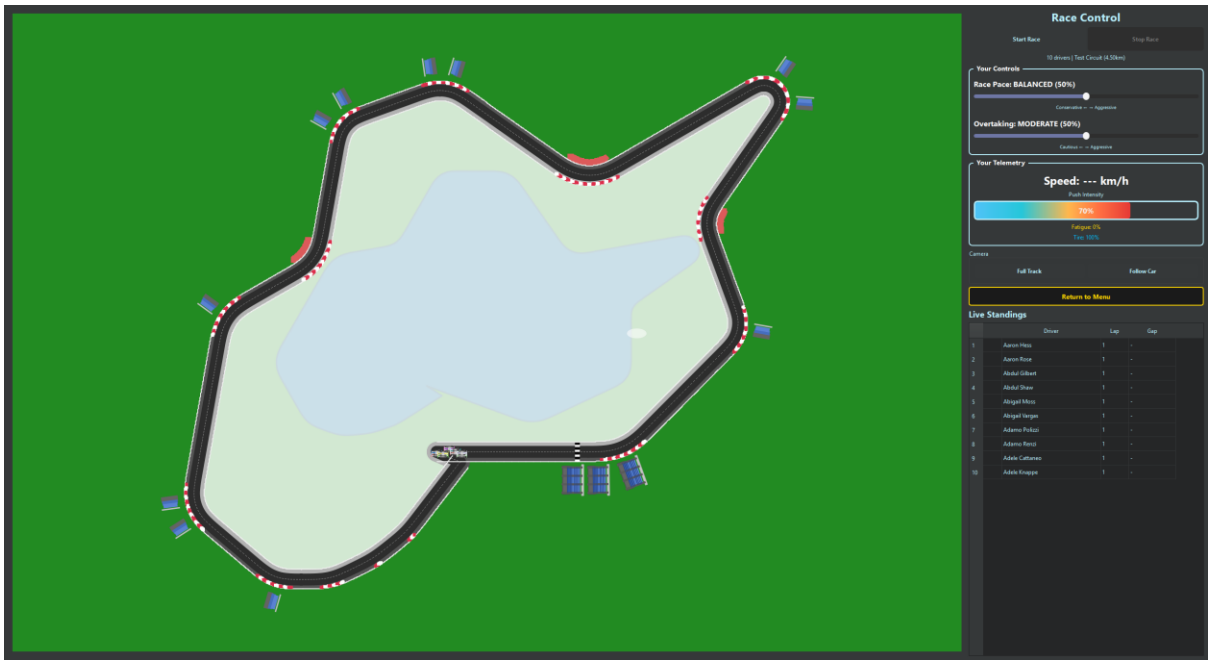


2.37.1.2 Setup Screen

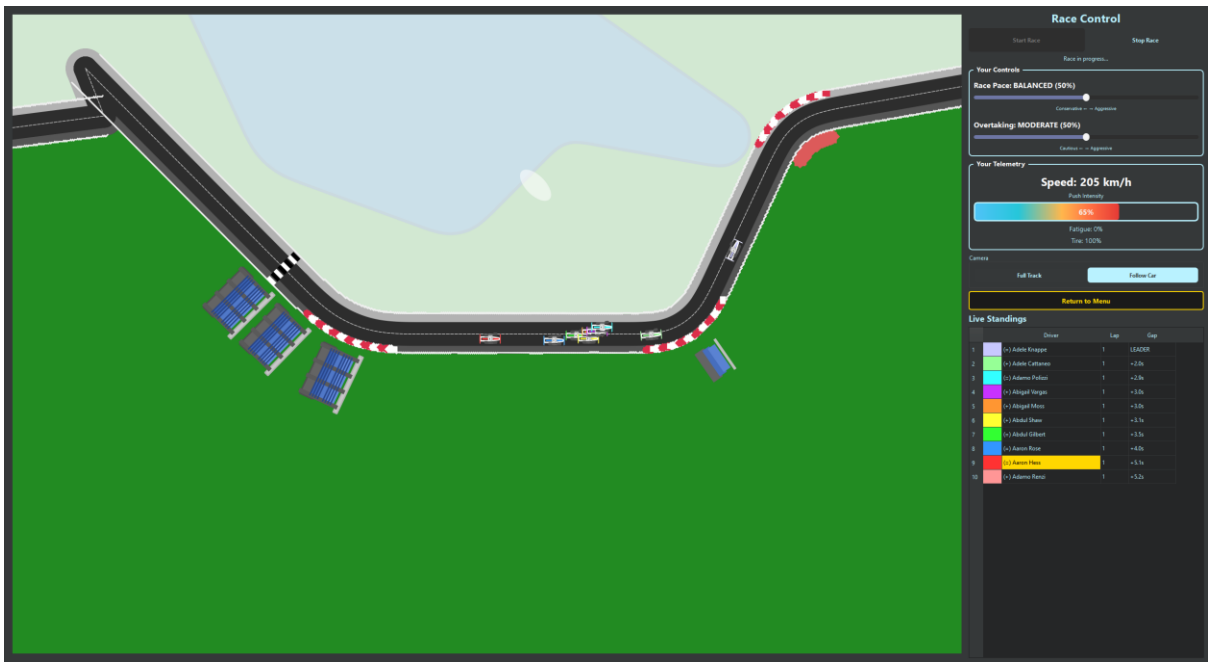


2.37.2 Race Screen

2.37.2.1 Pre-Race – Full Track Camera



2.37.2.2 Mid Race – Follow Car Camera



2.38 TESTING

2.39 EVALUATION

3 CONCLUSIONS

4 FURTHER DEVELOPMENT OR RESEARCH

5 BIBLIOGRAPHY

Faker Contributors, 2026. *Faker Documentation*. [Online]
Available at: https://faker.readthedocs.io/
[Accessed 26 4 2026].

FastAPI, 2026. *FastAPI Documentation*. [Online]
Available at: https://fastapi.tiangolo.com/
[Accessed 26 4 2026].

Gross, A., 2026. *Unidecode (PyPI project)*. [Online]
Available at: https://pypi.org/project/Unidecode/
[Accessed 26 4 2026].

Kumar, S., 2026. *python-dotenv Documentation*. [Online]
Available at: https://saurabh-kumar.com/python-dotenv/
[Accessed 26 4 2026].

NumPy Developers, 2026. *NumPy Documentation*. [Online]
Available at: https://numpy.org/doc/
[Accessed 26 4 2026].

Pydantic, 2026. *Pydantic Documentation*. [Online]
Available at: https://docs.pydantic.dev/
[Accessed 26 4 2026].

Python Pillow Team, 2026. *Pillow Documentation*. [Online]
Available at: https://pillow.readthedocs.io/
[Accessed 26 4 2026].

Python Software Foundation, 2026. *Black Documentation*. [Online]
Available at: https://black.readthedocs.io/
[Accessed 26 4 2026].

PyTorch Contributors, 2026. *PyTorch Documentation*. [Online]
Available at: https://pytorch.org/docs/stable/
[Accessed 26 4 2026].

Riverbank Computing Limited, 2026. *PyQt6 Documentation*. [Online]
Available at: https://www.riverbankcomputing.com/software/pyqt/
[Accessed 26 4 2026].

SciPy Community, 2026. *SciPy Documentation*. [Online]
Available at: https://docs.scipy.org/doc/scipy/
[Accessed 26 4 2026].

SQLite Consortium, 2026. *SQLite Documentation*. [Online]
Available at: https://www.sqlite.org/docs.html
[Accessed 26 4 2026].

Uvicorn, 2026. *Uvicorn Documentation*. [Online]
Available at: https://www.uvicorn.org/
[Accessed 26 4 2026].

World Wide Web Consortium, 2025. *WCAG 2.1 Understanding Docs*. [Online]
Available at: https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html
[Accessed 8 December 2025].

6 APPENDICES

6.1 PROJECT PROPOSAL



National College of Ireland

Project Proposal

Momentum

11/12/25

Computing

Software Development

2025/2026

Eoin Andrew Fitzsimons

X23151374

X23151374@student.ncirl.ie

6.1.1 Objectives

Momentum aims to design and implement a prototype motorsport management simulator that captures the scale, depth, and unpredictability of real-world racing ecosystems. Unlike existing titles, which currently focus on a limited range of disciplines (e.g., Formula 1 and limited feeder systems), it will model a wide variety of racing disciplines, each with its own unique challenges and culture. For example, rally racing will feature dynamic terrain and weather, requiring teams to adapt strategies for gravel, tarmac, or snow. Endurance racing will simulate human factors such as driver fatigue, teamwork, and mechanical reliability over long events. Street racing will emphasise customisation, driver personality, and the underground culture that shapes rivalries and alliances. By capturing these distinctions, the simulator will aim to represent a more varied and context-sensitive motorsport environment, where each series presents different management challenges.

The objective is to simulate a persistent game world where events unfold across all leagues, regardless of whether the player is directly involved. By doing so, the system mirrors the emergent qualities of Football Manager, supporting diverse career trajectories that may differ across playthroughs.

A further objective is to offer a high degree of customisation. Players will be able to modify or generate new teams, drivers, and competitions. Procedural generation will be a cornerstone, enabling large-scale fictional ecosystems that feel authentic while avoiding licensing restrictions.

The project will demonstrate implementation across key areas including simulation design and AI-driven decision-making. The intended prototype will model core systems such as driver progression, economic management, league standings, and race outcomes, with a focus on achieving computational efficiency.

The project's objective is to produce a management simulator that can serve as a foundation for future extension. It seeks to address an identified gap in current games by emphasising simulation depth and customisation, rather than prioritising graphical fidelity.

6.1.2 Background

My passion for motorsport spans a wide range of disciplines, from open-wheel racing to rally, endurance, and street racing. While some recent games, such as The Crew Motorfest, successfully capture the unique feel and culture of different racing series, they focus on driving experiences rather than management. Existing management games, on the other hand, tend to concentrate on a single discipline or lack the depth and variety needed to authentically represent the full spectrum of motorsport. No current title combines the breadth of racing types, the distinctiveness of each discipline, and the emergent, persistent world of a management simulation. This project seeks to address that gap by integrating the unique characteristics of multiple racing series into a single, dynamic management experience.

Having spent extensive hours in Football Manager, I have experienced the appeal of large, complex simulations that generate emergent stories. Despite having a stronger interest in Formula 1 and motorsport generally than in football, I find myself drawn more to Football Manager than to existing motorsport management titles. This reveals an important insight: the attraction lies not in the sport itself, but in the richness of the simulated world.

By comparison, current motorsport titles such as F1 Manager and Motorsport Manager are limited. Their focus is on presentation and visuals rather than world depth. The lack of customisation and the narrow scope of simulated leagues lead to repetitive experiences over time.

This project seeks to address this gap by exploring how the thematic appeal of motorsport can be combined with the systemic depth characteristic of Football Manager. The objectives outlined in Section 1.0 will be met by designing algorithms that support persistent multi-league simulation, driver development models, and financial/strategic systems. Procedural content generation will be used to overcome licensing limitations and create a fictional but believable motorsport ecosystem.

6.1.3 State of the Art

Current motorsport management titles include Motorsport Manager (2016) and F1 Manager (2022–2024). These games present detailed races and authentic visuals, but their simulation depth is fundamentally limited. Driver development is linear and predetermined, making long-term play predictable. Once a player has learned the optimal strategies, the outcomes become repetitive. In contrast, Football Manager maintains replayability by simulating an immense global ecosystem of leagues, players, and staff, where no two careers are the same. This difference highlights the gap between motorsport management games and the broader standard of emergent sports simulations.

Another limitation of existing motorsport titles is their lack of customisation. Players cannot easily expand leagues, create fictional scenarios, or reshape the competitive environment. The games are tightly constrained by licensing agreements and static structures, which prevents the freedom that makes Football Manager engaging. Furthermore, the game worlds are small in scale, restricted to a handful of categories such as Formula 1 or Formula 2, with little simulation occurring outside the top tiers. By comparison, Football Manager tracks hundreds of leagues and thousands of clubs across multiple nations, with events unfolding regardless of direct player involvement.

This project aims to distinguish itself by adopting that philosophy. The simulation will extend beyond open-wheel racing, with multiple tiers within each discipline. Every league will progress whether the player manages there or not, creating a sense of a living world. Because official licensing is not available, all teams, drivers, and championships will be procedurally created, which also allows experimentation with unique structures, rivalries, and narratives. By prioritising breadth, depth, customisation, and emergent systems, this project will investigate how principles seen in Football Manager's large-scale simulation might be adapted to a motorsport context.

It is worth noting that some recent non-management racing games, such as The Crew Motorfest, have made significant strides in capturing the unique feel, culture, and challenges of different racing disciplines. These titles demonstrate that it is possible to authentically represent the diversity of motorsport, from rally to endurance to street racing. However, they focus on the driving experience and do not offer the persistent, emergent management simulation that is central to this project. No current management title integrates the breadth of racing types, the distinctiveness of each discipline, and a persistent simulation world. This project aims to explore how these elements can be combined within a prototype, developing a management simulator that reflects both the variety and complexity of motorsport ecosystems.

6.1.4 Technical Approach

Momentum is being built using agile software principles. The system is organised into three main parts: the Simulation Engine, Database Management, and the Front-End Interface. Each part works together to create a realistic and engaging motorsport management experience.

6.1.4.1 Simulation Engine

The simulation engine is the heart of the system. Its design is inspired by popular motorsport management games and research into multi-agent systems. The engine handles things like how drivers improve over time, how teams manage their finances, how races are decided, and how multiple leagues can run at once. Race results are calculated using Monte Carlo simulations, which take into account driver skills, car performance, track details, weather, and random events. The financial side uses probability-based models to reflect real-world unpredictability, such as changes in sponsorship or unexpected costs. This means, for example, that a team with less money might have to take more risks with their strategy, which can change how the whole league plays out.

6.1.4.2 Database Management

The database management system is responsible for creating and storing all the teams, drivers, tracks, and championships. It uses algorithms to generate these elements based on certain rules and probabilities, ensuring that each league is balanced and diverse. Driver traits like skill, temperament, and fatigue are set according to the needs of the league. Tracks are created with different weather, locations, and surfaces, which all affect how races play out. Team budgets and resources are also assigned by the database, helping to create a dynamic and evolving competition. The system keeps an eye on things like driver diversity and league balance, making automatic adjustments to keep the game fair and interesting. All aspects of content creation and generation are managed here, so references to procedural generation are now part of database management.

6.1.4.3 Front-End Interface

The front-end interface, built in Python, displays race data, standings, driver stats, and financial dashboards. It shows the results of simulations and AI decisions, and updates in real time as the game progresses. When users make decisions or change strategies, these are sent to the back end, which updates the league data. The interface provides instant feedback on how the league is evolving and how the AI and database management systems are affecting the game.

6.1.4.4 Iteration, Feedback, and Scalability

The development plan is broken into clear phases, with regular feedback and review. Progress is measured by tracking things like driver development, AI improvements, financial stability, and the diversity of the database-generated content. The system is designed to be modular, so new features or motorsport types can be added without having to rebuild everything. Data is stored efficiently to allow for long-term, multi-league play, making Momentum a robust and replayable motorsport management simulator.

6.1.5 Technical Details

Momentum uses Python for both the front-end dashboards and the back-end simulation logic and database management. The system relies on NumPy for fast calculations, Pandas for handling data, and SciPy for advanced statistics. The database uses SQLite to store all the information about drivers, teams, tracks, and championships, making it easy to save and load multi-season histories. All content creation and generation is handled by the database management system.

6.1.5.1 Race Outcome Simulation

Race results are produced by a real-time, agent based engine. Each driver is modelled as an autonomous AI agent that makes decisions on every simulation tick, targeting a frame rate of about 30 ticks per second. Agents consider their attributes, car condition, tyre state, fuel, nearby rivals, team orders and track conditions when choosing actions such as attack, defend, overtake or conserve. Discipline specific rules are applied by the engine to reflect differences between single seater, rally, endurance and other formats. Randomness is retained for low probability events such as mechanical failures or incidents so outcomes remain unpredictable, but most race behaviour emerges from agent interactions rather than repeated sampling. Performance sensitive results are cached where appropriate to keep leaderboards and UI queries responsive.

6.1.5.2 Driver Progression

Driver attributes like speed, consistency, aggression, adaptability, and fatigue are tracked as numbers that can change over time. How a driver develops depends on their past results, which are stored in the database. Growth and decline are modelled using probability-based functions, so each driver's career is unique. Fatigue and injuries are also tracked, with recovery periods built in, making each season different and challenging.

6.1.5.3 Data Management

All teams, drivers, tracks, and championships are created and managed by the database management system. It uses algorithms to ensure that everything is balanced and diverse, with different locations and themes for tracks and championships. Before anything is saved, the system checks that the league will be fair and interesting. This approach makes every playthrough unique and replayable, and all content generation is now part of database management.

6.1.5.4 Performance Optimisation & Integration

The Python backend uses fast, vectorised calculations and keeps frequently used data in memory for speed. League states are saved at the end of each season, so progress is never lost. The system uses JSON to keep everything in sync between the front end and database management. The design makes it easy to add new features or motorsport types in the future, without having to change the core of the game.

6.1.6 Special Resources Required

6.1.6.1 Hardware Requirements

Momentum's Python GUI and simulation backend have modest hardware requirements. For typical development and use:

- Minimum Requirements:

  CPU: Dual-core processor (e.g., Intel i3 or AMD equivalent)

  RAM: 4 GB

  Storage: 500 MB free disk space

  GPU: Integrated graphics (no dedicated GPU required)

  Display: 1366x768 resolution

- Recommended Requirements:

  CPU: Quad-core processor (e.g., Intel i5 or AMD Rysen 3)

  RAM: 8 GB

  Storage: 1 GB free disk space

  GPU: Integrated or basic dedicated GPU (optional)

  Display: 1920x1080 resolution

  These specifications are sufficient for running and developing the Python GUI, simulation, and database management systems. Higher-end hardware is only needed if running very large-scale simulations.

6.1.6.2 Software and Libraries

The front end is built in Python, which handles all the graphics, user interface, and telemetry displays. The back end also runs on Python 3.11, using NumPy, Pandas, and SciPy for calculations and data handling. All persistent data is stored in SQLite, which is lightweight and works across platforms. This setup is enough for multi-season and multi-league play and can be upgraded to a bigger database if needed for very large simulations. SQLite also allows safe rollbacks during testing.

6.1.6.3 Optional Resources

Cloud-based virtual machines (like AWS, Azure, or GCP) can be used for large-scale testing of multi-league simulations, helping to optimise performance. Git version control is used for safe, collaborative development. Automated testing is handled by tools like GitHub Actions or GitLab CI. Since all content is generated by the database management system, there is no need for licensed assets. Optional plugins can improve telemetry displays for advanced analysis.

6.1.6.4 Summary

Together, these resources provide all the computing power, software, and support needed to run a persistent, scalable motorsport management simulator, without needing expensive enterprise solutions.

6.1.7 Project Plan

6.1.7.1 Project Overview

Momentum will be developed in clear, manageable phases, using agile methods. Each phase builds on the last, with milestones to track progress. The plan ensures that the core systems are working before expanding the database management system. Regular feedback and testing help refine the algorithms and keep the simulation balanced and realistic.

6.1.7.2 Requirements and Design

The first phase is about defining what the project will do, how it will be structured, and how the database management system will work. The main goals are to have realistic driver progression, dynamic race results, and persistent multi-league play, with all teams, drivers, tracks, and championships created and managed by the database system. The database will be created before the game and packaged with it, so that every player has the same starting point. Non-functional goals include making sure the system is efficient, scalable, and easy to maintain. Design documents, diagrams, and flowcharts will be created to clarify and organise the project plan. This phase also sets performance targets and database designs and defines how the Python components will communicate.

6.1.7.3 Core Simulation Engine Development

The next phase is building the simulation engine, which is the core of the project. This includes setting up league structures, how drivers improve, how finances work, and how races are decided. Driver skills, fatigue, and personality are modelled using probability-based systems. Team finances, including budgets and sponsorships, are also handled with realistic randomness. Race results are calculated using Monte Carlo simulations, considering all the important factors. Data is managed efficiently to keep the game running smoothly, even with many leagues and seasons.

6.1.7.4 Database Management

Once the simulation engine is working, the focus shifts to the database management system. This part is responsible for creating and storing all teams, drivers, tracks, and championship schedules. It uses algorithms to make sure everything is diverse and balanced, with different locations and themes. Before anything is saved, the system checks that the league will be fair and interesting. This phase also adds tools for batch creation and automatic checking of content, making sure every playthrough is unique and replayable.

6.1.7.5 Front-End Integration

The Python front end is connected to the backend to show race data, leaderboards, team dashboards, and driver stats. Updates are sent smoothly to keep the interface responsive. The user interface is tested for clarity and ease of use. Integration ensures that all changes from the simulation and database management systems are shown accurately in the game.

6.1.7.6 Iterative Testing and Refinement

Testing happens in cycles, with milestones for things like driver development, financial stability, AI variety, realistic race results, and multi-league play. Feedback from each cycle is used to improve the algorithms and keep the simulation balanced. The system is monitored to make

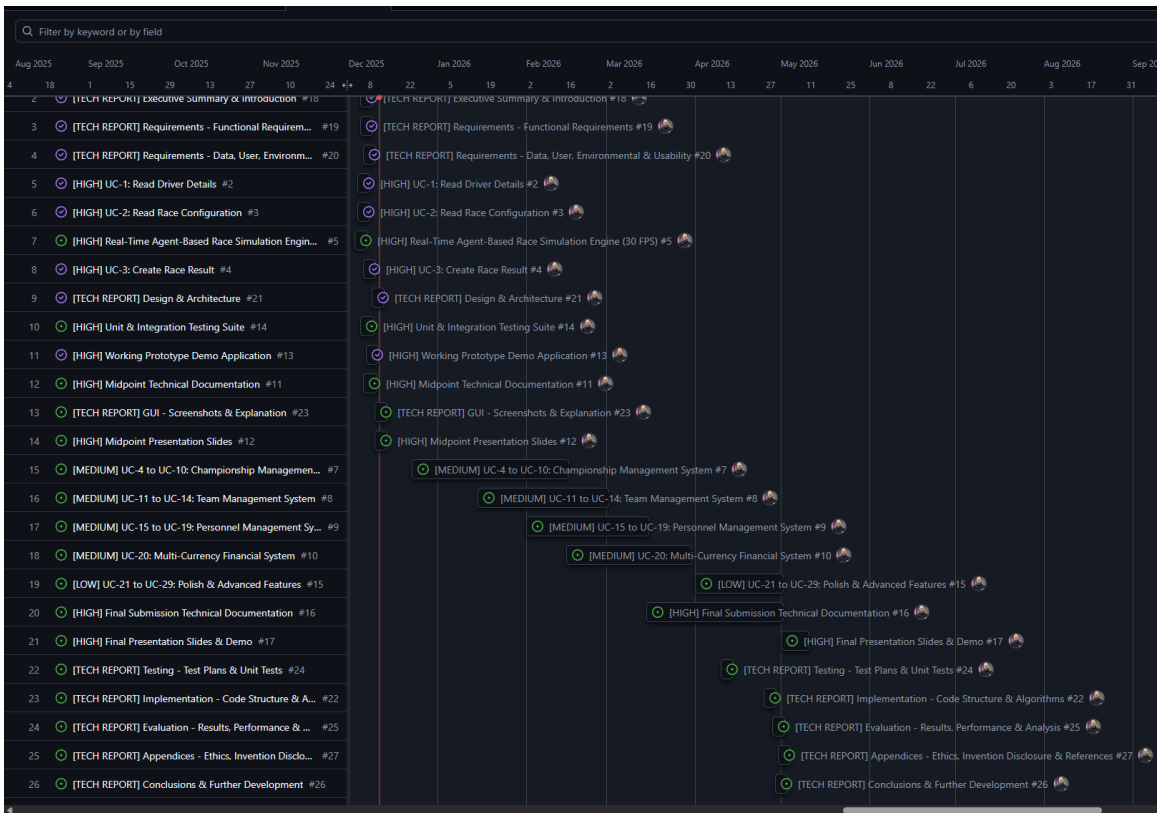
sure rivalries, race outcomes, and league progression all feel realistic. Performance is checked and improved as needed to keep everything running smoothly.

6.1.7.7 Final Evaluation and Documentation

The final phase reviews how the system behaves, the user experience, stability, and overall performance. All code is documented, and detailed reports are created on simulation accuracy and database management. This phase makes sure that Momentum works as a complete, persistent, and scalable motorsport management simulator, meeting all the goals set out at the start.

The project plan is designed to move smoothly from design to deployment, with clear milestones and dependencies. Regular feedback and testing ensure that Momentum delivers a realistic and engaging experience, while staying easy to maintain and expand in the future.

6.1.7.8 Gantt Chart



6.1.8 Testing

6.1.8.1 Testing Strategy

Testing for Momentum is structured across multiple levels to ensure system stability, performance, and fidelity of simulation outcomes. Unit, integration, system, and user testing are all incorporated to validate both functional and non-functional requirements. Each level of testing is designed to produce measurable results and inform iterative refinement cycles throughout development. Ethical considerations, including data privacy, informed consent, and participant anonymity, are integrated into all user evaluation procedures.

6.1.8.2 Unit Testing

Unit testing validates individual modules of the simulation. Driver progression, financial systems, race outcome calculations, and AI decision-making routines are tested against expected behaviours and edge cases. Probabilistic models for driver skill development, fatigue, and personality traits are verified for correct statistical distribution and consistency. Financial operations are checked for accuracy, ensuring budgets, sponsorships, and costs are correctly computed under stochastic conditions. Race outcome algorithms, including Monte Carlo simulations, are validated to maintain realistic variability while ensuring balance between drivers and teams.

6.1.8.3 Integration Testing

Integration testing ensures proper communication between the Python back-end, front-end, AI modules, and procedural content generation systems. JSON-based data exchanges are verified for accuracy and latency, and SQLite storage operations are tested for integrity across multi-season and multi-league scenarios. Metrics include data consistency, transaction success rates, and persistence reliability. Integration tests also verify that AI decisions, procedural generation outputs, and race telemetry are correctly reflected in the user interface, maintaining coherent emergent behaviours across the simulation.

6.1.8.4 System Testing

System-level testing evaluates Momentum under full operational conditions. Multi-league, multi-season simulations are run to assess emergent behaviour, computational performance, and scalability. Metrics include league variability, AI decision diversity, race outcome distribution, system load, and memory usage. Stress testing is performed to evaluate performance under high concurrency and extended simulation periods. Any bottlenecks in AI computation, procedural content generation, or database operations are identified and addressed to maintain real-time responsiveness and persistent state accuracy.

6.1.8.5 User Evaluation

User testing measures interface usability, engagement, and comprehension of the management systems. Participants interact with the full front-end, reviewing dashboards, race summaries, and team management tools. Quantitative metrics include task completion time, accuracy of actions, and interaction frequency. Qualitative metrics are gathered via structured surveys, focusing on perceived realism, clarity, and enjoyment. Ethical safeguards are enforced, including informed consent, voluntary participation, and secure handling of collected data. Feedback informs both interface improvements and algorithmic refinements to maintain realistic, emergent gameplay.

Overall, the testing framework ensures that Momentum achieves persistent, scalable, and believable motorsport simulation. Iterative results from unit, integration, system, and user testing directly guide algorithm adjustments, AI behaviour tuning, procedural content balancing, and front-end enhancements, guaranteeing a coherent, high-quality, and engaging management experience.

6.2 REFLECTIVE JOURNALS

Supervision & Reflection Template

Student Name	Eoin Fitzsimons
Student Number	X23151374
Course	Computing
Supervisor	

Month: September

What?

Came up with idea, a motorsport management simulator game, I found that all others in the sector are too shallow.

I have installed unity and activated the student plan.

I have downloaded notion to keep track of structure of the project.

I have decided to use a python backend with SQLite data storage and Unity front end

I use GitHub for version control

So What?

That alleviated the pressure of coming up and sifting through ideas, so that was a great success, but I am still at stage one essentially as I have nothing beyond an idea currently.

It's a direction to go in, but a large hill to tackle.

It gave me a structure to write my notes into

I have a structure in mind

I have backups if I don't like a change I made

Now What?

I need to come up with a proper name, a plan of attack, (which actually works as a name), and some basic branding that will improve the initial start in my opinion.

Need to figure out how to learn unity

Need to plan out my notes on the project structure.

Need to learn the languages

Plan branch paths and naming schemes

Student Signature

Supervision & Reflection Template

Student Name	Eoin Fitzsimons
Student Number	X23151374
Course	Computing
Supervisor	Emer Thornbury

Month: October

What?

I started building a Python prototype for my game. I began work on the team management, car development, financial management, and the race weekend structure and connected them through a basic Python GUI. The current version is only a prototype. None of the systems are fully finished, and the race simulation is not yet implemented. The main goal was to test ideas and system integration before moving to a more advanced platform.

So What?

The prototype allowed me to experiment with the game's logic and data flow, and to identify key challenges early. While the Python GUI is functional enough for testing, it is not intended for the final product. My main success was establishing a working foundation for the game's systems. The biggest challenge is that the project is still at an early stage, with many features incomplete or missing.

Now What?

Continue developing and refining the core systems in Python until the logic is solid.

Use the Python prototype as a reference and testing ground for gameplay mechanics.

Focus on implementing a real race simulation and improving system completeness before porting to Unity.

Student Signature	Eoin Fitzsimons

Supervision & Reflection Template

Student Name	Eoin
Student Number	X23151374
Course	Computing
Supervisor	

Month: November

What? Reflect on what has happened in your project this month? took a break to work on other assignments proposal, CRUD use cases, and feature priority mostly planning stuff
So What? Consider what that meant for your project progress. What were your successes? What challenges still remain? Certainly helped give a direction with use cases and priorities, realised initial proposal was just plain stupid, overly ambitious, and unrealistic for a college project.
Now What? What can you do to address outstanding challenges? Get started on coding around use cases

Student Signature	

Supervision & Reflection Template

Student Name	Eoin
Student Number	X23151374
Course	Computing
Supervisor	

Month: December

What? Reflect on what has happened in your project this month? Built the real-time agent engine and connected it to the GUI. Fixed a set of code warnings and cleaned up some function signatures. Ran tests (quick checks and longer validation).
So What? Consider what that meant for your project progress. What were your successes? What challenges still remain? The project now simulates live racing cars move continuously, and agents make per-frame decisions attack/defend/manage This change gives visible, behaviour for demos, but some docs still describe the old Monte Carlo approach. also found balance issues too many DNFs in long races that need tuning.
Now What? What can you do to address outstanding challenges? Rewrite the technical report and proposal to describe the agent-based approach. Tune incident parameters to reduce DNFs Run GUI tests and import track data for the midpoint demo.

Student Signature	

6.3 OTHER MATERIALS USED

Any other reference material used in the project for example evaluation surveys etc.